

KIERUNEK: ELEKTRONIKA

PRACA DYPLOMOWA
MAGISTERSKA

Opracowanie systemu diagnostyki akustycznej
maszyn przemysłowych z wykorzystaniem
zaawansowanych technik analizy dźwięku
i sztucznej inteligencji

Development of an acoustic diagnostic system for industrial
machinery using advanced sound analysis techniques
and artificial intelligence

AUTOR:

Damian Jarczok

PROMOTOR:

dr inż. Piotr Staroniewicz

Składam serdeczne wyrazy podziękowania

Panu dr inż. Piotrowi Staroniewiczowi

za poświęcony czas.

Za wspieranie mnie przy tworzeniu projektu
dziękuję znajomym oraz rodzinie.

Spis treści

1. Wstęp.....	4
1.1 Uzasadnienie wyboru tematu	4
1.2 Cel i zakres pracy	7
2. Wprowadzenie do diagnostyki akustycznej.....	8
2.1 Diagnostyka stanu technicznego maszyn – metody i znaczenie	8
2.2 Akustyczne symptomy uszkodzeń maszyn	10
2.3 Przegląd istniejących rozwiązań w diagnostyce akustycznej.....	13
3. Techniki analizy sygnałów dźwiękowych.....	15
3.1 Cyfrowe przetwarzanie dźwięku – podstawy	15
3.2 Ekstrakcja cech akustycznych	17
3.2.1 Analiza widmowa i spektrogramy.....	17
3.2.2 Współczynniki cepstralne mel (MFCC) i ich zastosowanie.....	19
3.2.3 Poziomy dźwięku L_{Aeq} i L_{Apeak}	22
4. Sztuczna inteligencja w diagnostyce akustycznej.....	24
4.1 Uczenie maszynowe w analizie sygnałów audio.....	24
4.1.1 Klasyczne metody uczenia maszynowego	25
4.1.2 Głębokie sieci neuronowe i ich zastosowania	27
4.2 Konwolucyjne sieci neuronowe (CNN)	30
4.2.1 Architektura i zasada działania CNN	30
4.2.2 Zastosowanie CNN w diagnostyce audio.....	32
5. Projekt i implementacja systemu.....	34
5.1 Wymagania systemu.....	34
5.1.1 Wymagania funkcjonalne.....	35
5.1.2 Wymagania нефункционалне.....	38
5.2 Architektura systemu diagnostycznego	39
5.2.1 Architektura funkcjonalna	40

5.2.2	Architektura techniczna i komponentowa	42
5.2.3	Technologie, biblioteki i narzędzia.....	45
5.2.4	Model danych i struktura bazy danych	47
5.3	Moduł akwizycji i przetwarzania danych akustycznych	49
5.3.1	Akwizycja danych z mikrofonu i plików	50
5.3.2	Obsługa formatów danych	51
5.3.3	Ekstrakcja cech sygnału	52
5.3.4	Normalizacja i augmentacja danych	54
5.4	Moduł sztucznej inteligencji	56
5.4.1	Projekt architektury modelu CNN	56
5.4.2	Proces treningu, walidacji i testu	58
5.4.3	Dobór parametrów	61
5.4.4	Wybór i implementacja metryk	63
5.5	Moduł monitorowania w czasie rzeczywistym.....	67
5.5.1	Obsługa sesji i strumieniowania danych.....	67
5.5.2	Analiza danych pomiarowych i klasyfikacja	69
5.5.3	System powiadomień i alertów	72
5.6	Interfejs użytkownika i konfiguracja systemu	75
5.6.1	Panel użytkownika i jego funkcje	76
5.6.2	Formularz konfiguracji diagnostyki.....	84
6.	Eksperymenty i wyniki.....	88
6.1	Metodyka eksperymentów i konfiguracja bazowa	88
6.1.1	Cel i zakres eksperymentów	88
6.1.2	Układ pomiarowy i środowisko testowe.....	89
6.1.3	Konfiguracja bazowa i punkt odniesienia.....	93
6.2	Analiza wpływu parametrów systemu diagnostycznego	96
6.2.2	Parametry treningu.....	99

6.2.3	Architektura modelu	104
6.2.4	Parametry ekstrakcji cech.....	115
6.2.5	Augmentacja danych	121
6.2.6	Dane wejściowe i podział zbioru.....	128
6.2.7	Dobór optymalnej konfiguracji parametrów modelu	131
6.3	Testy działania systemu w środowisku rzeczywistym	133
6.3.1	Konfiguracja monitoringu	133
6.3.2	Skuteczność klasyfikacji w warunkach rzeczywistych	135
6.3.3	Dobór optymalnej konfiguracji parametrów monitoringu	138
6.4	Podsumowanie i wnioski z eksperymentów	139
7.	Podsumowanie i perspektywy rozwoju	146
7.1	Ograniczenia obecnego rozwiązania	146
7.2	Możliwości wdrożenia systemu w przemyśle	148
7.3	Kierunki dalszego rozwoju i badań nad systemem	149
7.4	Uwagi końcowe	150
	Spis źródeł literaturowych	152
	Spis rysunków.....	154
	Spis tabel.....	155

1. Wstęp

1.1 Uzasadnienie wyboru tematu

Zapewnienie niezawodnej pracy maszyn i urządzeń technicznych wiąże się z koniecznością stałego monitorowania ich pracy. Nieplanowane awarie często prowadzą do kosztownych przestojów i strat produkcyjnych, dlatego wczesne wykrywanie usterek stanowi istotny element utrzymania ciągłości pracy w nowoczesnym przemyśle. Tradycyjne metody utrzymywania urządzeń, takie jak reakcyjne usuwanie awarii czy zaplanowane przeglądy prewencyjne nie zawsze są skuteczne ze względu na nieodpowiedni czas reakcji czy wysokie koszty takich działań. W odpowiedzi na te aspekty upowszechniła się koncepcja utrzymania predykcyjnego (ang. *predictive maintenance*), które opiera się na zasadzie nieprzerwanej diagnostyki urządzenia oraz ingerencji w nie w momencie, gdy działania serwisowe będą faktycznie potrzebne. Opracowane zostały międzynarodowe standardy, takie jak ISO 17359 czy ISO 13374 podkreślające konieczność stałego monitorowania kondycji urządzeń oraz analizy danych pomiarowych w celu wczesnego wykrywania symptomów uszkodzeń. W kontekście takiego rodzaju prewencji nieustannie rozwijane są inteligentne systemy diagnostyczne, które integrują sieci odpowiednich czujników i analizę sygnałów z wykorzystaniem sztucznej inteligencji. Przykładem takich rozwiązań są nowoczesne systemy typu IIoT (ang. *Industrial Internet of Things*) umożliwiające zbieranie danych o pracy maszyn w czasie rzeczywistym i wykrywanie anomalii za pomocą analizy sygnałów takich jak wibracje czy hałas, wykorzystując przy tym uczenie maszynowe. Badania pokazują, że takie podejście znacząco zwiększa skuteczność wykrywania usterek oraz pozwala tym samym ograniczyć zarówno nieplanowane przestoje i koszty związane z serwisem. Dane akustyczne i wibracje są powszechnie uznawane za skuteczne wskaźniki pozwalające określić stan techniczny maszyny. Zmiany w drganiach lub dźwięku często wskazują na zużycie elementów, niewłaściwe ustawienie, niezrównoważenie lub inne usterki, które mogą negatywnie wpłynąć na wydajność urządzeń [1].

Urządzenia podczas pracy emitują charakterystyczne dźwięki, które po przeanalizowaniu mogą dostarczyć cennych informacji na temat ich stanu

technicznego. Doświadczony operator często jest w stanie już po samym dźwięku rozpoznać nietypową pracę maszyny i powiązać ją z określonym typem uszkodzenia. Taka subiektywna ocena oparta na zmyśle słuchu może być jednak zawodna i silnie zależna od doświadczenia człowieka oraz chociażby warunków otoczenia. Pojawia się zatem potrzeba zautomatyzowania tego procesu przy użyciu metod obliczeniowych i pomiarowych, aby uzyskać obiektywny i powtarzalny wynik diagnostyczny. Jedną z głównych zalet diagnostyki akustycznej jest fakt, że mikrofony umożliwiają rejestrowanie sygnałów bez fizycznego kontaktu z daną maszyną, a także ze stosunkowo dużej odległości, co pozwala prowadzić monitoring całkowicie bez zakłócania pracy urządzenia. Dodatkowo, monitoring akustyczny może odbywać się podczas normalnej pracy urządzenia, a zatem nie wymaga jego zatrzymania nawet na moment, co jest szczególnie istotne w przypadku np. dużych fabryk. Kolejnym atutem takiego rozwiązania jest stosunkowo niska cena i prostota oprzyrządowania. Typowy przetwornik elektroakustyczny (np. mikrofon, hydrofon) jest mniej kosztowny w zakupie, utrzymaniu czy instalacji niż czujnik drgań. Z tego względu techniki diagnostyki akustycznej są atrakcyjnym uzupełnieniem klasycznych metod diagnostycznych i znajdują coraz szersze zastosowania w różnych gałęziach przemysłu.

Należy jednak zaznaczyć, że wykorzystanie sygnałów akustycznych do diagnostyki maszyn wiąże się również z szeregiem wyzwań. Dźwięk wydobywający się z urządzenia znajdującego się w wielkopowierzchniowej hali produkcyjnej może cechować się dużym udziałem pogłosu i innych niepożądanych zakłóceń wydobywających się z innych sąsiadujących urządzeń. To sprawia, że diagnozowanie lub identyfikacja usterek w rzeczywistych warunkach przemysłowych za pomocą sygnałów dźwiękowych jest zadaniem trudnym i wymaga zastosowania metod analizy sygnałów do konkretnego przypadku. Tradycyjne algorytmy detekcji oparte na stałych progach również zawodzą przy bardziej złożonych uszkodzeniach, które nie powodują jednoznacznych zmian pojedynczego parametru sygnału. W obliczu tych wyzwań konieczne staje się wykorzystanie algorytmów bardziej inteligentnych, zdolnych do wychwytywania złożonych wzorców w danych akustycznych. Aktualnie prowadzone są intensywne prace badawcze nad doбором odpowiednich cech akustycznych oraz metod klasyfikacji i grupowania, które pozwolą na skuteczną i automatyczną ocenę stanu maszyn przemysłowych na podstawie dźwięku [2].

Literatura wykazuje, że podejścia oparte na głębokim uczeniu (ang. *deep learning*) mają przewagę nad klasycznymi technikami analizy sygnałów, a modele oparte na sieciach neuronowych są w stanie wydobywać z sygnału złożone wzorce i nieliniowe zależności, dzięki czemu osiągają wysoką skuteczność w detekcji nieprawidłowości nawet skomplikowanych układów mechanicznych. W przypadku diagnostyki akustycznej uczenie maszynowe potrafi przewyższyć metody oparte na analizie widma czy amplitudy sygnału, zwłaszcza przy wykrywaniu wczesnych, subtelnych oznak uszkodzeń, które dla klasycznych metod pozostają niewykrywalne [3].

Rosnąca popularność sztucznej inteligencji w diagnostyce technicznej wynika również ze zdolności adaptacji algorytmów do danych i uczenia się na ich podstawie. Mogą one zostać wytrenowane na historycznych nagraniach dźwiękowych awarii, aby móc następnie rozpoznawać podobne wzorce z nowych danych od pracującej maszyny. W przeciwieństwie do tradycyjnych metod, podejścia oparte na sztucznej inteligencji oferują wyższą dokładność i zdolność adaptacji, co czyni je idealnymi do zastosowań w czasie rzeczywistym w środowiskach przemysłowych [1]. Wyzwanie stanowi natomiast dostępność odpowiednio rozbudowanych oraz reprezentatywnych zbiorów nagrań do treningu modelu z uwagi na stosunkowo rzadkie pojawianie się poważnych uszkodzeń, a ponadto każda z maszyn może generować inne widmo akustyczne – nawet dla podobnej lub identycznej usterki. Wskutek tego modele głębokiego uczenia w diagnostyce dźwiękowej często borykają się z problemem niewystarczającej ilości danych treningowych. Jednym z aktualnych trendów badawczych mających zaradzić tym ograniczeniom jest zastosowanie metod augmentacji danych, czyli sztucznego rozszerzania zbioru treningowego o dodatkowe próbki wygenerowane na bazie istniejących nagrań (np. poprzez dodanie szumu, zmianę wysokości dźwięku, miksowanie sygnałów, transformacje spektrogramów itp.). Jak wykazano w literaturze, potrzeba stosowania technik augmentacji danych jest niepodważalna, ponieważ podejście to stało się potężnym narzędziem do generowania syntetycznych zbiorów danych (obrazów, dźwięków, tekstów itp.) i znacząco przyczyniło się do poprawy wydajności modeli głębokiego uczenia. Innymi słowy, zastosowanie odpowiednich metod augmentacji pozwala znacząco poprawić uogólnianie modeli akustycznych i zwiększyć ich skuteczność w przypadku niewielkiej ilości danych treningowych [4].

Biorąc pod uwagę przytoczone powyżej korzyści oraz wyzwania, opracowanie funkcjonalnego systemu diagnostyki maszyn przemysłowych wiąże się z dużym znaczeniem zarówno praktycznym jak i naukowym. Wymaga ono wielodyscyplinarnego podejścia, łączącego wiedzę z zakresu akustyki i przetwarzania sygnałów oraz wykorzystania technik sztucznej inteligencji. Można obserwować stale rosnącą liczbę publikacji literaturowych poświęconych tej tematyce, co potwierdza jednocześnie aktualność i istotność problemu. Pojawiły się już prace opisujące zastosowania diagnostyki audio do takich celów jak monitorowanie m.in. silników elektrycznych, łożysk tocznych, pomp, przekładni czy pojazdów autonomicznych. Dostępne są również opracowania przeglądowe, które podsumowują dotychczasowe osiągnięcia w tej dziedzinie i obejmują analizy porównawcze algorytmów uczenia maszynowego, metod akustycznej detekcji uszkodzeń czy integrowania ze sobą różnych źródeł danych (tj. akustyczne, wibracyjne, prądowe) w ramach systemów diagnostyki dla Przemysłu 4.0. To właśnie ta obszerna baza informacji stanowi fundament dla niniejszej pracy i uzasadnia wybór tematu – opracowanie systemu diagnostyki akustycznej maszyn przemysłowych z wykorzystaniem zaawansowanych technik analizy dźwięku i sztucznej inteligencji. Odpowiada on na aktualne potrzeby przemysłu, wpisuje się w światowe trendy w diagnostyce maszyn oraz stwarza możliwość naukowego wkładu poprzez zaproponowanie nowych rozwiązań w tym obszarze.

1.2 Cel i zakres pracy

Zasadniczym celem pracy było opracowanie prototypu systemu diagnostyki akustycznej maszyn przemysłowych, który za pomocą algorytmów sztucznej inteligencji będzie analizował dźwięk i automatycznie rozpoznawał stany awaryjne. System ma na celu umożliwiać wczesne wykrywanie nieprawidłowości w pracy urządzeń na podstawie sygnałów dźwiękowych, co ma docelowo pozwolić na zapobieganie generalnym awariom poprzez wdrożenie strategii utrzymania zorientowanego na bieżący stan, znane jako CBM (ang. *Condition-Based Maintenance*). W zakresie pracy uwzględniono zarówno studia teoretyczne jak i prace projektowo-eksperymentalne. Na podstawie przeglądu literatury z tej dziedziny sformułowano wymagania i założenia dla zaprojektowanego systemu. Kolejnym aspektem było opracowanie metod akwizycji i przetwarzania sygnałów akustycznych, w czym zawierał się wybór odpowiedniego oprzyrządowania, przygotowanie układu akwizycji dźwięku i określenie metodyki

pomiarów. Następnym etapem był dobór oraz implementacja algorytmów sztucznej inteligencji do interpretacji zebranych danych, a w szczególności modeli głębokiego uczenia maszynowego, które na podstawie cech sygnału dźwiękowego są w stanie klasyfikować stan maszyny. Ważną częścią pracy jest trening zaproponowanych modeli wykorzystujących dostępne lub zebrane dane dla różnych maszyn. W ramach pracy określono również kryteria oceny skuteczności systemu, takie jak zdolność do rozróżniania poszczególnych uszkodzeń, czułość ich detekcji, wartość błędów klasyfikacji itp.

Podsumowując, zakres pracy obejmował przeprowadzenie eksperymentów weryfikujących działanie systemu w warunkach zbliżonych do rzeczywistych oraz analizę otrzymanych wyników. Na jej podstawie sformułowano wnioski odnoszące się do przydatności zaproponowanych metod wraz z ich ograniczeniami oraz potencjalnymi kierunkami dalszych usprawnień.

2. Wprowadzenie do diagnostyki akustycznej

2.1 Diagnostyka stanu technicznego maszyn – metody i znaczenie

Diagnostyka techniczna to uporządkowany zestaw narzędzi i metod służących do oceny kondycji systemów technicznych, w tym identyfikacji źródeł ich zmian, sposobu rozwoju oraz potencjalnych skutków funkcjonalnych [5]. Ujmując to zagadnienie inaczej, jest to proces zbierania i analizowania informacji o urządzeniu w celu wykrycia nieprawidłowości, oceny stopnia zużycia podzespołów oraz przewidywania prawdopodobieństwa wystąpienia awarii. Zasadniczym celem stosowania metod diagnostycznych jest poprawa niezawodności maszyn poprzez zapobieganie poważnym uszkodzeniom, minimalizacja nieplanowanych przestojów oraz optymalizacja kosztów ich utrzymania. Są one kluczowe w wielu gałęziach przemysłu takich jak energetyka, produkcja czy transport.

Na przestrzeni lat wykształciły się różne metody diagnostyki stanu technicznego maszyn, a każda z nich skupia się na nieco innym typie sygnałów i objawów.

Do głównych należą m.in.:

- **Diagnostyka drganiowa** – analiza drgań mechanicznych generowanych przez badaną maszynę. Jest ona powszechnie stosowana do wykrywania niewyważenia, niewspółosiowości czy uszkodzeń w łożyskach.
- **Diagnostyka termograficzna** – ocena rozkładu temperatur przeprowadzana np. za pomocą kamer termowizyjnych w celu zidentyfikowania przegrzewających się komponentów, które świadczą o tarcu lub przeciążeniu.
- **Diagnostyka olejowa (tribologiczna)** – badanie próbek oleju smarowego pod kątem obecności zanieczyszczeń, takich jak cząstki metalu czy wody, które pozwala wnioskować o zużyciu elementów oraz stanie układu smarowania.
- **Diagnostyka wizyjna** – inspekcja wzrokowa lub z wykorzystaniem kamer (np. endoskopowych) w celu wykrycia potencjalnych nieprawidłowości takich jak pęknięcia, zużycia, nieszczelności itp.
- **Diagnostyka procesowa** – monitorowanie parametrów procesu technologicznego (np. przepływu, ciśnienia) mająca na celu pośrednią ocenę stanu urządzenia.
- **Diagnostyka elektryczna** – pomiary parametrów elektrycznych, głównie prądu i napięcia. Jest szczególnie istotna dla urządzeń elektroenergetycznych i napędów.
- **Diagnostyka akustyczna** – ocena emisji dźwiękowej maszyny za pomocą parametrów takich jak poziom hałasu lub charakterystyki sygnału akustycznego. Przydatna zwłaszcza do wykrywania usterek w układach napędowych i generujących znaczny poziom dźwięku.

Każda z wymienionych powyżej metod dostarcza specyficznych informacji o stanie technicznym maszyny. Często praktyką jest stosowanie podejścia zintegrowanego, łączącego dane z wielu źródeł (np. wibracje, dźwięk, temperatura) w celu uzyskania pełniejszego obrazu kondycji urządzenia. Autorzy literatury podkreślają, że sygnały wibroakustyczne zawierają bogate informacje o stanie dynamicznym maszyny, ponieważ każdej zmianie stanu technicznego urządzenia towarzyszy zmiana charakterystyk generowanych przez niego sygnałów drganiowych

i akustycznych. Sygnały te są uznawane powszechnie za skuteczne wskaźniki kondycji mechanicznej maszyn, dlatego diagnostyka wibroakustyczna stanowi trzon monitorowania stanu wielu urządzeń obecnych w przemyśle [6][7]. W przypadku instalacji u ruchu ciągłym, takich jak elektrownie czy zakłady produkcyjne, gdzie nagła usterka maszyny może prowadzić do wysoce odczuwalnych skutków finansowych i środowiskowych, ogromne znaczenie ma diagnostyka techniczna eksploatacyjna używanej przez człowieka infrastruktury technicznej. Umożliwia ona identyfikację pierwszych oznak degradacji technicznej, zanim dojdzie do ich eskalacji prowadzącej do potencjalnej awarii [8]. Jak zauważają Kowalski i Ewert, poważne awarie maszyn zakłócają ciągłość procesów technologicznych, a w skrajnych przypadkach skutkują całkowitym zatrzymaniem instalacji, co przekłada się na wzrost kosztów produkcyjnych. W związku z tym istotne jest wdrażanie metod umożliwiających szybkie wykrywanie usterek oraz skuteczne przekazywanie informacji obsłudze technicznej [6].

Należy podkreślić, że stale rosnące wymagania odnośnie niezawodności oraz bezpieczeństwa eksploatacji sprawiają, że zapotrzebowanie na metody skutecznej diagnostyki technicznej systematycznie rośnie. Równolegle z nią rozwijają się standardy i normy inżynierskie porządkujące wiedzę z tego zakresu. Za przykład można tutaj podać normę ISO 17359 zawierającą ogólne wytyczne przeprowadzania monitorowania i diagnostyki urządzeń przemysłowych, natomiast z kolei ISO 22096 określa zasady stosowania metody emisji akustycznej w diagnostyce maszyn. Istnienie takich norm ułatwia tworzenie ustandaryzowanych systemów do monitorowania stanu technicznego umożliwiając porównywalność wyników oraz definiując kwalifikacje personelu diagnostycznego (np. certyfikowanego wg. ISO 18436 z zakresu drgań i emisji akustycznych) [9].

2.2 Akustyczne symptomy uszkodzeń maszyn

Każda maszyna podczas pracy generuje charakterystyczne dla niej sygnały akustyczne.

Jej stan techniczny w istotny sposób wpływa na te emisje dźwiękowe, które można podzielić na kilka podstawowych rodzajów, różniących się charakterystyką i źródłem powstawania:

- **Hałas tonalny** – wyraźne tony o określonych częstotliwościach, powstające na przykład wskutek drgań elementów mechanicznych lub pracy silników elektrycznych.
- **Hałas impulsowy** – krótkotrwałe, gwałtowne dźwięki (trwające zwykle poniżej 1 sekundy), takie jak uderzenia czy stuknięcia, charakteryzujące się szybkim wzrostem i opadaniem poziomu dźwięku.
- **Hałas szumowy** – dźwięk o szerokim spektrum częstotliwości, typowy dla przepływu powietrza lub gazów wokół elementów maszyny, np. wentylatorów czy sprężarek.
- **Hałas ustalony** – hałas o względnie stałym poziomie natężenia, zmieniającym się mniej niż 5 *dB* w czasie.
- **Hałas nieustalony** – hałas o zmiennym poziomie natężenia, którego wartość zmienia się w czasie o więcej niż 5 *dB*.
- **Hałas infradźwiękowy** – dźwięki o częstotliwościach poniżej 20 *Hz*, niesłyszalne dla ucha ludzkiego, emitowane przez maszyny przepływowe.
- **Hałas ultradźwiękowy** – dźwięki o częstotliwościach powyżej 10 *kHz*, powstające np. w zgrzewarkach czy narzędziach pneumatycznych.
- **Hałas mechaniczny** – powstający w wyniku tarcia, uderzeń i drgań elementów mechanicznych maszyny.
- **Hałas aerodynamiczny** – generowany przez przepływ powietrza lub gazów wokół elementów maszyny.
- **Hałas technologiczny** – związany najczęściej z procesami obróbki materiałów, takimi jak skrawanie czy przecinanie.
- **Hałas elektryczny (magnetyczny)** – powstający w maszynach elektrycznych na skutek działania pola magnetycznego [2].

Do typowych symptomów akustycznych awarii zalicza się m.in. wzrost ogólnego poziomu hałasu, pojawienie się nowych, nietypowych dźwięków lub zmiana rytmu pracy maszyny. Przykładowo, uszkodzenia łożysk tocznych często objawiają się narastającym

szumem i drżeniem – pojawia się charakterystyczne dudnienie lub chrobot spowodowany przemieszczaniem się kulek po defekcie bieżni. Defekty przekładni zębatych mogą wywoływać stukot lub świst o częstotliwości powiązanej z obrotem zębatek (np. wskutek wyszczerbienia zębów). Luźne elementy konstrukcyjne generują wibracje i drgania słyszalne jako grzechot lub stuki przy określonych obrotach. W maszynach przepływowych, takich jak pompy czy turbiny, hałas może być powodowany niestabilnością przepływu lub kawitacją, co słyszalne jest jako szum o dużej częstotliwości, czasem z dodatkowymi wysokoczęstotliwościowymi tonami (gwizdy, piski). Takie zmiany akustyczne sygnalizują pogorszenie kondycji elementów maszyny często zanim jeszcze obniży się wydajność procesu lub nieprawidłowość zostanie wykryta przez inne czujniki.

Szczególnym rodzajem sygnałów akustycznych będącym wykorzystywanym w diagnostyce jest emisja akustyczna (EA), czyli fale sprężyste o dużej częstotliwości generowane bezpośrednio w materiale konstrukcji na skutek dynamicznych zjawisk fizycznych, takich jak powstawanie pęknięć, tarcie czy odkształcenia. Czujniki emisji akustycznej rejestrują krótkotrwałe impulsy energii sprężystej, których analiza umożliwia wykrywanie mikropęknięć czy innych uszkodzeń już na bardzo wczesnym etapie. Metoda ta jest bardzo czuła, ponieważ potrafi wykryć zjawiska zachodzące wewnątrz elementów zanim staną się wyraźnie słyszalne lub widoczne gołym okiem. Z tego względu sprawdzają się one przy monitorowaniu pękniętych zbiorników ciśnieniowych czy konstrukcji lotniczych. Międzynarodowe normy, takie jak wspomniana już poprzednio ISO 22096, szczegółowo opisują zasady aplikacji EA w monitorowaniu maszyn [9].

Diagnostyka akustyczna, bazująca na analizie sygnałów dźwiękowych generowanych przez maszyny, znajduje coraz szersze zastosowanie w warunkach przemysłowych. Jej wartość wynika przede wszystkim z możliwości prowadzenia obserwacji w sposób ciągły i nieinwazyjny, czyniąc ją szczególnie użyteczną w kontekście monitorowania maszyn pracujących w trybie ciągłym lub trudno dostępnych.

2.3 Przegląd istniejących rozwiązań w diagnostyce akustycznej

Akustyczna diagnostyka maszyn znalazła zastosowanie zarówno w tradycyjnych systemach monitorowania jak i nowoczesnych i inteligentnych rozwiązaniach opartych na elektronice i algorytmach sztucznej inteligencji. Klasyczne podejście do tego tematu polegało na prostej analizie sygnału dźwiękowego przez człowieka lub za pomocą podstawowych narzędzi takich jak analizatory widma. W zakładach przemysłowych przez wiele dekad praktykowany był tzw. monitoring operatora polegający na nasłuchiwanie przez doświadczonego mechanika pracy urządzeń podczas obchodów, a następnie bieżącej oceny tego czy dźwięk odbiega od normy. Wspomagano się też prostymi przyrządami, takimi jak stetoskopy mechaniczne czy mierniki poziomu dźwięku, które pozwalały ocenić obecność nowych tonów lub wzrosty pasm hałasu mogące świadczyć o zużyciu elementów. Zasadniczą rolę odgrywała wiedza ekspercka pozwalająca powiązać dany symptom akustyczny z konkretnym uszkodzeniem. Pierwsze zautomatyzowane systemy monitoringu skupiały się głównie na sygnałach wibracyjnych, natomiast notowano również próby wykorzystania mikrofonów do detekcji anomalii akustycznych. Typowy system diagnostyczny działał offline, a personel wykonywał pomiary okresowo i poddawał je analizie *ad hoc*. Analiza za pomocą transformaty Fouriera była podstawowym narzędziem – na jej podstawie identyfikowano obecność charakterystycznych dla uszkodzeń częstotliwości. Diagnostyka tego typu bywa czasochłonna i ściśle zależna od kompetencji diagnosty, co ogranicza jej skuteczność w niektórych przypadkach.

W ostatnich latach nastąpił dynamiczny rozwój nowoczesnych systemów diagnostyki akustycznej, na co złożyły się m.in. czynniki takie jak ogólny postęp w elektronice, upowszechnienie Internetu Rzeczy (IoT) oraz znakomity skok w dziedzinie algorytmów sztucznej inteligencji i informatyki. W naukowej literaturze pojawia się wiele doniesień o skuteczności systemów sztucznej inteligencji w diagnozowaniu maszyn na podstawie dźwięku. W przeglądzie 68 prac poświęconych tej tematyce przeprowadzonym przez Shaikh *et al.* z 2021 roku wykazano, że diagnostyka akustyczna wykorzystująca sztuczną inteligencję staje się coraz bardziej dojrzałą i wszechstronną dziedziną. Zostały w niej zidentyfikowane najczęściej stosowane etapy przetwarzania: zbieranie sygnałów audio, wybór cech, ekstrakcja cech oraz klasyfikacja z użyciem różnych technik. Analizowane systemy były stosowane do

diagnozy łożysk, pomp i przekładni. W większości przypadków osiągnano wysoką skuteczność sięgającą nawet powyżej 95% dokładności. W pracy wskazano szczególne znaczenie transformacji czasowo-częstotliwościowej oraz technik głębokiego uczenia. Zwrócono również uwagę, że połączenie sygnałów akustycznych z innymi modalnościami takimi jak drgania, istotnie zwiększa niezawodność diagnozy [2].

Wiele praktycznych rozwiązań diagnostyki akustycznej jest obecnie na etapie wdrażania pilotażowego lub dostępnych wyłącznie jako specjalistyczne produkty. Przykładowo, zostały opracowane systemy monitorowania hałasu maszyn w czasie rzeczywistym – montowane na hali produkcyjnej sieci mikrofonów mogą stale nasłuchiwać określone urządzenia, a dołączone do nich oprogramowanie analizuje wzorce dźwiękowe. W momencie, gdy wykryty zostanie sygnał wskazujący na potencjalną usterkę, system automatycznie generuje alarm dla obsługi. Dostępne są również tzw. kamery akustyczne, które łączą obraz oraz detekcję dźwięku umożliwiając lokalizację źródła hałasu na obrazie maszyny, co znacząco ułatwia zlokalizowanie wadliwego komponentu. Urządzenia te są wykorzystywane m.in. do badania maszyn wirujących lub wykrywania nieszczelności w postaci uleciającego się gazu lub cieczy ujawniającego się jako sygnał dźwiękowy, który kamera rejestruje. Innym przykładem są systemy diagnostyki silników spalinowych bazujące na analizie akustycznej, czyli czujniki spalania stukowego wykrywają nieprawidłowe spalanie w cylindrach poprzez analizę fal ciśnienia (dźwięku) i sygnalizują konieczność regulacji pracy silnika zanim dojdzie do uszkodzeń mechanicznych.

Najnowsze osiągnięcia w rzeczonyj dziedzinie skupiają się na wykorzystaniu sztucznej inteligencji i uczenia maszynowego do automatycznej analizy dźwięku. Zasadniczo, dąży się do tego, by rola człowieka sprowadzała się jedynie do nadzoru nad systemem, zamiast ręcznej interpretacji sygnałów. Tendencję tę zauważono już wcześniej, a obecnie poszukuje się rozwiązań, które pomogą zminimalizować udział ludzkich diagnostów i uczynić proces w pełni automatycznym. Metody oparte na teorii sieci neuronowych są zdolne do realizacji tego zadania [6]. W ostatnich latach, głównie za sprawą znacznego wzrostu mocy obliczeniowej nowoczesnych komputerów, ta idea jest realizowana na szerszą skalę. Najnowsza literatura wskazuje liczne przykłady zastosowania głębokich sieci neuronowych do rozpoznawania dźwięków świadczących o awarii: od wykorzystania konwolucyjnych sieci CNN do klasyfikacji spektrogramów hałasu maszynowego, po modele autoenkoderów do detekcji anomalii akustycznych bez

potrzeby wcześniejszego scharakteryzowania uszkodzeń. Kolejnym takim przykładem jest praca Nasir i Banaras z początku 2025 roku, którzy zaproponowali architekturę IoT zintegrowaną ze sztuczną inteligencją do monitorowania maszyn, gdzie połączenie czujników systemu z analizą danych opartą na sztucznej inteligencji również znacząco poprawiło skuteczność wykrywania i klasyfikacji uszkodzeń. Jest to potwierdzeniem, że diagnostyka audio, wspierana nowoczesnymi technologiami, znacząco podnosi efektywność utrzymania ruchu, a jej rozwój stanowi naturalne przedłużenie ewolucji – od tradycyjnych, subiektywnych metod słuchowych do zaawansowanych, inteligentnych i zautomatyzowanych systemów predykcyjnych [7].

3. Techniki analizy sygnałów dźwiękowych

3.1 Cyfrowe przetwarzanie dźwięku – podstawy

Dźwięk jest sygnałem ciągłym w czasie, jednak w celu jego analizy komputerowej wymagane jest przekształcenie go do postaci dyskretniej. Proces ten obejmuje próbkowanie, czyli pobieranie wartości sygnału w danych odstępach czasu oraz kwantyzację, czyli zaokrąglenie tych wartości do skończonego zbioru poziomów, zależnego od rozdzielczości bitowej. Zgodnie z twierdzeniem o próbkowaniu Nyquista-Shannona, sygnał ciągły o ograniczonym paśmie częstotliwości do B można jednoznacznie zrekonstruować z próbek, jeśli częstotliwość próbkowania f_s jest co najmniej dwukrotnie większa od maksymalnej częstotliwości sygnału ($2B$) [10]. Wynika z powyższego warunek $f_s \geq 2B$, którego spełnienie zapobiega zjawisku aliasingu, podczas digitalizacji sygnału. W przypadku sygnałów słyszalnych typowe jest próbkowanie z częstotliwością $44,1\text{ kHz}$ lub 48 kHz . W zastosowaniach diagnostycznych częstotliwość próbkowania dobiera się adekwatnie do najwyższych istotnych częstotliwości emisji audio maszyny, np. dla maszyn wirujących o prędkościach rzędu kilku tysięcy obrotów na minutę pasmo takie może sięgać kilkunastu kHz , stąd wymagane jest próbkowanie rzędu dziesiątek kHz , aby zarejestrować wszystkie istotne składowe.

Po cyfryzacji sygnał audio reprezentowany jest za pomocą ciągu dyskretnych próbek, co umożliwia stosowanie algorytmów DSP (ang. *Digital Signal Processor*). Podstawową reprezentacją sygnału w dziedzinie czasu jest przebieg amplitudy

w dziedzinie czasu, jednakże często bardziej użyteczne jest przejście do dziedziny częstotliwości w celu wglądu do struktur widmowych. Wykorzystywana jest do tego celu transformacja Fouriera, a najczęściej wykorzystywaną jest DFT (ang. *Discrete Fourier Transform*), obliczaną efektywnie metodą FFT (ang. *Fast Fourier Transform*). Za jej pomocą możliwe jest przekształcenie sygnału $x[n]$, gdzie n oznacza numer próbki w zespolone widmo $X[k]$, opisujące zawartość częstotliwościową sygnału.

Matematycznie można to przedstawić wzorem:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1,$$

gdzie N oznacza liczbę próbek analizowanego segmentu sygnału. Widmo to można interpretować jako zbiór składowych sinusoidalnych o dyskretnych częstotliwościach. W praktycznych zastosowaniach analizę widmową realizuje się poprzez podział dłuższego sygnału na krótsze odcinki (ramki) i wyznaczenie widma dla każdej z nich, co pozwala założyć lokalną stacjonarność sygnału. FFT stanowi podstawowe narzędzie analizy częstotliwościowej sygnałów dźwiękowych. Widmo amplitudowe przedstawia rozkład energii sygnału na poszczególne częstotliwości, co umożliwia identyfikację dominujących harmonicznym związanych z takimi rzeczami jak obroty czy drgania elementów urządzenia.

Warto zwrócić uwagę, że w praktycznej realizacji transformacji Fouriera istotną rolę odgrywa okienkowanie sygnału. Transformata traktuje analizowany odcinek jak jeden okres sygnału okresowego, dlatego nagłe ucięcie ramki powoduje tzw. przeciek widma. Aby temu zapobiec, przed przystąpieniem do obliczeń próbki ramki są przemnażane przez okno (np. Hamminga, Gaussa, Bartletta itd.), które łagodnie wygasza sygnał na samych brzegach ramki. Powoduje to redukcję wpływu nieciągłości i poprawia rozdzielczość amplitudową otrzymanego widma, które może być dalej wykorzystane do detekcji cech charakterystycznych [11].

Samo przekształcenie sygnału ciągłego do postaci cyfrowej nie ogranicza się wyłącznie do próbkowania. Istotnym elementem procesu cyfryzacji jest także kwantyzacja, czyli przypisanie każdej próbce jednej z możliwych wartości reprezentowanych przy pomocy określonej liczby bitów. Wybór rozdzielczości bitowej wpływa bezpośrednio na precyzję odwzorowania amplitudy sygnału – im większa liczba bitów, tym niższy poziom szumu kwantyzacji i większy zakres dynamiczny.

Po kwantyzacji sygnał zostaje zapisany jako sekwencja wartości numerycznych, które mogą być poddane dalszej obróbce. W typowych formatach plików audio, takich jak WAV, dane te są zapisywane w sposób nieskompresowany, umożliwiając wierną analizę bez strat jakości. Taka forma reprezentacji jest punktem wyjścia dla wszystkich kolejnych etapów przetwarzania, w tym ekstrakcji cech i klasyfikacji.

3.2 Ekstrakcja cech akustycznych

Celem ekstrakcji cech akustycznych jest wydobywanie z surowego sygnału dźwiękowego takich parametrów, które najlepiej opiszą stan źródła dźwięku i będą mogły zostać wykorzystane do detekcji lub rozpoznawania określonych zjawisk. Surowy sygnał czasowy zawiera znaczną ilość informacji, jednak nie wszystkie z nich są przydatne. Właśnie dlatego dokonuje się transformacji sygnału do reprezentacji cech o mniejszej wymiarowości, ale większej informacyjności. W kontekście systemów diagnostycznych z elementami uczenia maszynowego etap ten nazywany jest inżynierią cech (ang. *feature engineering*) i ma istotne znaczenie dla skuteczności klasyfikatorów [2].

W niniejszym rozdziale skupiono się na wybranych, najistotniejszych technikach analizy sygnałów audio, do których należą: analiza widmowa wraz z jej rozszerzeniem w postaci spektrogramów, współczynniki cepstralne oraz parametry poziomu dźwięku. Metody te znalazły nowoczesne zastosowania w diagnostyce akustycznej maszyn, o czym świadczą liczne badania i artykuły naukowe. W poniższych podrozdziałach omówiono zarówno fundamenty teoretyczne każdej z technik oraz przykłady ich praktycznego wykorzystania we współczesnych systemach diagnostycznych.

3.2.1 Analiza widmowa i spektrogramy

Analiza widmowa polega na rozłożeniu sygnału na składowe częstotliwościowe i stanowi jedno z podstawowych narzędzi w akustyce diagnostycznej maszyn. Jak wspomniano w poprzednim podrozdziale, realizuje się ją najczęściej poprzez obliczenie dyskretnej transformaty Fouriera dla sygnału lub jego fragmentu. Widmo ujawnia obecność określonych częstotliwości, które mogą być charakterystyczne dla pracy normalnej lub określonych usterek. Przykładowo, uszkodzenie elementu wirującego często powoduje pojawienie się dodatkowych składowych widmowych przy częstotliwości obrotów lub jej wielokrotnościach, a z kolei asymetria bądź niewyważenie

mogą skutkować ich zwiększoną amplitudą. Analizując widma sygnału akustycznego można zatem wnioskować o stanie maszyny na podstawie identyfikacji tych charakterystycznych elementów.

W wielu zastosowaniach diagnostycznych informacje o obecności lub sile określonych składowych widmowych są wykorzystywane jako dane wejściowe dla algorytmów analitycznych. Przykładem są analizy silników elektrycznych, w których często stosowanym narzędziem przekształcania sygnału jest transformata Fouriera (FFT). Dodatkowo analizie poddaje się również obwiednię sygnału akustycznego oraz jego funkcję autokorelacyjną. Porównując widma sygnałów pochodzących z maszyn sprawnych i uszkodzonych, można zaobserwować dodatkowe składowe w widmie maszyny z usterką, co umożliwia postawienie trafnej diagnozy. Autorzy prac poświęconych tej tematyce zaznaczają też istotną rolę bezinwazyjności metody [11].

Standardowa analiza FFT dostarcza informacji o składzie częstotliwościowym w określonym odcinku czasowym. W wielu przypadkach istotna jest jednak zmienność widma w czasie. Do zobrazowania jak widmo sygnału ewoluuje w czasie wykorzystuje się spektrogramy. Są one reprezentacją czasowo-częstotliwościową, najczęściej przedstawianą jako trójwymiarowa mapa z osią czasu, częstotliwości oraz intensywnością lub odcieniem koloru symbolizującym poziom sygnału. Spektrogram wyznacza się wykonując FFT dla kolejno nakładających się okien czasowych i zestawiając otrzymane widma jedno za drugim. Spektrogram można powiązać z krótkoczasową transformatą Fouriera (STFT), która jest definiowana jako transformata Fouriera sygnału przemnożonego przez przesuwane w czasie okno $w(t)$. Podniesiony do potęgi moduł STFT pozwala uzyskać gęstość mocy w funkcji czasu i częstotliwości:

$$S(t, f) = |STFT_x(t, f)|^2$$

Obliczenia sprowadzają się do powtarzanej wielokrotnie FFT i są rozszerzeniem klasycznej analizy widmowej o wymiar czasowy.

Spektrogramy są niezwykle przydatne w diagnostyce akustycznej, ponieważ wiele zjawisk eksploatacyjnych ma charakter niestacjonarny bądź przejściowy. Powszechnym przykładem może być tzw. chatter, czyli drgania wzbudne w obrabiarkach CNC, które występują tylko w pewnych warunkach obróbki, a ich identyfikacja wymaga obserwacji widma w funkcji czasu. We współczesnych

aplikacjach coraz częściej wykorzystuje się automatyczną analizę spektrogramów z wykorzystaniem głębokiego uczenia, a najpopularniejsze podejście opiera się na potraktowaniu spektrogramu jak obrazu zawierającego wzorce rozpoznawane przez konwolucyjne sieci neuronowe.

3.2.2 Współczynniki cepstralne mel (MFCC) i ich zastosowanie

Współczynniki cepstralne (ang. *Mel-Frequency Cepstral Coefficients*, MFCC) są jednymi z najchętniej wykorzystywanych parametrów sygnałów audio, szczególnie w dziedzinie analizy sygnałów mowy lub jak w niniejszym przypadku – diagnostyki akustycznej. MFCC są parametrami, które opisują kształt widma sygnału w skali częstotliwości mel, czyli dopasowanej do dźwięku postrzeganego przez ludzi. Innymi słowy, współczynniki te reprezentują obwiednie widma sygnału dźwiękowego uwzględniając nieliniowości percepcji poszczególnych częstotliwości. Skala ta przypisuje większą rozdzielczość mniejszym częstotliwościom, a mniejszą wysokim. Cepstralne współczynniki mel są szeroko stosowane z uwagi na ich znakomitą zdolność do kondensacji informacji o barwie dźwięku (ang. *timbre*), która w zagadnieniach związanych z ludzką mową pozwala odróżniać poszczególne głoski, natomiast w diagnostyce akustycznej maszyn może pomóc zróżnicować stany pracy lub typy uszkodzeń na podstawie subtelnych różnic w brzmieniu urządzenia.

Algorytm wyznaczania współczynników MFCC obejmuje szereg etapów przetwarzania sygnału akustycznego i składa się z następujących kroków:

1) Preemfaza

Na wstępie sygnał akustyczny poddawany jest filtracji polegającej na wzmocnieniu górnych częstotliwości, kompensując naturalne tłumienie tych składowych i poprawiając ogólną efektywność ekstrakcji cech.

2) Segmentacja i okienkowanie

Sygnał dzielony jest na częściowo nakładające się ramki, przy czym każda ramka jest mnożona przez okno ograniczając tym samym efekt nieciągłości brzegowych i poprawiając rozdzielczość widmową.

3) Transformacja Fouriera (FFT)

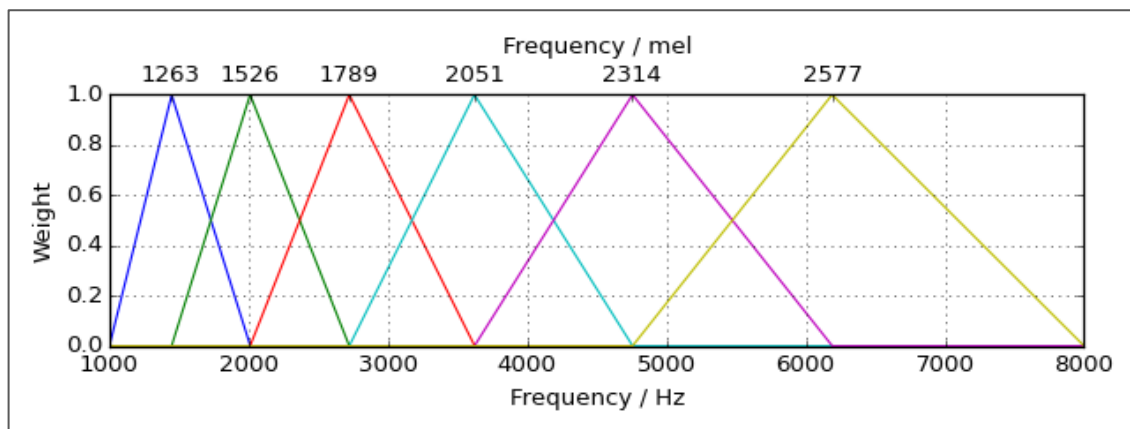
Dla każdej ramki obliczana jest transformata Fouriera, której celem jest przejście z dziedziny czasu do dziedziny częstotliwości. Wynikiem jest widmo amplitudowe dla danej ramki, przedstawiające rozkład energii w funkcji częstotliwości.

4) Filtracja widma na skalę mel

Uzyskane widmo poddaje się analizie przy użyciu banku filtrów trójkątnych ułożonych w skali mel, czyli skali odwzorowującej percepcyjną czułość ludzkiego ucha. Każdy filtr sumuje energię spektralną w przypisanym mu paśmie, co skutkuje uzyskaniem wartości energii w M pasmach mel:

$$E(m) = \sum_{k=f(m-1)}^{f(m+1)} |X(k)|^2 H_m(k),$$

gdzie $H_m(k)$ to charakterystyka m -tego filtra mel, a $f(m)$ to indeksy przypisane poszczególnym pasmom.



Rys. 1. Bank filtrów trójkątnych w skali mel używany przy obliczaniu współczynników MFCC [12]

5) Operacja logarytmu

Energia z każdego pasma $E(m)$ jest następnie przekształcana za pomocą funkcji logarytmicznej. Operacja ta odwzorowuje nieliniowy charakter percepcji głośności zgodnie z prawem Webera-Fechnera i redukuje dynamiczny zakres sygnału:

$$L_m = \log E(m)$$

6) Transformacja DCT (kompresja cepstralna)

W końcowym etapie stosuje się dyskretną transformację cosinusową do logarytmicznych energii pasm mel.

DCT pełni rolę kompaktującą i dekorrelacyjną, skupiając najważniejszą informację w kilku pierwszych współczynnikach:

$$c_k = \sum_{m=1}^M L_m \cos \left[\frac{\pi k}{M} \left(m - \frac{1}{2} \right) \right], \quad k = 1, 2, \dots, K.$$

Zwykle wybiera się $K \ll M$, odrzucając wyższe współczynniki, które zawierają drobne szczegóły widma. W efekcie otrzymujemy końcowy wektor $[c_1, c_2, \dots, c_K]$, stanowiący zwartą reprezentację spektralnych właściwości sygnału w skali mel [13].

Współczynniki MFCC są niezwykle użyteczne ze względu na to, że zachowują najistotniejsze cechy spektralne badanego sygnału, takie jak wysokość tonu czy formanty, które pełnią kluczową rolę w diagnostyce. Pierwotnie zostały one opracowane dla automatycznego rozpoznawania mowy ze względu na ich zdolność do rozróżniania cech artykulacyjnych. Okazały się również bardzo użyteczne przy pracy z DNN, na co wskazują liczne prace badawcze, ponieważ potrafią skutecznie wyodrębnić istotne cechy widma dźwięku, pomijając informacje redundantne lub nieistotne [2].

Współczynniki MFCC w tym kontekście są zazwyczaj wykorzystywane jako wektor danych wejściowych dla klasyfikatorów lub algorytmów uczenia maszynowego, które na ich podstawie potrafią rozpoznać stan maszyny. Zarejestrowane dźwięki mogą zostać przetworzone do sekwencji, które następnie będą służyć do trenowania modeli, takich jak klasyfikatory SVM czy sieci neuronowe, w tym ich głębokie odmiany takie jak CNN lub LSTM. Badania wykazują, że podejście to jest bardzo skuteczne – w pracy Sarfraz *et al.* z 2024 roku wykorzystano cechy MFCC do zasilenia klasyfikatora ekstremalnych maszyn liczących (ELM) w zadaniu przeprowadzenia diagnostyki silników spalinowych, osiągając dokładność w rozpoznawaniu usterek na poziomie 92% [14]. W innym artykule, którego celem była detekcja uszkodzeń torów kolejowych, zastosowane zostały wyselekcjonowane współczynniki mel jako dane wejściowe dla klasyfikatora opartego na metodzie SVM, uzyskując dokładność sięgającą 97%. Autorzy pracy podkreślili, że podejście to wyraźnie przewyższyło tradycyjne metody implementacji [15].

3.2.3 Poziomy dźwięku L_{Aeq} i L_{Apeak}

Analiza częstotliwościowa i współczynniki cepstralne mogą dostarczyć szczegółowych informacji o strukturze sygnału. Jest jednak wiele sytuacji, w których istotne są także bardziej globalne miary takie jak poziom dźwięku. Przy diagnostyce maszyn, szczególnie z punktu widzenia bezpieczeństwa, emisja hałasu do otoczenia jest przydatnym parametrem, który dostarcza dodatkowych, ogólnych informacji o pracy urządzenia. Do tego celu można wykorzystać dwie komplementarne wartości: równoważny poziom dźwięku z krzywą ważenia A – L_{Aeq} oraz szczytowy poziom dźwięku z krzywą ważenia A – L_{Apeak} .

Poziom L_{Aeq} to umowny stały poziom dźwięku, który w zadanym czasie T zawiera taką samą energię akustyczną, co dźwięk o zmiennym poziomie zmierzony w tym samym czasie. Można stwierdzić, że jest to średni w sensie energetycznym poziom dźwięku w określonym przedziale czasu, uwzględniający dodatkowo korekcję A .

Formalnie definicję można wyrazić wzorem:

$$L_{Aeq,T} = 10 \log_{10} \left(\frac{1}{T} \int_0^T \left(\frac{p_A(t)}{p_0} \right)^2 dt \right),$$

gdzie $p_A(t)$ to chwilowe ciśnienie skorygowane krzywą ważenia A , T to czas obserwacji, a $p_0 = 20 \mu Pa$ jest poziomem odniesienia. W praktycznych zastosowaniach często korzysta się z uproszczonej, logarytmicznej definicji, gdzie L_{Aeq} to wartość średnia kwadratowa poziomu dźwięku z krzywą ważenia A w czasie T , wyrażona w decybelach. Poziom równoważny dźwięku A to uśredniona wartość energetyczna odpowiadająca poziomowi stałemu, którego działanie przez ten sam czas dostarcza taką samą ilość energii jak hałas o zmiennym poziomie ciśnienia akustycznego [16]. Z tej informacji wynika, że L_{Aeq} dobrze koreluje ze skutkiem energetycznym hałasu, dlatego miara ta jest podstawą jego oceny w środowisku pracy i ogólnym – pojawia się w normach i przepisach jako główny parametr oceny. W diagnostyce maszyn może być wykorzystany do śledzenia zmian głośności maszyn w czasie. Wzrost emitowanego poziomu dźwięku może sygnalizować postępujące zużycie lub rozregulowanie elementu roboczego urządzenia, nawet jeśli analiza spektralna nie wykazuje jeszcze wyraźnych nowych składowych.

Poziom L_{Apeak} określa natomiast szczytowy poziom dźwięku z ważeniem A zarejestrowany podczas pomiaru. W odróżnieniu od wspomnianego wcześniej L_{Aeq} , odpowiada najwyższej wartości amplitudy ciśnienia akustycznego bez wykonywania jego uśrednień w czasie.

Zdefiniować go można wzorem:

$$L_{Apeak} = 20 \log_{10} \left(\frac{\max_{t \in [0, T]} |p_A(t)|}{p_0} \right).$$

Wartości i oznaczenia we wzorze dla L_{Apeak} pozostają takie same jak w przypadku wzoru na L_{Aeq} . Instrumentalnie mierzy się go z użyciem detektora szczytowego o bardzo krótkiej stałej czasowej (rzędu μs), a to pozwala uchwycić nawet bardzo krótkotrwałe dźwięki. Z powyższego bezpośrednio wynika fakt, że L_{Apeak} jest istotny przede wszystkim w ocenie hałasów impulsowych i udarowych, charakteryzujących się gwałtownymi skokami ciśnienia akustycznego (np. uderzenia, trzaski, wybuchy). Dźwięki o takiej charakterystyce mają znikomy wpływ na równoważny poziom dźwięku, natomiast mogą być niebezpieczne dla słuchu ze względu na dużą amplitudę, dlatego normy BHP oraz akustyczne zazwyczaj określają dopuszczalne wartości chwilowe niezależnie od tych uśrednionych w czasie. W środowisku pracy, polskie przepisy regulują wartość szczytową poziomu dźwięku na jaką może być narażony pracownik do $L_{Cpeak} = 135 dB$, aby zapobiec urazom aparatu słuchu przy gwałtownych impulsach [16]. Odnosząc się do akustycznej diagnostyki maszyn, wartość L_{Apeak} może służyć do detekcji incydentów akustycznych wskazujących na anomalie – znaczny wzrost tej wartości dla pracującego urządzenia, nawet przy odpowiednim poziomie L_{Aeq} , powinien zwrócić uwagę w kontekście potencjalnej nieprawidłowości.

W diagnostyce maszyn na podstawie dźwięku istotne jest uwzględnienie zarówno parametrów opisujących średni poziom dźwięku w danym czasie, jak i tych, które charakteryzują jego chwilowe wartości szczytowe. Pierwsze z nich umożliwiają ocenę długookresowego wpływu hałasu, jego zmian w czasie i ogólnej emisji generowanej podczas pracy urządzenia. Drugie natomiast są niezbędne przy analizie impulsowych i krótkotrwałych anomalii, które mogą mieć ważny wkład diagnostyczny, choć nie wpływać znacząco na średni poziom dźwięku. Z tego względu dokumentacje

techniczne maszyn, zgodnie z obowiązującymi normami i dyrektywami unijnymi, zawierają zwykle zarówno dane o poziomach emisji uśrednionych, jak i chwilowych.

W zastosowaniach diagnostycznych oba typy parametrów mogą pełnić funkcję prostych, ale efektywnych cech stanu maszyny. W połączeniu z bardziej zaawansowanymi metodami, jak analiza widmowa czy cepstralna, miary te umożliwiają kompleksową ocenę emisji dźwiękowej maszyny – od struktury częstotliwościowej sygnału, po jego globalne właściwości energetyczne, co przekłada się na wysoką skuteczność takich systemów w praktyce przemysłowej.

4. Sztuczna inteligencja w diagnostyce akustycznej

4.1 Uczenie maszynowe w analizie sygnałów audio

Współczesna diagnostyka maszyn coraz częściej wykorzystuje metody uczenia maszynowego do przetwarzania i analizy sygnałów akustycznych. Zarejestrowane dźwięki, pochodzące z pracy urządzeń, zawierają istotne informacje o stanie technicznym podzespołów – takich jak łożyska, przekładnie czy silniki – a ich właściwa interpretacja posłużyć do wykrycia usterek oraz oceny kondycji maszyny. Uczenie maszynowe umożliwia automatyczne wykrywanie wzorców w danych dźwiękowych bez konieczności ręcznego definiowania wszystkich reguł decyzyjnych. Typowy proces rozpoczyna się od przekształcenia sygnału akustycznego na formę umożliwiającą jego obróbkę – zazwyczaj jest to wektor cech opisujących właściwości sygnału w różnych domenach: czasowej, częstotliwościowej lub czasowo–częstotliwościowej. Najczęściej stosowane są tutaj reprezentacje takie jak współczynniki cepstralne, parametry widmowe oraz transformaty, które pozwalają uchwycić strukturę dźwięku i zmiany zachodzące w czasie.

Zgodnie z informacjami zawartymi w poprzednim rozdziale, sygnał audio jest dzielony na krótkie, zachodzące na siebie ramki czasowe, a każda z nich jest następnie poddawana analizie i transformatom, które pozwalają wyodrębnić charakterystyczne cechy. Do tego procesu istotny jest wybór odpowiedniego modelu uczącego. Bardziej złożone algorytmy, takie jak głębokie sieci neuronowe, są w stanie

samodzielnie uczyć się reprezentacji cech z surowych danych, jednak wymagają większych zbiorów treningowych i mocy obliczeniowej.

4.1.1 Klasyczne metody uczenia maszynowego

Wśród tradycyjnych algorytmów ML szczególnie często wykorzystuje się Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), drzewa decyzyjne oraz ich pochodne takie jak np. Random Forest oraz metody pozwalające zredukować wymiarowość danych jak analiza głównych składowych (ang. *Principal Component Analysis*, PCA). Różnią się one charakterem (np. liniowość i nieliniowość) oraz sposobem przetwarzania cech akustycznych (np. algorytmy parametryczne i nieparametryczne), co determinuje ich zalety i ograniczenia w zastosowaniu diagnostycznym.

SVM (ang. *Support Vector Machines*) jest algorytmem, który stara się znaleźć hiperpłaszczyznę, która będzie maksymalnie oddzielać klasy próbek w przestrzeni cech, czyli maksymalizuje margines separacji między nimi. Zwykle dobrze sobie radzi przy pracy z małą ilością próbek uczących i potrafi dobrze generalizować. Działanie SVM może zostać sformułowane jako poniższy problem optymalizacji funkcji kwadratowej:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{podlega warunkom} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, N.$$

Powyższa optymalizacja polega na zmaksymalizowaniu marginesu między klasami za pomocą minimalizacji $|w|$ [17]. W celu sprostania danym nieliniowo separowalnymi, algorytm ten używa jądra (ang. *kernel*) przemieniając cechy do przestrzeni wyższych wymiarów. W zastosowaniach akustycznych SVM jest często stosowany na zbiorach cech takich jak MFCC czy współczynniki widmowe. Do jego zalet należy dobra skuteczność przy niewielkich małych i średnich zbiorach danych, odporność na przetrenowanie dzięki maksymalizacji marginesu czy możliwość stosowania małych jąder. Zasadniczym ograniczeniem jest natomiast duży koszt obliczeniowy przy większych zbiorach danych i aspekty praktyczne, takie jak brak bezpośredniej interpretacji prawdopodobieństw. Dodatkowo, do prawidłowego działania niezbędne jest odpowiednie dobranie parametrów i typu jądra.

K-Nearest Neighbours jest nieparametryczną metodą polegającą na przypisaniu klasy próbce na podstawie głosowania k najbliższych sąsiadów w przestrzeni cech. Nie wymaga ona etapu treningu – wszystkie dane uczące się są zapamiętywane, a decyzja

dla nowego punktu opiera się na odległościach do wzorców uczących. Mocną stroną k-NN jest zdecydowanie brak wstępnych założeń o rozkładzie danych, a co się z tym wiąże naturalna obsługa wielu klas. Metoda staje się natomiast bardzo kosztowna obliczeniowo przy dużych zbiorach danych, wrażliwość na skalę poszczególnych cech czy słabnięcie przy dużym wymiarze cech. Algorytm ten w diagnostyce akustycznej może zostać użyty do klasyfikacji stanów maszyny szczególnie wtedy, gdy liczba cech jest niewielka.

Algorytmy typu drzew decyzyjnych budują strukturę hierarchiczną poprzez rekurencyjne podziały danych w oparciu o proste reguły, takie jak progi na poszczególnych cechach.

Na każdym węźle drzewa wybierana jest cecha, która najlepiej separuje klasy, najczęściej mierzone takimi kryteriami jak entropia lub współczynnik Giniego [18]:

$$H = - \sum_j p_j \log_2(p_j).$$

Zaletami drzew decyzyjnych są łatwa interpretowalność wyników, szybkie decyzje i brak konieczności normalizacji cech. Algorytmy te mogą być natomiast podatne na nadmierne dopasowanie (ang. *overfitting*) do danych uczących, co pogarsza uogólnianie. Charakteryzują się też tendencją do niestabilności – mała zmiana może znacząco zmienić strukturę drzewa. Mimo to w diagnostyce są atrakcyjnym rozwiązaniem jako modele dające intuicyjne reguły.

Kolejnym wartym uwagi algorytmem jest Random Forest – metoda bazująca na drzewach decyzyjnych. Polega na tworzeniu wielu warstw drzew na podstawie losowych podzbiorów cech i/lub próbek, z czego każdy model jest pojedynczym drzewem, a finalna predykcja jest najczęściej uśrednieniem lub głosowaniem większościowym wyników. To podejście charakteryzuje się znacznie wyższą stabilnością i dokładnością niż pojedyncze drzewo. Prawdopodobieństwo przeuczenia jest redukowane dzięki automatycznemu mechanizmowi szacowania ważności cech. Ograniczeniami są takie aspekty jak zmniejszona interpretowalność i większe zapotrzebowanie na moc obliczeniową. Tak jak pozostałe wymienione wcześniej algorytmy, Random Forest również bywa używany w diagnostyce za sprawą swej wydajności – często osiąga wysoką skuteczność klasyfikacji usterek przy minimalnej kalibracji parametrów.

Analiza głównych składowych (PCA) to nienadzorowana technika, która znajduje nowe osie układu współrzędnych wyznaczonych przez wektory własne

macierzy kowariancji danych [17]. Umożliwia zmniejszenie wymiaru wektora cech przy zachowaniu jak największej wariancji danych. Technika ta znajduje szerokie zastosowanie jako etap wstępnego przetwarzania danych – umożliwia oczyszczenie przestrzeni cech oraz ich efektywną wizualizację. Nowe zmienne, czyli główne składowe, stanowią liniowe kombinacje oryginalnych cech i są uporządkowane według malejącej wariancji, jaką wyjaśniają. Do głównych zalet PCA należy ograniczenie szumu oraz redundancji danych, co przekłada się na przyspieszenie działania późniejszych algorytmów klasyfikacyjnych i możliwość identyfikacji dominujących kierunków zmienności w zbiorze danych. Do ograniczeń tej metody zalicza się przede wszystkim założenie liniowości relacji między cechami, przez co metoda nie oddaje złożonych, nieliniowych struktur, a ponadto może usuwać cechy istotne dla klasyfikacji, jeśli te charakteryzują się niską wariancją.

Klasyczne metody uczenia maszynowego odgrywają istotną rolę w analizie sygnałów dźwiękowych maszyn, oferując zestaw sprawdzonych narzędzi do klasyfikacji, regresji oraz wstępnego przetwarzania danych. Wyżej przytoczone elementy ML, choć różnią się pod względem założeń teoretycznych i charakteru działania, znajdują zastosowanie w podobnym schemacie diagnostycznym: po ekstrakcji cech akustycznych umożliwiają efektywne mapowanie sygnałów na etykiety stanu technicznego badanej maszyny. W praktyce przemysłowej i eksperymentalnej omawiane algorytmy cechują się wysoką skutecznością w detekcji usterek, przy relatywnie niskich wymaganiach sprzętowych i obliczeniowych. Ich przejrzystość i interpretowalność, szczególnie w przypadku drzew decyzyjnych i k-NN sprawiają, że są one szczególnie atrakcyjne w zastosowaniach wymagających kontroli nad logiką podejmowania decyzji. Mimo rosnącej popularności głębokich sieci neuronowych, klasyczne metody pozostają solidnym fundamentem w tej dziedzinie [17].

4.1.2 Głębokie sieci neuronowe i ich zastosowania

Głębokie sieci neuronowe (ang. *Deep Neural Network*, DNN) są wielowarstwowymi modelami obliczeniowymi składającymi się z neuronów, z których każdy przetwarza sygnały na wejściu za pomocą sumy ważonej sygnałów wejściowych oraz zastosowanie nieliniowej funkcji aktywacji.

W typowej głębokiej sieci neuronowej neuron generuje wyjście o postaci:

$$a = \phi \left(\sum_i w_i x_i + b \right),$$

gdzie w_i są wagami, b – biasem, a ϕ – nieliniową funkcją aktywacji. Do najczęściej stosowanych funkcji aktywacji należą funkcje:

- **ReLU (ang. *Rectified Linear Unit*):**

$$\phi(x) = \max(0, x),$$

- **funkcja logistyczna (sigmoid):**

$$\sigma(x) = 1/(1 + e^{-x}),$$

- **softmax:**

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K.$$

Uczenie DNN odbywa się zwykle metodą wstecznej propagacji (ang. *backpropagation*), która polega na obliczaniu gradientu funkcji kosztu E względem wag i aktualizowaniu ich algorytmem spadku gradientu.

Jako przykład można podać proste uczenie stochastyczne, gdzie wagi aktualizuje się według reguły

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}},$$

co prowadzi do zmiany wagi w_{ij} w kierunku minimalizacji błędu. Dla problemów klasyfikacji typowo stosuje się funkcję kosztu w postaci entropii krzyżowej. Algorytm wstecznej propagacji umożliwia uczenie wielu warstw DNN poprzez efektywne propagowanie informacji o błędzie od warstwy wyjściowej do wejściowej [19].

Jedną z najpowszechniejszych architektur DNN są konwolucyjne sieci neuronowe, zbudowane z warstw splotowych (ang. *convolutional*), agregujących (ang. *pooling*) oraz w pełni połączonych (ang. *fully connected*). W danych wielowymiarowych pozwalają one na automatyczne uczenie hierarchii cech przestrzennych, a dzięki lokalnym połączeniom i współdzieleniu wag sieci efektywnie

wykrywają wzorce lokalne w obrazach. Do zastosowań diagnostycznych w akustyce, po przekształceniu sygnału dźwiękowego do reprezentacji dwuwymiarowej, takiej jak widmo czy spektrogram, CNN uczą się wykrywać wzorce oznaczające dane stany maszyny [20].

Rekurencyjne sieci neuronowe (ang. *Recurrent Neural Network*, RNN) zaprojektowano do przetwarzania danych sekwencyjnych, takich jak sygnały czasowe. Dzięki sprzężeniom zwrotnym wyjście neuronu w danym kroku czasowym może być przekazywane jako dodatkowe wejście w kroku następnym, umożliwiając uwzględnianie kontekstu czasowego. Klasyczne RNN mają jednak ograniczenia związane z zanikiem gradientu przy długich sekwencjach, co powoduje utrudnienie nauki długoterminowych zależności. Problemy te rozwiązuje architektura LSTM (ang. *Long Short-Term Memory*), wprowadzająca bramki i pamięć pozwalającą na przekazywanie istotnych informacji przez dłuższe interwały czasowe. LSTM stał się standardowym typem RNN w zastosowaniach wymagających analizy długich sekwencji, takich jak mowa, analiza maszyn, itp. Istnieją też warianty upraszczające (np. GRU) lub niezależne (np. IndRNN) rozwiązujące podobne problemy [20].

Kolejną wartą uwagi klasą DNN są autoenkodery (AE) – sieci uczone w sposób nienadzorowany mające na celu rekonstrukcję zadanych im danych wejściowych. Autoenkoder składa się z części kodera, która odwzorowuje wejście na zwężoną reprezentację (ang. *bottleneck*) oraz dekodera, którego zadaniem jest rekonstrukcja wejścia z tej reprezentacji. Celem jest nauczenie wydajnej reprezentacji kodowania zbioru danych bez nadzoru. Autoenkodery mogą służyć m.in. do redukcji wymiaru danych, detekcji anomalii czy wstępnego uczenia cech. Przykładowo, istnieją wariacyjne autoenkodery (VAE), które wprowadzają dodatkowy warunek generatywny i uczą się z gęstszym rozkładem latentnym [20].

Głębokie sieci neuronowe znakomicie wpasowują się w zagadnienie diagnostyki akustycznej. Ich zdolność do automatycznego uczenia się reprezentacji danych sprawia, że są one szczególnie przydatne w analizie sygnałów akustycznych, które charakteryzują się dużą złożonością czasowo-częstotliwościową.

4.2 Konwolucyjne sieci neuronowe (CNN)

Konwolucyjne sieci neuronowe są rodzajem DNN zoptymalizowany do analizy danych o charakterze przestrzennym, takich jak obrazy czy jak w niniejszym przypadku – widma sygnałów. Korzenie CNN sięgają badań nad funkcjonowaniem kory wzrokowej, a jednym z pierwszych praktycznych modeli był Neokognitron (1979), wprowadzający hierarchiczne wydobywanie cech. Pierwszą działającą architekturą, zaproponowaną w 1998 roku przez LeCun *et al.*, było LeNet-5 – sieć składająca się z pięciu warstw, w tym konwolucyjnych i poolingowych, przeznaczona do rozpoznawania ręcznie pisanych cyfr. Jednym z większych przełomów w dziedzinie tego rodzaju sieci neuronowych nastąpił w 2012 roku, kiedy model AlexNet, który dzięki użyciu nieliniowości ReLU oraz losowemu wyłączaniu neuronów (ang. *dropout*) znacząco poprawił skuteczność klasyfikacji obrazów, obniżając błąd klasyfikacji w konkursie ImageNet z 26% do 15%.

4.2.1 Architektura i zasada działania CNN

Zasadniczym elementem sieci CNN jest warstwa splotowa (konwolucyjna), czyli liniowa operacja, w której dla każdego przesuwu jądra mnoży się jego wagi przez wartości wejścia.

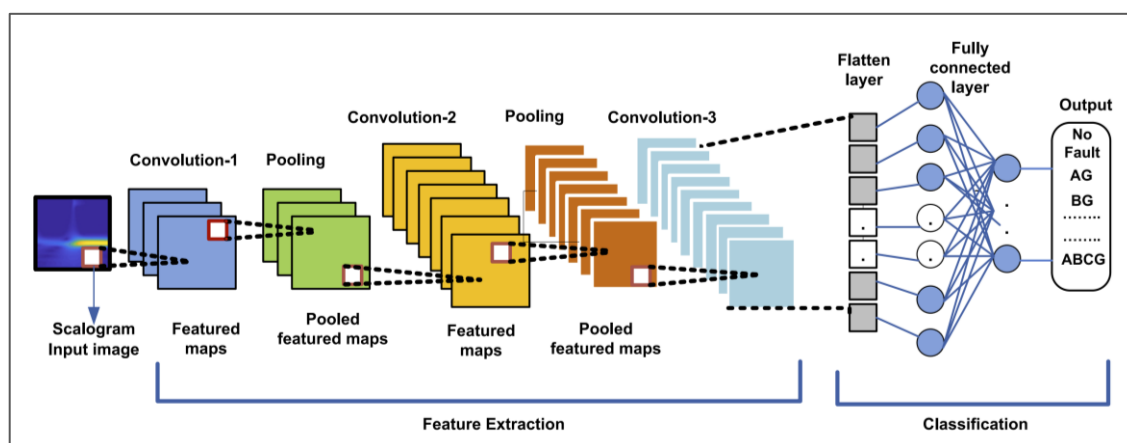
Wyjście $y[i, j]$ można zapisać jako sumę iloczynów wag o odpowiadających im elementów danych wejściowych:

$$y[i, j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[i + m, j + n] w[m, n],$$

gdzie $w[m, n]$ to jądro o wymiarach $M \times N$. Każdy filtr uczy się wykrywać określone cechy lokalne, takie jak krawędzie czy tekstury na zadanym obrazie. Po warstwie konwolucji zazwyczaj stosuje się funkcję aktywacji, najczęściej ReLU, która wprowadza nieliniowość i przyspiesza proces uczenia algorytmu. Funkcja ta skutecznie przewycięża problemy z zanikiem gradientu, pozwalając na efektywne uczenie głębokich sieci [21].

Innym kluczowym elementem jest warstwa agregacji (poolingu), która redukuje wymiary przestrzenne aktywacji i zmniejsza liczbę parametrów. Najczęściej stosuje się max-pooling, wybierający maksymalną wartość z lokalnego obszaru cech, dzięki czemu sieć zachowuje najistotniejsze cechy przy zredukowanej rozdzielczości mapy cech,

co przyspiesza obliczenia i działa jako forma uogólnienia [22]. Po warstwach splotu i agregacji następują zwykłe warstwy w pełni połączone, które przekształcają uzyskane reprezentacje do wymiaru wyjściowego, takiego jak wektora prawdopodobieństwa klas. Proces uczenia CNN wykorzystuje algorytm propagacji wstecznej. Sieć na wejściu otrzymuje obraz lub inną reprezentację, a następnie propaguje go do przodu przez kolejne warstwy obliczając błąd względem oczekiwanych wyników. W następnym kroku obliczane są gradienty błędu względem wag, które następnie są aktualizowane metodą spadku gradientu.



Rys. 2. Przykładowa architektura konwolucyjnej sieci neuronowej (CNN) [23]

Sieci konwolucyjne znalazły zastosowanie w wielu dziedzinach. Są przede wszystkim podstawą współczesnego przetwarzania obrazu – wykorzystywane są m.in. w rozpoznawaniu twarzy, pojazdów na drodze czy analizy tekstu, a także w medycznych systemach wizyjnych do wykrywania zmian nowotworowych. Ponadto, CNN sprawdzają się również w zadaniach związanych z dziedziną audio. Mając odpowiednie wejście, mogą klasyfikować typy dźwięków i zdarzeń akustycznych. W badaniach stwierdzono, że architektury oparte na tym rodzaju sieci sprawdzają się doskonale w analizie dźwięku maszyn przemysłowych, a nawet klasyfikacji dźwięków środowiskowych [21].

W diagnostyce akustycznej maszyn szczególnego znaczenia nabiera odpowiednie przygotowanie danych wejściowych do dostarczenia sieciom neuronowym. Ponieważ dźwięk rejestrowany w postaci sygnału czasowego jest jednowymiarowy, przed jego analizą za pomocą CNN należy dokonać konwersji do reprezentacji dwuwymiarowej, na przykład w formie spektrogramu lub mapy współczynników MFCC. Reprezentacje te uwzględniają zarówno wymiar czasowy,

jak i częstotliwościowy, umożliwiając sieciom neuronowym stosowanie operacji splotu analogicznych do przetwarzania obrazów. Dzięki temu możliwe jest automatyczne wykrywanie wzorców charakterystycznych dla pracy danej maszyny (np. szumy, drgania czy zakłócenia), bez konieczności wcześniejszej, manualnej inżynierii cech. Sieci te skutecznie wykrywają różnego rodzaju nieprawidłowości w pracy maszyn na podstawie sygnałów dźwiękowych. Podejście to znalazło szerokie zastosowanie w diagnostyce łożysk czy silników elektrycznych, gdzie CNN wyraźnie przewyższały tradycyjne algorytmy, które bazują na ręcznie wybranych cechach. Dodatkową zaletą architektury CNN jest jej podatność na różnorodne techniki augmentacji danych, takie jak wprowadzanie sztucznych szumów, zmiany fazy czy przesunięcia czasowe, co przekłada się na większą odporność modelu na zakłócenia w danych pomiarowych i podnosi ogólną skuteczność klasyfikacji.

4.2.2 Zastosowanie CNN w diagnostyce audio

Sieci konwolucyjne w akustycznej diagnostyce maszyn zyskały na popularności dzięki znakomitej zdolności znajdowania wzorców w graficznych reprezentacjach sygnałów. Modele głębokiego uczenia, takie jak autoenkodery, wariacyjne autoenkodery (VAE) oraz CNN, wykazują bardzo dobre właściwości w zakresie uczenia reprezentacji danych oraz identyfikacji typów uszkodzeń w trybie nienadzorowanym. W szczególności CNN są dobrze przystosowane do analizy danych niestrukturalnych, takich jak sygnały akustyczne, i potrafią skutecznie rozpoznawać ich charakterystyczne wzorce częstotliwościowe dzięki zastosowaniu wielowymiarowych filtrów [24].

Powołując się na praktyczne implementacje, warto wspomnieć o projekcie zbliżonym merytorycznie do niniejszego, natomiast zrealizowanym jako aplikacja mobilna. Celebioglu *et al.* zaproponowali system wykrywania uszkodzeń łożysk z wykorzystaniem nagrań akustycznych pozyskiwanych za pomocą smartfona. Dane dźwiękowe pochodzące z maszyn obrotowych w różnych stanach technicznych posłużyły do wytrenowania jednowymiarowej konwolucyjnej sieci neuronowej (1D-CNN), a uzyskany model został zaimplementowany na urządzeniu mobilnym w celu prowadzenia diagnostyki w czasie rzeczywistym. Dzięki takiemu podejściu możliwe stało się nisko kosztowe, mobilne monitorowanie stanu technicznego maszyn bez potrzeby stosowania drogich czujników [25]. Badania wykazują również, że sygnały akustyczne mogą zawierać bogatszą informację do celów diagnostyki niż wibracyjne.

Przytaczając argumenty mówiące o zaletach wykorzystania CNN w diagnostyce, warto porównać je z innymi rodzajami sieci neuronowych pod tym kątem. W wielu eksperymentach modele oparte na sieciach konwolucyjnych znacząco przewyższyły klasyczne algorytmy, takie jak SVM czy ELM. W badaniach Kumara *et al.* CNN wykazała się zdecydowanie wyższą skutecznością w detekcji uszkodzeń przekładni mechanicznych, osiągając nawet 100% dokładności klasyfikacji dla wszystkich stanów technicznych, w porównaniu do 86,15% w przypadku ELM oraz 83,06% dla SVM. Autorzy podkreślili fakt, że proponowana metoda oparta na CNN była skuteczniejsza dla każdego ze stanów diagnostycznych przekładni, co świadczy o jej uniwersalności i dokładności [26].

Warto również wspomnieć, że coraz częściej stosuje się hybrydowe rozwiązania łączące sieci konwolucyjne z rekurencyjnymi (np. RNN czy LSTM), które pozwalają równocześnie uchwycić lokalne wzorce czasowo-częstotliwościowe oraz zależności czasowe w sekwencjach. Jak podają autorzy jednego z przeglądów poświęconych monitorowaniu w architekturze IoT, sieci LSTM dobrze odwzorowują zależności czasowe w danych sekwencyjnych pochodzących z czujników, czyniąc je komplementarnymi wobec klasycznych CNN. Niemniej jednak, w wielu zastosowaniach sama architektura CNN okazuje się wystarczająco skuteczna i prostsza w implementacji. W dostępnych opracowaniach często wskazuje się konkretne architektury konwolucyjnych sieci neuronowych, które znalazły zastosowanie w diagnostyce maszyn.

W zależności od specyfiki problemu, struktury te są często modyfikowane np. poprzez rozbudowę o dodatkowe warstwy, zastosowanie technik regularyzacji czy wprowadzenie mechanizmów uwagi. Dla przykładu, architektura EFENet została zaprojektowana jako połączenie klasycznych warstw splotowych z uwagą kanałową, co poprawia zdolność modelu do selektywnego przetwarzania istotnych cech sygnału. Z kolei w innych badaniach wykorzystano iteracyjne podejście z transferem wiedzy (Transfer Learning) oparte na sieci VGG-19, które umożliwiło skuteczne wykrywanie usterek silników indukcyjnych w warunkach przemysłowych. W kontekście zastosowań, gdzie obliczenia mają być wykonywane na danym urządzeniu (ang. *edge computing*) coraz częściej wykorzystuje się także lżejsze, zoptymalizowane pod kątem ograniczonych zasobów obliczeniowych i umożliwiające detekcję w czasie rzeczywistym [7][24].

Z matematycznego punktu widzenia, podstawowa operacja splotu polega na przemieszczaniu tzw. okna filtrującego (jądra) po sygnale i obliczaniu ważonej sumy elementów, co w przypadku wyżej przytoczonego 1D-CNN można zapisać jako:

$$y(n) = \sum_{i=0}^{k-1} x(n+i) \cdot h(i),$$

gdzie $h(i)$ oznacza jądro splotowe, a $x(n)$ to próbki wejściowego sygnału akustycznego [26]. Takie przekształcenie jest zwykle uzupełniane przez funkcje aktywacji (np. ReLU), normalizację oraz operacje agregujące.

Do obliczania straty klasyfikacyjnej często stosuje się entropię krzyżową, którą formalnie zapisuje się jako:

$$L = - \sum_i y_i \log(\hat{y}_i),$$

gdzie y_i to etykiety klas, a $\hat{y}_i$ to prawdopodobieństwa wynikające z predykcji modelu [26][27].

Konwolucyjne sieci neuronowe stanowią jedno z kluczowych narzędzi we współczesnej diagnostyce maszyn, zwłaszcza w analizie sygnałów akustycznych. Dzięki zdolności do automatycznego wydobywania złożonych cech czasowo-częstotliwościowych, umożliwiają skuteczne wykrywanie usterek, klasyfikację stanów technicznych czy identyfikację anomalii. W wielu badaniach dowiedziono ich przewagi nad klasycznymi algorytmami, zarówno pod względem skuteczności, jak i odporności na zakłócenia. Dobrze zdefiniowane podstawy matematyczne tych modeli pozwalają na ich precyzyjną analizę oraz optymalizację pod kątem konkretnych zastosowań.

5. Projekt i implementacja systemu

5.1 Wymagania systemu

Niniejszy projekt stanowił złożone wyzwanie, wymagające starannego określenia oczekiwań wobec opracowanego rozwiązania. Klarowne zdefiniowanie założeń wymagań systemowych było fundamentem dalszych prac, umożliwiając stworzenie architektury systemu dopasowanej do rzeczywistych potrzeb użytkowników oraz

uwarunkowań technicznych. Opracowanie wymagań zostało oparte zarówno na analizie praktycznych scenariuszy zastosowania, jak i na wnioskach płynących z dostępnych badań i literatury dotyczącej inteligentnych systemów diagnostycznych opartych na zastosowaniu głębokiego uczenia.

5.1.1 Wymagania funkcjonalne

System został zaprojektowany jako aplikacja webowa oparta na frameworku Django, której głównym celem jest możliwość prowadzenia analizy stanu technicznego urządzeń przemysłowych na podstawie danych akustycznych.

Do najważniejszych wymagań funkcjonalnych, wynikających z założeń projektowych należą:

1) Obsługa akwizycji danych

Jedną z podstawowych funkcjonalności było zapewnienie możliwości przechwytywania sygnału akustycznego zarówno z mikrofonu w czasie rzeczywistym, jak i z wcześniej nagranych plików dźwiękowych w formacie WAV. Aplikacja powinna automatycznie wykrywać źródła danych, a następnie umożliwiać użytkownikowi wybór trybu pracy. W tym kontekście istotna jest także synchronizacja oraz jednolite przetwarzanie danych wejściowych, niezależnie od ich źródła, tak aby mogły być bezpośrednio poddane ekstrakcji cech oraz analizie modelu klasyfikacyjnego.

2) Przetwarzanie dźwięku i ekstrakcja cech akustycznych

System musi zapewniać możliwość automatycznego przetwarzania sygnału dźwiękowego zarówno w trybie ciągłym (monitoring czasu rzeczywistego), jak i wsadowym (trening i testowanie na zapisanych danych). W obu przypadkach aplikacja musi dokonywać ekstrakcji parametrów sygnałowych takich jak współczynniki MFCC, poziom dźwięku równoważny oraz poziom szczytowy. Przetwarzanie to powinno odbywać się w sposób zoptymalizowany pod kątem wydajności, z uwzględnieniem konfigurowalnych parametrów.

3) Dynamiczna detekcja i obsługa klas diagnostycznych na podstawie struktury folderów

Wymagane jest, aby system był zdolny do samodzielnego rozpoznawania liczby oraz nazw klas diagnostycznych na podstawie zawartości struktury folderów z danymi. Dzięki temu użytkownik może analizować dowolny typ maszyny, a także zdefiniować nowe stany awaryjne poprzez dodanie katalogów z plikami dźwiękowymi odpowiadającymi danym klasom. Mechanizm ten umożliwia natychmiastowe dopasowanie się systemu do nowego przypadku użycia bez konieczności rekonfiguracji kodu źródłowego.

4) Interaktywny formularz konfiguracji parametrów modelu i treningu

System musi udostępniać użytkownikowi interfejs umożliwiający edycję kluczowych parametrów treningu i architektury sieci neuronowej. Formularz ten powinien być dostępny z poziomu aplikacji internetowej i mieć walidację poprawności wprowadzonych danych, tak aby użytkownik nie mógł ustawić konfiguracji uniemożliwiającej uruchomienie treningu.

5) Zachowanie i ponowne wykorzystanie konfiguracji modelu oraz parametrów normalizacji

Wymagane jest, aby system automatycznie zapisywał pełną konfigurację modelu treningowego oraz wszystkie parametry, które mają wpływ na późniejsze działanie modelu. Dotyczy to w szczególności parametrów takich jak średnia i odchylenie standardowe wykorzystane do normalizacji danych, liczba oraz nazwy klas, długość MFCC, liczba współczynników cech, a także szczegóły architektury sieci. W momencie przejścia do monitoringu użytkownik musi mieć pewność, że system będzie przetwarzał dane w dokładnie taki sam sposób, jak podczas treningu.

6) Możliwość monitoringu w czasie rzeczywistym

System musi być zdolny do przechwytywania sygnału z mikrofonu w czasie rzeczywistym, podziału go na próbki o zadanim czasie trwania, ekstrakcji cech i klasyfikacji przez wytrenowany model CNN. Otrzymane wyniki muszą być interpretowane zgodnie z typem klasyfikacji oraz porównywane z progami

diagnostycznymi ustalonymi przez użytkownika. Klasyfikacja powinna odbywać się wraz z rejestrowaniem wyników oraz wizualną prezentacją statusu w interfejsie.

7) Obsługa augmentacji danych

System musi umożliwiać sztuczne rozszerzanie zbioru danych treningowych w celu poprawy generalizacji modelu, szczególnie w kontekście nie zrównoważonych danych pomiędzy klasami. Augmentacja powinna być stosowana tylko dla klas z usterkami i opierać się na kombinacji przekształceń dźwięku takich jak zmiana wysokości tonu, dodanie szumu tła, modulacja czasu trwania czy zmiana głośności. Dzięki temu użytkownik może elastycznie dostosować stopień rozszerzenia zbioru w zależności od dostępnych danych i jakości nagrań.

8) Zapis historii pomiarów i wyników diagnostyki

Aplikacja musi posiadać mechanizm zapisywania wyników analizy monitoringu. Każdy zapis powinien zawierać informację o dacie i godzinie pomiaru, wartościach parametrów, nazwie klasy wybranej przez model (lub prawdopodobieństwach wszystkich klas), a także statusie diagnostycznym. Taki system rejestracji wyników umożliwi analizę trendów, ocenę stabilności pracy maszyny oraz może służyć jako materiał dowodowy w przypadku wystąpienia usterek.

9) Interfejs formularza konfiguracji z dynamicznym zapisem ustawień

Wymagane jest, aby wszystkie najistotniejsze parametry działania systemu mogły być ustawiane przez użytkownika za pomocą interfejsu. Obejmuje to konfigurację parametrów monitoringu, architektury modelu, augmentacji, progów diagnostycznych, parametrów akwizycji i zapisu wyników.

10) Obsługa powiadomień diagnostycznych

W momencie wykrycia usterki lub jej podejrzenia, system powinien móc, jeśli użytkownik tego zażyczy, wysłać informacje o tym fakcie na telefon lub inne urządzenie. Powiadomienia te muszą zawierać wszystkie niezbędne parametry modelu oraz nazwę klasy, która została zdiagnozowana jako najbardziej prawdopodobna. Możliwość włączenia lub wyłączenia tej funkcji musi być dostępna z poziomu aplikacji.

5.1.2 Wymagania niefunkcjonalne

W tej części pracy zostały opisane wymagania jakościowe, które nie odnoszą się bezpośrednio do konkretnych funkcji realizowanych przez system, lecz definiują oczekiwane cechy i właściwości projektu, wpływające na jego użyteczność, efektywność, niezawodność oraz możliwości rozwoju i utrzymania. W kontekście opracowanego systemu diagnostyki akustycznej maszyn, wymagania te zostały sformułowane na podstawie analizy środowiska pracy, potrzeb użytkownika końcowego oraz przyjętych założeń technologicznych i metodologicznych.

Poniżej przedstawiono szczegółowo najistotniejsze aspekty, z podziałem na konkretne aspekty:

1) Przejrzystość i ergonomia interfejsu użytkownika

System powinien zapewniać wysoką czytelność oraz intuicyjność w obsłudze. Oczekuje się, że użytkownik, niezależnie od poziomu zaawansowania technicznego, będzie w stanie samodzielnie skonfigurować system, uruchomić proces monitorowania oraz zinterpretować wyniki diagnostyczne. W związku z tym istotne jest, aby wszystkie pola formularzy, przyciski, wykresy i komunikaty były jednoznacznie opisane, a układ graficzny odpowiadał zasadom projektowania systemów HMI (ang. *Human-Machine Interface*).

2) Skalowalność i możliwość rozbudowy systemu

System powinien być zaprojektowany w sposób modułowy, umożliwiający jego rozbudowę o nowe funkcje, algorytmy, modele oraz źródła danych. Struktura kodu oraz organizacja folderów i komponentów Django muszą wspierać łatwą implementację nowych klas urządzeń, formatów danych, metod przetwarzania lub typów powiadomień. Projekt powinien również uwzględniać możliwość dodania nowych warstw modelu, alternatywnych metod ekstrakcji cech czy rozbudowy systemu o funkcje zarządzania użytkownikami i uprawnieniami.

3) Odporność na błędy

Wymaga się, aby system zachowywał stabilność działania nawet w przypadku pojawienia się błędów w źródłach danych (np. brak mikrofonu, uszkodzony plik WAV), braku połączenia internetowego, błędów konfiguracji lub problemów z sesją

użytkownika. System musi zawierać mechanizmy przechwytywania i logowania wyjątków oraz komunikaty informujące użytkownika o występujących problemach w sposób zrozumiały i nienaruszający działania pozostałych komponentów aplikacji.

4) Kompatybilność międzyplatformowa

System powinien być możliwy do uruchomienia na szerokim zakresie komputerów osobistych z systemem operacyjnym Windows 10 lub nowszym, bez konieczności stosowania specjalistycznego sprzętu. Wymaga się, aby komponenty aplikacji, w tym biblioteki Python, były w pełni kompatybilne z architekturą x86_64. Konfiguracja środowiska powinna być możliwie uproszczona.

5) Czytelność wyników diagnostycznych

System powinien zapewniać zrozumiały sposób prezentowania wyników diagnostyki. Rezultaty muszą być jednoznaczne i zorientowane również na użytkownika nietechnicznego. Wymagane jest, aby informacje diagnostyczne były wyświetlane w formie tekstowej, czytelnej i powiązanej z dodatkowymi szczegółami, takimi jak wartości prawdopodobieństw, numery klas oraz poziomach zarejestrowanego dźwięku.

6) Zgodność z dobrymi praktykami inżynierii oprogramowania

W ramach ogólnych wymagań niefunkcjonalnych system powinien być zaprojektowany z zachowaniem zasad modularności, separacji logiki aplikacji od warstwy prezentacji oraz przejrzystości kodu. Struktura projektu Django powinna pozwalać na łatwą modyfikację poszczególnych komponentów. Logika przetwarzania danych powinna być oddzielona od warstwy graficznej i konfiguracyjnej.

5.2 Architektura systemu diagnostycznego

W niniejszym rozdziale przedstawiona została szczegółowa struktura systemu, obejmująca zarówno perspektywę funkcjonalną, jak i techniczną z uwzględnieniem sposobu implementacji, zależności między komponentami i środowiska uruchomieniowego.

Przyjęta architektura systemu została zaprojektowana w sposób modułowy i skalowalny, co umożliwia przyszłą rozbudowę aplikacji o dodatkowe funkcjonalności. Celem tej części pracy jest pełne zobrazowanie struktury logicznej oraz

implementacyjnej systemu, jak również zidentyfikowanie wszystkich komponentów, które odpowiadają za poszczególne etapy cyklu działania aplikacji – od wejścia sygnału do wyprowadzenia końcowego wyniku diagnostycznego.

5.2.1 Architektura funkcjonalna

Architektura funkcjonalna systemu odnosi się do logicznej organizacji głównych funkcji aplikacji oraz ich wzajemnych powiązań w kontekście całego przepływu informacji – od danych wejściowych, przez przetwarzanie i klasyfikację, aż po wizualizację wyników i reakcje systemowe. Jej celem jest zapewnienie przejrzystego i jednoznacznego modelu operacyjnego, który umożliwia śledzenie transformacji danych oraz identyfikację roli każdego z komponentów. W poniższej części skupiono się na obsłudze najważniejszego elementu aplikacji – diagnostyki w czasie rzeczywistym.

System diagnostyczny składa się z następujących funkcjonalnych modułów: akwizycji danych, przetwarzania sygnału, ekstrakcji cech, klasyfikatora CNN, interpretacji wyników, systemu powiadomień oraz interfejsu użytkownika. Moduł akwizycji danych odpowiada za pobieranie sygnału akustycznego – użytkownik może wybrać plik audio lub użyć mikrofonu.

W kodzie widoku `get_realtime_data` zastosowano bibliotekę `librosa` do wczytania pliku WAV:

```
full_audio, _ = librosa.load(file_path, sr=sample_rate)
```

W przypadku monitoringu z mikrofonu sygnał jest rejestrowany przez `sounddevice`:

```
audio = sd.rec(
    int(duration * sample_rate),
    samplerate=sample_rate,
    channels=1,
    dtype=np.float32,
    blocking=True,
    device=device_index if device_index is not None else None
)
```

Następnie moduł przetwarzania sygnału wykonuje niezbędne operacje na surowych danych: normalizuje amplitudę, usuwa wartości NaN lub nieskończoności oraz obcina wartości przekraczające zakres.

Przykładowo, kod sprawdza i przycina nadmiarowe wartości:

```
if not np.all(np.isfinite(audio)):
    return JsonResponse({'status': 'error', 'message': 'Błąd sygnału: dane
nieprawidłowe, ...'})
...
audio = np.clip(audio, -1.0, 1.0)
```

W etapie ekstrakcji cech wyliczane są współczynniki MFCC przy użyciu biblioteki librosa:

```
mfcc = librosa.feature.mfcc(
    y=audio,
    sr=sample_rate,
    n_mfcc=n_mfcc,
    n_fft=n_fft,
    hop_length=hop_length,
    center=False
)
```

Wynik jest normalizowany przy użyciu parametrów mean i std obliczonych podczas treningu modelu. Moduł klasyfikatora ładuje wytrenowany model CNN (architektura zdefiniowana w kodzie treningu) i wykonuje predykcję.

Przykładowo, w widoku monitoringu wykonywane jest:

```
input_tensor = tf.convert_to_tensor(mfcc_processed, dtype=tf.float32)
prediction = model.predict(input_tensor, verbose=0)[0]
```

Wynik predykcji (wektor prawdopodobieństw) jest następnie interpretowany. W przypadku trybu binarnego sumowane są prawdopodobieństwa klas usterki, a przy wieloklasowym dobierana jest klasa z największym prawdopodobieństwem. Interpretacja wyników polega na zestawieniu wyniku modelu z progami progowymi zapisanymi w konfiguracji diagnostyki (pola `t_low`, `t_high`, `laeq_low`, `laeq_high`).

Na przykład jeśli zmierzony poziom L_{Aeq} przekracza wartości progów, status ustalany jest na *fault*:

```
if current_leq < config.laeq_low or current_leq > config.laeq_high:
    status = "fault"
    display_status = "Usterka:  $L_{Aeq}$ "
```

W przeciwnym razie decyzja oparta jest na predykcji modelu i progach procentowych t_{low} , t_{high} : przy predykcji poniżej t_{low} uznajemy stan za normalny, pomiędzy za „podejrzenie usterki”, powyżej za „usterkę”. W module interpretacji zapisywany jest też wynik diagnostyki do bazy danych oraz do pliku wynikowego (`save_monitoring_result_csv`). Jeśli wynik wskazuje *warning* lub *fault* i użytkownik włączył powiadomienia (`config.send_notification`), uruchamiany jest moduł wysyłania powiadomień. Jest on zaimplementowany funkcją `send_notification`, która korzysta z API Pushbullet, wysyłając tekst powiadomienia i załącznik audio. Przykładowo, gdy wystąpi usterka, tytuł powiadomienia to *WYKRYTO USTERKĘ* lub *PODEJRZENIE USTERKI*, a treść obejmuje takie dane jak prawdopodobieństwo usterki, poziomy L_{Aeq} , L_{Apeak} i odczyty wszystkich klas.

Moduł panelu użytkownika zapewnia interfejs do konfiguracji i monitoringu. Z poziomu przeglądarki użytkownik wybiera maszynę, ewentualny plik audio i steruje przyciskami startu oraz zatrzymania. Elementy takie jak lista maszyn są dynamicznie wypełniane przez JavaScript wywołujący API Django. Interfejs wyświetla aktualny status, wykres słupkowy prawdopodobieństw (dla klasyfikacji wieloklasowej) lub pojedynczy pasek (dla binarnej) oraz rozbudowany widok widma MFCC rysowany na kanwie HTML5. Logi pomiarów dopisywane są do tabeli w panelu.

Całe przetwarzanie danych przebiega w pętli monitoringu: przyciski na stronie powodują włączenie/ wyłączenie flag w sesji, a co ustalony interwał (konfigurowalny w polu `interval`) front-end wysyła zapytanie AJAX do widoku `get_realtime_data`. Ten zwraca JSON z aktualnym statusem, który jest następnie aktualizowany w interfejsie użytkownika przez skrypt JavaScript metody `updateUI`. Dzięki temu użytkownik na bieżąco widzi postęp pomiarów i wyniki.

5.2.2 Architektura techniczna i komponentowa

Architektura techniczna stanowi implementację struktury funkcjonalnej, uwzględniającą konkretne komponenty oprogramowania, sposób ich organizacji, komunikację między modułami oraz środowisko uruchomieniowe.

System zaprojektowano jako aplikację typu klient-serwer z warstwową separacją logiki, warstwy prezentacji i komponentów odpowiedzialnych za przetwarzanie danych.

Aplikacja jest zbudowana w architekturze Django (Model-View-Template) z wyraźnym rozdziałem warstw: warstwa prezentacji (HTML/CSS/JS), warstwa logiki biznesowej (widoki Django, backend) i warstwa danych (modele i baza danych).

Po stronie backendu komponenty Django są zorganizowane w aplikację *diagnostics*. Główne widoki aplikacji w plikach *views_monitoring.py*, *views_train_model_thread.py*, *views_results.py* realizują logikę monitoringu i treningu modeli.

Na przykład, widok `monitoring(request)` renderuje szablon *diagnostics/monitoring.html* i przekazuje do niego nazwy dostępnych modeli maszyn:

```
machines = [
    f.replace('.keras','')
    for f in os.listdir(WYNIKI_PATH)
    if f.endswith('.keras')
]
classification_type = request.session.get("classification_type",
"multiclass")
return render(request, 'diagnostics/monitoring.html', {'machine_names':
machines, 'classification_type': classification_type})
```

Szablon ten definiuje strukturę interfejsu użytkownika z formularzami i kontenerami na wykresy (np. `<select id="machineSelect">`, `<canvas id="predictionChart">` w *monitoring.html*). Komunikacja między frontendem a backendem odbywa się częściowo przez tradycyjne widoki renderujące strony, a częściowo przez wywołania AJAX. W skryptach frontend JavaScript (pliki *monitoring.js* i *training.js*) używany jest API przeglądarki `fetch` do żądań pod konkretne ścieżki Django.

Jako przykład można podać zdefiniowane w kodzie *monitoring.js* adresy endpointów:

```
const API_ENDPOINTS = window.API_ENDPOINTS || {
  start: '/diagnostics/start_monitoring/',
  getData: '/diagnostics/get_realtime_data/',
  stop: '/diagnostics/stop_monitoring/',
  getFolders: '/diagnostics/get_machine_folders/',
  getAudioFiles: '/diagnostics/get_audio_files/',
};
```

Metoda `startMonitoring()` wysyła formularz metodą POST do `start_monitoring/`, a następnie okresowo funkcja `setInterval` pobiera nowe dane z `get_realtime_data/`. Analogicznie, w module treningu *training.js* po wciśnięciu przycisku *Rozpocznij trening* wykonywane jest fetch do `/diagnostics/start_training/`, co uruchamia nowy wątek treningowy w backendzie. Następnie co sekundę pobierane są dane statusu treningu z `/diagnostics/get_training_progress/` i na ich podstawie aktualizowane są wykresy postępu w pliku *training.js* oraz pasek postępu.

Warstwa prezentacji obejmuje szablony Django (.html) oraz pliki statyczne (CSS, JS). Szablony wykorzystują dziedziczenie z *base.HTML* oraz Bootstrap dla stylów. Mają one strukturę kart i formularzy – na przykład *train_model.HTML* zawiera formularz wyboru folderów treningowych wygenerowany przez Django (pole `training_data`) oraz kontenery dla wykresów progresu. Wizualizacja danych w czasie rzeczywistym inicjalizowana jest w *training.js*). Pliki są następnie ładowane na stronach i łączą się z komponentami backendu poprzez endpointy typu REST.

Komunikacja między komponentami odbywa się więc poprzez:

- Wyświetlanie i obsługa formularzy Django (adresy URL definiowane w *urls.py*). Wywołania AJAX z frontendu do widoków Django (`start_training`, `get_training_progress`, `start_monitoring`, `get_realtime_data` itd.).
- Równoległe wątki do treningu modelu (funkcja `train_model_thread` wywoływana w tle przy użyciu `threading.Thread`).
- Zapis wyników do plików i bazy danych (modele `DiagnosticResult`, pliki CSV wyników monitoringowych).

Separator warstw ilustrują fragmenty kodu: w widoku `start_training` logika tworzenia i startu wątków jest oddzielona od warstwy prezentacji, a odpowiedź JSON informuje frontend o rozpoczęciu treningu. Natomiast prezentacja wyników w interfejsie (tablica wyników, wykresy, powiadomienia wizualne) jest realizowana wyłącznie po stronie frontendu przez JS, bez bezpośredniej ingerencji backendu.

5.2.3 Technologie, biblioteki i narzędzia

Wybór technologii, bibliotek i narzędzi wykorzystanych w projekcie systemu diagnostyki był podyktowany zarówno wymaganiami funkcjonalnymi i нефункциональными, jak i specyfiką zadania, a w szczególności koniecznością analizy danych audio w czasie rzeczywistym, przetwarzania sygnału, uczenia głębokiego oraz implementacji interaktywnego interfejsu użytkownika. Poniżej szczegółowo przedstawiono wszystkie najważniejsze komponenty technologiczne zastosowane w projekcie, z uwzględnieniem ich funkcji oraz uzasadnienia doboru.

System wykorzystuje następujące technologie i biblioteki:

- **Django (Python)** – główny framework webowy. Wykorzystywany jest standardowy stos Django: modele (*models.py*), formularze (*forms.py*), widoki (*views.py*), szablony (*templates/diagnostics/*.HTML*), system autentykacji. Przykładem jest klasa `DiagnosticConfigForm` definiowana w *forms.py* wbudowująca interfejs do wpisania parametrów diagnostyki.
- **Python** – język backendu, w nim zaimplementowano całą logikę. W kodzie używane są biblioteki systemowe i zewnętrzne.
- **HTML/CSS/Bootstrap** – służą do tworzenia interfejsów (szablony *home.HTML*, *monitoring.HTML*, *train_model.HTML*, *home_admin.HTML*, *home_pracownik.HTML* itd.).
- **JavaScript** – wspomaga interaktywność stron: pliki *monitoring.js* i *training.js* realizują dynamiczne zachowanie UI, wysyłają zapytania AJAX do backendu oraz odpowiadają za tworzenie wykresów. Przykładowo w *training.js* nasłuchuje zdarzenia `submit` formularza i uruchamia trening poprzez `fetch` do `/diagnostics/start_training/`.

- **Biblioteki DSP:** librosa (obliczanie MFCC, wczytywanie plików audio) oraz sounddevice (przechwytywanie dźwięku z mikrofonu). Przykład użycia: `mfcc = librosa.feature.mfcc(...)`.
- **TensorFlow/Keras** – główna biblioteka sztucznej inteligencji. Modele CNN są tworzone i trenowane przy użyciu Keras (funkcja `build_model(config)` w kodzie treningu), a następnie zapisywane za pomocą `model.save(...)`. W monitoringu ładowany jest model przez `tf.keras.models.load_model(model_path)`, a następnie wywoływana jest metoda `model.predict`.
- **Skrypty pomocnicze:** klasa SMOTE do oversamplingu w treningu, metryki Keras takie jak Accuracy czy AUC, własna funkcja `focal_loss` do nauki z balansem klas.
- **System kolejkowania/treningu:** zastosowano wątki Python (`threading.Thread`) do równoległego uruchomienia treningu modelu dla wybranych folderów z danymi. Wątek zapisuje postęp do pliku JSON (`training_progress.json`), skąd frontend odczytuje go przez API. Nie użyto zewnętrznej kolejki, ale prosty model wielowątkowy.
- **Formularze Django** – służą do walidacji i obsługi danych wejściowych. Przykładowo `TrainingDataForm` definiuje pole wyboru wielu folderów treningowych oraz nazwę maszyny, z walidacją unikalności i minimalnej liczby elementów. Formularz `DiagnosticConfigForm` (w *forms.py*) generuje pola konfiguracyjne AI (m.in.: liczbę epok, wielkość wsadu, hiperparametry CNN) wraz z widgetami i walidacjami.
- **Bazy danych i migracje Django** – modele (`Machine`, `DiagnosticConfig`, `DiagnosticResult`) są mapowane na tabele w relacyjnej bazie danych – domyślnie jest to SQLite. Framework Django automatycznie tworzy i waliduje te tabele.

Każda z wymienionych technologii ma swoje zastosowanie w konkretnych miejscach systemu. Na przykład librosa służy w widoku monitoringu (funkcje `get_realtime_data`) do obliczeń DSP, TensorFlow – w kodzie treningu

`train_model_thread` i inferencji w `get_realtime_data`, natomiast *forms.py* definiuje strukturę interfejsu konfiguracyjnego. Cały stos technologiczny tworzy spójny system, w którym komponenty Django (modele, widoki, szablony) współpracują z bibliotekami AI/DSP i kodem frontendu w JavaScript.

5.2.4 Model danych i struktura bazy danych

Zaprojektowanie spójnego modelu danych oraz odpowiedniej struktury bazy danych stanowiło istotny element pracy. Celem tego etapu było zapewnienie integralności przechowywania informacji niezbędnych do działania systemu, w tym danych konfiguracyjnych, wyników treningu, historii monitoringu oraz metadanych modeli uczenia maszynowego.

W ramach niniejszego projektu zdecydowano się na zastosowanie wbudowanej bazy danych SQLite, zintegrowanej z Django. Wybór ten był podyktowany lokalnym charakterem aplikacji, niewielką skalą danych oraz wymogiem prostoty wdrożenia – baza SQLite nie wymaga instalacji zewnętrznego serwera ani konfiguracji sieciowej, co pozwala na natychmiastową gotowość do działania nawet na komputerach bez dostępu do Internetu.

Model danych opiera się na trzech głównych klasach Django `Machine`, `DiagnosticConfig` i `DiagnosticResult`, zdefiniowanych w *models.py*. Klasa `Machine` reprezentuje pojedynczą maszynę użytkownika: zawiera pole `user` (`ForeignKey` do modelu użytkownika) i `name` (`string`), z zapewnieniem unikalności pary. Dzięki temu każdy użytkownik może mieć wiele maszyn, a nazwa jest unikatowa dla danego użytkownika.

Przykładowy fragment definicji modelu:

```
unique_together = ('user', 'name')
```

Klasa `DiagnosticConfig` przechowuje parametry konfiguracji diagnostyki dla danego użytkownika (`OneToOneField` do użytkownika) – dotyczy on całego systemu, np. progów `t_low`, `t_high`, `laeq_low`, `laeq_high`, czas trwania próbki, parametry uczenia itd.

Przykładem takiego pola jest:

```
t_low = models.FloatField(
    default=33,
    verbose_name="Dolny próg klasyfikacji [%]",
    validators=[MinValueValidator(0), MaxValueValidator(100)]
)
```

Model posiada także pola zaawansowane, np. `JSONField` `conv_filters`, `pooling_sizes`, `dense_units` przechowujące listy struktur sieci CNN. Wszystkie pola są wypełniane na podstawie formularza `DiagnosticConfigForm`. Ważną właściwością jest walidacja pól – np. funkcje `validate_conv_filters` sprawdzają poprawność list filtrów. Pole `machine_name` w `DiagnosticConfig` także jest unikatowe (domyślnie puste, nadpisywane przez formularz), co wspiera identyfikację konfiguracji z konkretną maszyną. Dodatkowo, `send_notification` określa czy wysyłać powiadomienia PUSH.

Klasa `DiagnosticResult` przechowuje pojedyncze wyniki diagnostyki: posiada klucz obcy `machine` do obiektu `Machine`, znacznik czasu `timestamp`, prawdopodobieństwo usterki, przewidzianą klasę oraz zmierzone poziomy akustyczne (pola `laeq`, `lapeak` mogą pozostać puste):

```
class DiagnosticResult(models.Model):
    machine = models.ForeignKey(Machine, on_delete=models.CASCADE,
    verbose_name="Maszyna")
    timestamp = models.DateTimeField(auto_now_add=True, verbose_name="Czas
    diagnostyki")
    probability = models.FloatField(verbose_name="Prawdopodobieństwo
    usterki")
    prediction = models.CharField(max_length=200,
    verbose_name="Predykcja")
    leq = models.FloatField(null=True, blank=True, verbose_name="Poziom
    dźwięku Leq")
    lapeak = models.FloatField(null=True, blank=True, verbose_name="Poziom
    dźwięku LApeak")
    )
```

Wyniki te zapisywane są w bazie za każdym przebiegiem monitoringu, a mogą być później przeglądane w panelu.

W modelu `DiagnosticConfig` ustawiono dodatkowe ograniczenia walidacyjne na pola, np. wartości liczbowe z odpowiednim zakresem (np. `MinValueValidator(0)` na procent), a także pola `num_conv_layers` itp., co ułatwia synchronizację z formularzem.

Część pól jest wyłączona z edycji (np. pola JSON z listami warstw), co zostało zrealizowane w `DiagnosticConfigForm` ukrywając je jako `HiddenInput`.

Przykładowe fragmenty kodu modeli z załączonych plików ilustrujące strukturę:

```
class Machine(models.Model): ... name = models.CharField(max_length=200,
verbose_name="Nazwa maszyny") ... unique_together = ('user', 'name')
class DiagnosticConfig(models.Model): ... t_low =
models.FloatField(default=33, verbose_name="Dolny próg klasyfikacji [%]"),
validators=[MinValueValidator(0), MaxValueValidator(100)]) ...
send_notification = models.BooleanField(default=False)
class DiagnosticResult(models.Model): machine =
models.ForeignKey(Machine,...), timestamp =
models.DateTimeField(auto_now_add=True)... probability =
models.FloatField(...), prediction = models.CharField(max_length=200) ...
```

Relacje między modelami są następujące: jeden Użytkownik (`User`) ma jeden lub wiele `Machine`; konfiguracja diagnostyczna (`DiagnosticConfig`) jest powiązana z użytkownikiem; każda maszyna (`Machine`) może mieć wiele wyników diagnostyki (`DiagnosticResult`) (klucz obcy w `DiagnosticResult`). Cały system zarządza także metadanymi treningu – po zakończonym trenowaniu modelu zapisywany jest plik JSON z parametrami (*machine_name_config.json* w folderze wyników) oraz model Keras. Te dane nie są relacyjne, ale wspierają proces inferencji (są wczytywane w `get_realtime_data`).

Struktura bazy danych odzwierciedla encje świata rzeczywistego (maszyny, konfiguracje, wyniki), z polem na konfigurację AI i relacją wyniki→maszyna. Pola są odpowiednio typowane (`FloatField`, `IntegerField`, `JSONField`, `BooleanField`) i zabezpieczone walidatorami (np. `MinValueValidator`).

5.3 Moduł akwizycji i przetwarzania danych akustycznych

W niniejszym rozdziale szczegółowo omówiono procesy związane z przechwytywaniem dźwięku oraz interpretacją plików audio w aplikacji. Moduł akwizycji danych został zaprojektowany z uwzględnieniem różnorodnych źródeł sygnałów – przechwytywanych bezpośrednio w czasie rzeczywistym z mikrofonu i wcześniej zarejestrowanych próbek audio. Do głównych zadań modułu należą: ekstrakcja cech diagnostycznych, ich normalizacja, a także przygotowanie danych do treningu modeli poprzez techniki augmentacji.

W kolejnych podrozdziałach przedstawiono szczegółowy opis wszystkich operacji przetwarzania sygnału akustycznego – począwszy od ekstrakcji cech, poprzez ich normalizację, aż po strategię rozszerzania zbioru danych.

5.3.1 Akwizycja danych z mikrofonu i plików

System diagnostyki akustycznej realizuje dwa tryby akwizycji danych: w czasie rzeczywistym (monitoring) oraz offline (treningowy). W trybie monitoringu stosowany jest mikrofon podłączony do karty dźwiękowej – sygnał jest rejestrowany za pomocą biblioteki sounddevice.

Fragment kodu pokazujący nagrywanie sygnału z mikrofonu:

```
audio = sd.rec(
    int(duration * sample_rate),
    samplerate=sample_rate,
    channels=1,
    dtype=np.float32,
    blocking=True,
    device=device_index if device_index is not None else None
)
sd.wait()
audio = audio.flatten()
```

Dane są nagrywane z zadaną częstotliwością próbkowania i w formacie float32, a następnie „spłaszczane” do wektora próbek. Po rejestracji następuje usunięcie wartości NaN i Inf (np.nan_to_num), co zabezpiecza przed błędnymi danymi. Jeśli amplituda sygnału okaże się ekstremalnie mała (np. poniżej $\sim 1e-6$), pomiar jest pomijany jako zbyt cichy.

Jeśli natomiast wykryje się wartości niedefiniowane, zwracany jest błąd:

```
if not np.all(np.isfinite(audio)):
    return JsonResponse({'status': 'error', 'message': 'Błąd sygnału...'})
```

W trybie offline (moduł treningowy) dane pochodzą z wcześniej nagranych plików audio przechowywanych na dysku.

Funkcja librosa.load ładuje zawartość pliku do wektora próbek przy zachowaniu oryginalnej częstotliwości próbkowania (sr=None):

```
audio, sr = librosa.load(file_path, sr=None)
```

W efekcie każdy plik jest wczytywany jako sygnał monofoniczny typu float (biblioteka librosa domyślnie łączy ewentualne kanały do mono). W obu trybach kod sprawdza poprawność danych wejściowych. Oprócz wspomnianej obsługi NaN/Inf, realizowane jest także przycinanie amplitud do dopuszczalnego zakresu.

Jeśli wartość bezwzględna sygnału przekracza 1.0 (pełną skalę float32), próbki są obcinane do [-1,1]):

```
audio = np.clip(audio, -1.0, 1.0)
```

Ponadto, fragmenty sygnału krótsze niż połowa oczekiwanego okna pomiarowego są pomijane zgodnie z logiką zapobiegającą analizie bardzo krótkich odcinków.

5.3.2 Obsługa formatów danych

System domyślnie obsługuje pliki audio w formacie WAV (PCM). Jest to świadomy wybór projektowy, podyktowany faktem, że format ten umożliwia bezstratne przechowywanie sygnału, a jednocześnie gwarantuje pełną kompatybilność z bibliotekami wykorzystywanymi do ekstrakcji cech i przetwarzania sygnału. Wskazują na to fragmenty kodu odpowiedzialne za wczytywanie plików.

Funkcja zwracająca listę dostępnych plików audio przeszukuje katalog i wybiera tylko pliki z rozszerzeniem WAV:

```
files = [
    f for f in os.listdir(folder)
    if f.lower().endswith('.wav') and os.path.isfile(os.path.join(folder,
f))
]
```

Analogicznie, moduł treningowy iteruje po podkatalogach *normal/* i *abnormal/*, ignorując pliki inne niż WAV (sprawdzenie `if file.endswith('.wav')`).

W rzeczywistości pliki WAV mogą mieć różną głębokość bitową (np. 16- lub 24-bit), ale po załadowaniu do pamięci sygnał traktowany jest jako dane float, co unifikuje reprezentację. Nagrania są analizowane w trybie monofonicznym: sounddevice rejestruje

tylko jeden kanał (`channels=1`), a `librosa.load` domyślnie konwertuje stereo do mono. W efekcie cała obróbka cech zakłada sygnał jednowymiarowy.

Jeśli chodzi o długość nagrań, w trybie monitoring dane dzielone są na krótkie okna określone parametrem `duration`. W trybie treningu nie ma wstępnego limitu długości pliku: każde nagranie jest przetwarzane w całości, a następnie – po wyznaczeniu cech MFCC – ujednolicane do stałej liczby ramek.

5.3.3 Ekstrakcja cech sygnału

Proces ekstrakcji cech sygnału akustycznego stanowi jedno z fundamentalnych ogniw systemu diagnostycznego, umożliwiające przekształcenie surowych próbek dźwiękowych w formę zrozumiałą i przetwarzalną przez model sztucznej inteligencji. W niniejszym systemie wyodrębniane są trzy główne typy cech: współczynniki MFCC, równoważny poziom dźwięku L_{Aeq} oraz szczytowy poziom dźwięku L_{Apeak} .

Główną reprezentacją cech akustycznych są współczynniki MFCC. Zarówno w trybie monitoringu, jak i treningu wykorzystywana jest funkcja `librosa.feature.mfcc`. Typowe parametry obliczeń (pochodzące z konfiguracji) to liczba cepstrów `n_mfcc`, rozmiar okna FFT `n_fft` oraz długość kroku `hop_length`.

W kodzie treningowym wywołanie funkcji MFCC wygląda to w następujący sposób:

```
mfcc = librosa.feature.mfcc(  
    y=signal,  
    sr=sr,  
    n_mfcc=config.n_mfcc,  
    n_fft=config.n_fft,  
    hop_length=config.hop_length,  
)
```

W rezultacie otrzymujemy macierz MFCC o wymiarach (`n_mfcc`, `liczba_ramek`). Aby zapewnić jednakową długość cech dla wszystkich próbek, każdą taką macierz dopasowuje się do zdefiniowanej liczby ramek `config.mfcc_length`.

Dokonuje się tego przy pomocy `librosa.util.fix_length`:

```
mfcc = librosa.util.fix_length(mfcc, size=config.mfcc_length, axis=1)
```


W trybie rzeczywistym, po zebraniu fragmentu sygnału również obliczane są MFCC – kod ten znajduje się wewnątrz funkcji pobierającej dane z mikrofonu i wygląda analogicznie:

```
mfcc = librosa.feature.mfcc(  
    y=audio,  
    sr=sample_rate,  
    n_mfcc=n_mfcc,  
    n_fft=n_fft,  
    hop_length=hop_length,  
    center=False  
)
```

Należy zwrócić uwagę, że przy `center=False` nie jest dokonywane przesunięcie sygnału, co zmniejsza wprowadzenie sztucznych ram na końcach okna.

Po obliczeniu współczynniki MFCC są normalizowane: w trybie monitoringu używane są średnia i odchylenie standardowe wyuczone na zbiorze treningowym, np.:

```
mfcc_normalized = (mfcc - mfcc_mean) / mfcc_std
```

W trybie treningowym cały zbiór MFCC jest standardyzowany w analogiczny sposób. Ostatecznie przygotowany tensor cech ma wymiary (`liczba_próbek`, `n_mfcc`, `config.mfcc_length`, 1).

Przykładowo, w kodzie trenowania następuje przekształcenie tablicy:

```
X_train = X_train.reshape((-1, config.n_mfcc, config.mfcc_length, 1))
```

Taki układ (kanał wymiaru 1) odpowiada obrazom MFCC używanym jako wejście do warstw konwolucyjnych sieci CNN. Poza MFCC w module akwizycji obliczane są także poziomy akustyczne L_{Aeq} i L_{Apeak} .

W trybie treningowym, podczas czytania plików audio, kod wylicza A -ważony poziom ekwiwalentny i szczytowy według wzorów:

```

p_ref = 20e-6
audio = np.nan_to_num(audio, nan=0.0, posinf=1.0, neginf=-1.0)

weighted_audio = apply_a_weighting(audio, sr)
weighted_audio = np.clip(weighted_audio, -1.0, 1.0)

rms = np.sqrt(np.mean(np.square(weighted_audio)))
LAeq = 10 * np.log10(np.mean(np.square(weighted_audio)) / (p_ref ** 2)) if
rms > 0 else 0.0

peak = np.max(np.abs(weighted_audio))
LApeak = 20 * np.log10(peak / p_ref) if peak > 1e-12 else 0.0

```

Wartości te są następnie zapisywane w listach dla całego zbioru danych:

```

LAeq_list.append(LAeq)
LApeak_list.append(LApeak)

```

Z tych wartości zestawów treningowych wyciągana jest średnia L_{Aeq} oraz maksymalna L_{Apeak} , które mogą posłużyć jako dodatkowe wskaźniki głośności. W sekcji monitoringu każdy fragment sygnału jest oceniany podobnie za pomocą funkcji `calculate_laeq` i `calculate_lapeak`. Wyniki nie trafiają bezpośrednio do sieci neuronowej jako cechy, ale służą do ustalania progów alarmów czy dodatkowych analiz.

5.3.4 Normalizacja i augmentacja danych

W systemach wykorzystujących uczenie maszynowe, przygotowanie danych wejściowych stanowi ważny etap całego procesu diagnostycznego. Szczególnie istotne jest to w przypadku danych akustycznych, które cechują się dużą zmiennością i wrażliwością na warunki rejestracji. Z tego względu przed przystąpieniem do treningu modelu sieci neuronowej, konieczne było opracowanie skutecznych metod wstępnego przetwarzania danych – w tym normalizacji współczynników cech oraz ich augmentacji.

Przygotowanie danych przed treningiem obejmuje standaryzację oraz augmentację. Surowe tablice MFCC normalizuje się przez odjęcie średniej i podzielenie przez odchylenie standardowe (tzw. standaryzacja).

Implementacja funkcji `normalize_data` oblicza statystyki zbioru uczącego i zwraca znormalizowane dane:

```
if mean is None or std is None:
    mean = np.mean(X)
    std = np.std(X)
normalized = (X - mean) / std
```

W wątku trenowania wywoływane jest m.in.:

```
X_train, mean, std = normalize_data(X_train)
X_val, _, _ = normalize_data(X_val, mean, std)
X_test, _, _ = normalize_data(X_test, mean, std)
```

Dzięki temu ten sam wektor (mean, std) jest stosowany do walidacyjnego i testowego zestawu. Dodatkowo w module akwizycji audio wszystkie zaburzenia (NaN/Inf) zostały wcześniej wyeliminowane, a amplitudy ograniczane do (-1,1), zapewniając spójność danych wejściowych.

Aby poprawić ogólną jakość i różnorodność zbioru treningowego, stosuje się augmentację sygnałów. Najczęściej, tak samo jak w niniejszym przypadku, polega to na generowaniu dodatkowych próbek akustycznych z oryginalnego dźwięku za pomocą losowych przekształceń. W implementacji dostępne są następujące operacje:

- **Dodanie szumu:** do sygnału oryginalnego dokłada się biały szum Gaussowski o odchyleniu `noise_addition_level`. Kod tworzy wektor `noise = np.random.normal(0, config.noise_addition_level, len(signal))` i sumuje go z sygnałem.
- **Zmiana wysokości tonu (pitch_shift):** przy użyciu `librosa.effects.pitch_shift` zmieniana zostaje tonacja dźwięku o losową wartość w zadanym zakresie `pitch_shift_range` (w półtonach).
- **Rozciąganie w czasie (time_stretch_probability):** sygnał jest przyspieszany lub zwalniany (`librosa.effects.time_stretch`) w losowym współczynniku z danego zakresu.
- **Zmiana głośności:** sygnał mnoży się przez współczynnik amplitudy odpowiadający parametrowi `volume_gain_range` (konwersja z *dB* na wartość amplitudy)

Wszystkie powyższe operacje realizowane są w funkcji `augment_audio_single`. Dla każdego pliku oznaczonego jako klasa usterkowa (*abnormal*) tworzy się listę `augmented_signals`, która zawiera oryginalny sygnał oraz `augmentation_multiplier` losowych wariantów uzyskanych z `augment_audio_single`. Każdy wariant jest następnie przetwarzany na macierz MFCC tak jak oryginał i dopisywany do zbioru X (z odpowiadającą mu etykietą y). W ten sposób zbiór treningowy zostaje wzbogacony o syntetyczne próbki o nieco zmienionych cechach, co poprawia ogólną generalizację modelu.

Przygotowane w ten sposób dane (macierze MFCC znormalizowane) służą jako wejście do architektury CNN opisanej w dalszych częściach pracy. Wszystkie te operacje, czyli: wczytanie sygnału, obliczenie cech, normalizacja i augmentacja zapewniają, że dane treningowe są spójne, odporne na błędy i reprezentatywne dla rzeczywistych warunków pomiaru.

5.4 Moduł sztucznej inteligencji

Moduł sztucznej inteligencji stanowi centralny komponent systemu diagnostycznego – to właśnie on odpowiada za właściwą interpretację sygnałów akustycznych w kontekście rozpoznawania potencjalnych nieprawidłowości w pracy maszyny. Jego podstawą jest wytrenowany model konwolucyjnej sieci neuronowej (CNN), który przekształca współczynniki MFCC pozyskane ze strumienia danych akustycznych w kategorie klasyfikacyjne, tj. *normal* lub różne typy zidentyfikowanych usterek.

Cały moduł został zaprojektowany tak, aby działał możliwie optymalnie, skutecznie i elastycznie dla użytkownika. Uwzględnia zarówno potrzeby dokładnej diagnozy, jak i możliwość łatwego dostosowania do różnych scenariuszy zastosowania. Jest w pełni zintegrowany z pozostałymi komponentami aplikacji – zarówno na etapie treningu, jak i podczas działania w czasie rzeczywistym.

5.4.1 Projekt architektury modelu CNN

Architektura modelu konwolucyjnej sieci neuronowej (CNN), zastosowana w systemie diagnostycznym, została zaprojektowana w sposób umożliwiający efektywne przetwarzanie sygnałów akustycznych reprezentowanych jako współczynniki cepstralne MFCC. Celem tej architektury jest nie tylko klasyfikacja stanu

maszyny jako *normal* lub odpowiadający jednemu z typów usterek, ale również zapewnienie wysokiej stabilności predykcji, odporności na zakłócenia oraz zdolności do generalizacji na nowe dane.

Architektura sieci CNN została zaimplementowana jako sieć sekwencyjna Keras. Wejście do sieci stanowi macierz współczynników MFCC o wymiarach (n_mfcc, mfcc_length) wzbogacona o wymiar kanału, co daje kształt (n_mfcc, mfcc_length, 1).

W kodzie zadeklarowano to w funkcji tworzącej model jako warstwę wejściową:

```
input_shape = (config.n_mfcc, config.mfcc_length, 1)
model = models.Sequential()
model.add(layers.Input(shape=input_shape, name='input_layer'))
```

Model składa się z kolejnych bloków konwolucyjnych. W pętli dla i od 0 do config.num_conv_layers-1 dodawane są warstwy Conv2D z liczbą filtrów config.conv_filters[i], rozmiarem jądra (config.kernel_size, config.kernel_size), funkcją aktywacji config.activation_function oraz regularyzacją L2 z czynnikiem config.kernel_regularizer.

Przykładowo, kod definiuje blok splotu następująco:

```
filters = config.conv_filters[i] if i < len(config.conv_filters) else 32
model.add(layers.Conv2D(
    filters=filters,
    kernel_size=(config.kernel_size, config.kernel_size),
    activation=config.activation_function,
    padding='same',
    kernel_regularizer=l2(config.kernel_regularizer)
))
```

W każdym bloku po warstwie konwolucyjnej opcjonalnie dodawana jest warstwa agregująca MaxPooling2D o rozmiarze config.pooling_sizes[i] (domyślnie 2×2). Dzięki niej zmniejszana jest rozdzielczość danych i redukowana liczba parametrów. Bezpośrednio po tej warstwie stosowana jest warstwa tzw. przestrzennego odrzucania neuronów (SpatialDropout2D) z wartością config.conv_dropout_rate, co przeciwdziała przeuczeniu, a także normalizacja wsadowa (BatchNormalization), poprawiająca stabilność uczenia.

Po przetworzeniu przez bloki splotu następuje etap agregacji cech. W zależności od wybranej w konfiguracji metody config.pooling_type wykorzystywana jest albo

klasyczna warstwa spłaszczająca Flatten, albo globalne uśrednienie (GlobalAveragePooling2D), albo wybór maksimum (GlobalMaxPooling2D). Daje to wektor cech gotowy do podania do kolejnych warstw gęstych. W końcowej części sieci następuje jeden lub więcej bloków gęstych. W pętli po `config.num_dense_layers` dodawana jest warstwa gęsta z liczbą neuronów `config.dense_units[i]` i funkcją aktywacji `config.activation_function`, po której występuje warstwa `Dropout(config.dense_dropout_rate)`.

Przykładowy fragment definiujący jedną warstwę gęstą:

```
units = config.dense_units[i] if i < len(config.dense_units) else
64model.add(layers.Dense(
    units=units,
    activation=config.activation_function,
    kernel_regularizer=l2(config.kernel_regularizer)
))
model.add(layers.Dropout(config.dense_dropout_rate))
```

Na samym końcu umieszczona jest warstwa wyjściowa Dense z liczbą neuronów równą liczbie klas `num_classes` i aktywacją softmax, co zapewnia probabilistyczne wyjście dla klasyfikacji wieloklasowej.

Upraszczając cały mechanizm można powiedzieć, że model przyjmuje obraz MFCC o wymiarach (`n_mfcc`, `mfcc_length`, 1), przetwarza go przez seryjnie ułożone bloki konwolucji→pooling→dropout→normalizacja, agreguje cechy, a następnie przechodzi przez jedną lub więcej warstw gęstych. Wyjściem modelu jest wektor rozkładu prawdopodobieństwa nad klasami wygenerowany przez aktywację softmax w ostatniej warstwie.

5.4.2 Proces treningu, walidacji i testu

Przed treningiem dane akustyczne są podzielone na zbiory: folder *train/*, *val/* oraz *test/*. Wewnątrz każdego z nich każdy stan (np. *normal*, *usterka_** itp.) reprezentowany jest osobnym podfolderem. W kodzie ścieżka `train_path` i `test_path` wskazują na te foldery. Funkcja `load_dataset(train_path, config)` wczytuje wszystkie próbki treningowe i zwraca macierze `x_train` (obrazy MFCC) oraz etykiety `y_train` (numery klas).

W dalszej kolejności dane treningowe dzielone są wewnętrznie na podzbiory treningowy i walidacyjny, co w kodzie realizuje funkcja `train_test_split`:

```
X_train = X_train.reshape((-1, config.n_mfcc, config.mfcc_length, 1))
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=42)
```

Warto zwrócić uwagę, że `X_train` przed dzieleniem jest przekształcany na kształt 4D (liczbapróbek, `n_mfcc`, `mfcc_length`, 1), aby pasował do wejścia konwolucyjnej sieci.

Etykiety klas są następnie konwertowane na postać tzw. one-hot:

```
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_val = tf.keras.utils.to_categorical(y_val, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)
```

Dane wejściowe są także normalizowane (odesłane do funkcji `normalize_data`), a w celu uzupełnienia niezrównoważonych klas użyta została technika SMOTE na zbiorze treningowym.

Model kompilowany jest z wykorzystaniem optymalizatora i funkcji straty wybranych w konfiguracji.

W kodzie wykorzystano dynamiczny wybór optymalizatora Keras (`config.optimizer`) zadaną szybkością uczenia `config.learning_rate`, funkcję straty typu `focal_loss` z parametrami `config.focal_loss_gamma` i `config.focal_loss_alpha`, oraz metryki takie jak dokładność i AUC:

```
optimizer = getattr(tf.keras.optimizers, config.optimizer)
(learning_rate=config.learning_rate)
loss_fn = focal_loss(gamma=config.focal_loss_gamma,
alpha=config.focal_loss_alpha)
metrics = [
    CategoricalAccuracy(name='accuracy'),
    AUC(name='auc')
]
model.compile(optimizer=optimizer, loss=loss_fn, metrics=metrics)
```

Trening przebiega z określoną liczbą epok `config.epochs` i rozmiarem partii `config.batch_size`. Dane walidacyjne przekazywane są do metody `fit` jako para (`X_val`, `y_val`) w parametrze `validation_data`.

Proces uczenia monitorowany jest za pomocą:

```
EarlyStopping(monitor='val_loss', patience=config.patience,  
restore_best_weights=True)
```

Zapewnia ono przerwanie treningu, gdy strata walidacyjna nie poprawia się przez daną liczbę epok (przywracane są wagi z najlepszej epoki), a także ReduceLRonPlateau obniżający współczynnik uczenia przy stagnacji oraz własną klasę StopTrainingCallback() umożliwiającą przerwanie treningu z interfejsu użytkownika.

Wywołanie treningu ma postać:

```
model.fit(  
    X_train_resampled,  
    y_train_resampled,  
    batch_size=config.batch_size,  
    epochs=config.epochs,  
    callbacks=[  
        progress_callback,  
        EarlyStopping(monitor='val_loss', patience=config.patience,  
            restore_best_weights=True),  
        ReduceLRonPlateau(monitor='val_loss', factor=0.5,  
            patience=config.patience//2, min_lr=1e-6),  
        StopTrainingCallback()  
    ],  
    class_weight=class_weight_dict,  
    validation_data=(X_val, y_val),  
    verbose=2  
)
```

Przez całą naukę zbierane są wartości strat i metryk dla zbiorów treningowego i walidacyjnego (m.in. loss, accuracy, precision, recall, f1, auc), które zapisywane są do pliku historii *history_*.csv* przez funkcję callbacku. W programie zastosowano mechanizm, który automatycznie wybiera model z epoki o najniższej stracie walidacyjnej (najlepszy).

Po zakończeniu treningu model zostaje oceniony na zbiorze testowym. Dla zbioru testowego przygotowuje się dane tak samo jak treningowe (przekształcenie do kształtu (liczba próbek, n_mfcc, mfcc_length, 1)), a następnie wywołuje `model.evaluate(X_test, y_test)` w celu obliczenia finalnych metryk: stratę testową i dokładność. Wykonywana jest również predykcja `model.predict(X_test)` i obliczane są metryki za pomocą wewnętrznej funkcji `_calculate_metrics`.

Ostateczne miary są zapisywane do pliku postępu (JSON) po zakończeniu treningu:

```
test_metrics = model.evaluate(X_test, y_test, verbose=0)
y_test_pred_raw = model.predict(X_test, verbose=0)
final_metrics = progress_callback._calculate_metrics(y_test,
y_test_pred_raw)
model.save(os.path.join(machine_dir, f"{machine_name}.keras"))

data.update({
    'test_loss': float(round(test_metrics[0], 4)),
    'test_accuracy': float(round(test_metrics[1], 4)),
    'test_f1': float(final_metrics['f1']),
    'test_auc': float(final_metrics['auc']),
    'test_precision': float(round(test_metrics[2], 4)) if
len(test_metrics)>2
else None,
    'test_recall': float(round(test_metrics[3], 4)) if len(test_metrics)>3
else None
})
```

W ten sposób końcowy model (plik **.keras*) wraz z metadanymi i zapisanymi wynikami (plik JSON, CSV) jest dostępny do dalszych etapów (np. monitoringu w czasie rzeczywistym).

5.4.3 Dobór parametrów

System umożliwia eksperymentalne dobieranie wielu parametrów wpływających na architekturę sieci, ekstrakcję cech oraz proces uczenia. Parametry te definiowane są w modelu DiagnosticConfig i przekazywane w obiekcie config. Kluczowe przykłady to:

- **Parametry ekstrakcji MFCC:** `n_mfcc` (liczba współczynników MFCC) oraz `n_fft` (rozmiar okna FFT) określają jak przekształcamy sygnał akustyczny w cechy.

W kodzie są one definiowane jako pola modelu:

```
n_mfcc = models.IntegerField(default=20)
n_fft = models.IntegerField(default=2048)
```

Wartości te wpływają na kształt macierzy wejściowej i czułość analizy częstotliwościowej. Zwiększenie `n_mfcc` pozwala uchwycić więcej

szczegółów spektralnych kosztem wzrostu wymiaru wejścia, a `n_fft` decyduje o rozdzielczości FFT i długości ramki.

- **Liczba warstw i filtrów:** Pola `num_conv_layers` i `conv_filters` definiują głębokość sieci konwolucyjnej. Na przykład domyślnie `num_conv_layers = 3` oraz `conv_filters = [32, 32, 32]`. Użytkownik może eksperymentalnie zmieniać liczbę bloków konwolucyjnych i odpowiadającą im liczbę filtrów dla kolejnych warstw, co wpływa na pojemność modelu. Większa liczba filtrów zwykle zwiększa zdolność sieci do wyodrębniania cech, ale może prowadzić do przeuczenia i zwiększa wymagania obliczeniowe.
- **Rozmiar jądra konwolucji:** Parametr `kernel_size` (domyślnie 3) określa wymiar filtru w warstwie Conv2D. Definiowany jest jako: `kernel_size = models.IntegerField(default=3)`. W kodzie wartość ta jest przekazywana jako `(config.kernel_size, config.kernel_size)` przy tworzeniu warstwy konwolucyjnej. Większe jądra (np. 5×5) mogą wychwycić cechy globalne, ale zwiększają liczbę parametrów; mniejsze jądra (3×3) faworyzują cechy lokalne i są lżejsze obliczeniowo.
- **Współczynniki Dropout:** W celu regularyzacji dodano dwa parametry: `conv_dropout_rate` i `dense_dropout_rate` – współczynniki losowego wyłączania neuronów w warstwach konwolucyjnych i gęstych. Są one używane odpowiednio w `SpatialDropout2D(config.conv_dropout_rate)` po warstwach konwolucyjnych i w `Dropout(config.dense_dropout_rate)` w części gęstej. Eksperymenty z różnymi wartościami tego parametru wykazują, że zbyt duży dropout może ograniczyć zdolność sieci (niedouczenie), natomiast zbyt mały może skutkować przeuczeniem.
- **Uczenie:** Parametry uczenia to przede wszystkim `learning_rate` i optymalizator (np. Adam, SGD). W konfiguracji `learning_rate` ma domyślnie 0,001, natomiast wybrany optymalizator jest inicjalizowany np. jako: `Adam(learning_rate=config.learning_rate)`. Wysoka wartość tego parametru może przyspieszyć zbieżność, ale też powodować niestabilność; niski może wymagać więcej epok do spadku straty.

Pozostałe dobierane parametry to m.in. `batch_size` i `epochs` (rozmiar partii i liczba epok; w domyśle 64 i 30) oraz parametry funkcji strat (`focal_loss_alpha`, `focal_loss_gamma`). Wszelkie wartości ustawione przez użytkownika są zapisywane w pliku konfiguracyjnym i przekazywane do budowy modelu (np. `config.n_mfcc`, `config.kernel_size`) lub do procedury treningowej (np. `config.learning_rate` przy tworzeniu optymalizatora). Dobór tych parametrów był dokonywany eksperymentalnie: obserwowano, że np. zwiększenie liczby warstw konwolucyjnych i filtrów potrafi poprawić skuteczność, ale wymagało silniejszej regularyzacji (dropout, normalizacja) oraz dłuższego treningu. Parametry te można wielokrotnie modyfikować z poziomu konfiguratora systemu, co pozwala na śledzenie ich wpływu na dokładność klasyfikacji.

5.4.4 Wybór i implementacja metryk

W opracowanym systemie aspekt zastosowania odpowiednich metryk oceny skuteczności modelu klasyfikacyjnego jest bardzo istotny, ponieważ umożliwia obiektywną, mierzalną i porównywalną ewaluację działania modelu na danych, których wcześniej nie widział. Same wartości straty nie zawsze oddają rzeczywistą jakość klasyfikacji, szczególnie w przypadku niezbalansowanych zbiorów danych. Właściwy dobór i interpretacja metryk pozwala nie tylko wykryć przeuczenie lub niedouczenie modelu, ale także wskazuje, w jakim stopniu model radzi sobie z różnymi klasami oraz jakie może mieć ograniczenia w praktycznych zastosowaniach.

1) Macierz pomyłek

Podstawą do obliczenia większości metryk klasyfikacyjnych jest macierz pomyłek, która dla problemu dwuklasowego przyjmuje postać:

Tab. 1. Struktura macierzy pomyłek dla klasyfikacji binarnej

	Przewidziano pozytywną	Przewidziano negatywną
Rzeczywista pozytywna	True Positive (TP)	False Negative (FN)
Rzeczywista negatywna	False Positive (FP)	True Negative (TN)

Dla klasyfikacji wieloklasowej, macierz pomyłek rozszerza się do postaci kwadratowej macierzy o wymiarach $n \times n$, gdzie n to liczba klas.

2) Dokładność (Accuracy)

Dokładność jest podstawową miarą skuteczności klasyfikatora, definiowaną jako stosunek liczby poprawnych predykcji do całkowitej liczby przypadków:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

W kontekście klasyfikacji wieloklasowej, dokładność oblicza się jako:

$$Accuracy = \frac{\sum TP_i}{\sum (TP_i + FP_i + FN_i)}$$

Gdzie n to liczba klas, a TP_i , FP_i , FN_i to odpowiednio liczby prawdziwych pozytywnych, fałszywych pozytywnych i fałszywych negatywnych dla klasy i .

Dokładność jest intuicyjna i łatwa do interpretacji, jednak w przypadku nie zrównoważonych zbiorów danych może być myląca, ponieważ wysoka dokładność może wynikać z dominacji jednej klasy.

3) Precyzja (Precision)

Precyzja mierzy, jaki odsetek przewidzianych pozytywnych przypadków jest rzeczywiście pozytywny.

Dla klasyfikacji wieloklasowej, precyzję oblicza się dla każdej klasy osobno, a następnie można uśrednić wyniki

$$Precision = \frac{TP}{TP + FP}$$

- **Macro-average Precision:** Średnia arytmetyczna precyzji dla wszystkich klas.
- **Micro-average Precision:** Obliczana na podstawie sumy TP i FP dla wszystkich klas.

Precyzja jest szczególnie istotna w kontekstach, gdzie koszt fałszywych pozytywów jest wysoki.

4) Czulość (Recall)

Czulość, znana również jako trafność lub True Positive Rate (TPR), mierzy zdolność modelu do wykrywania wszystkich rzeczywistych pozytywnych przypadków.

Podobnie jak w przypadku precyzji, dla klasyfikacji wieloklasowej czulość oblicza się dla każdej klasy osobno, z możliwością uśrednienia wyników:

$$Recall = \frac{TP}{TP + FN}$$

- **Macro-average Recall:** Średnia arytmetyczna czulości dla wszystkich klas.
- **Micro-average Recall:** Obliczana na podstawie sumy TP i FN dla wszystkich klas.

Wysoka czulość jest kluczowa w aplikacjach, gdzie pominięcie pozytywnego przypadku (fałszywy negatyw) jest kosztowne.

5) Miara F1 (F1-score)

Miara F1 jest średnią harmoniczną precyzji i czulości, stanowiąc kompromis między tymi dwoma metrykami.

Dla klasyfikacji wieloklasowej, miarę F1 oblicza się dla każdej klasy osobno, a następnie uśrednia:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- **Macro-average F1 Score:** Średnia arytmetyczna miar F1 dla wszystkich klas.
- **Micro-average F1 Score:** Obliczana na podstawie sumy TP , FP i FN dla wszystkich klas.

Miara F1 jest szczególnie użyteczna w przypadkach nieźrównoważonych zbiorów danych, gdzie zarówno precyzja, jak i czulość są istotne.

6) Pole pod krzywą ROC (AUC)

Krzywa ROC (ang. *Receiver Operating Characteristic*) przedstawia zależność między czułością a współczynnikiem fałszywych pozytywów:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

AUC (ang. *Area Under the Curve*) mierzy zdolność modelu do rozróżniania między klasami. W przypadku klasyfikacji wieloklasowej stosuje się podejście, gdzie dla każdej klasy oblicza się AUC, traktując ją jako pozytywną, a pozostałe jako negatywne. Następnie wyniki są uśredniane:

- **Macro-average AUC** – Średnia arytmetyczna AUC dla wszystkich klas.
- **Micro-average AUC** – Obliczana na podstawie sumy TP , FP , TN i FN dla wszystkich klas.

AUC jest szczególnie przydatną metryką w ocenie modeli, gdzie ważna jest zdolność do rozróżniania między klasami, niezależnie od wybranego progu decyzyjnego [28].

W implementacji użyto biblioteki sklearn. W klasie `TrainingProgressCallback` po każdej epoce wywoływane jest przewidywanie modelu dla zbiorów treningowego i walidacyjnego, a następnie funkcje `precision_score`, `recall_score` i `f1_score` obliczają metryki na twardych etykietach.

Przykład fragmentu kodu pokazuje implementację tych obliczeń:

```
y_true_labels = np.argmax(y_true, axis=1)
y_pred_labels = np.argmax(y_pred_probs, axis=1)
precision = precision_score(y_true_labels, y_pred_labels,
average='macro', zero_division=0)
recall = recall_score(y_true_labels, y_pred_labels, average='macro',
zero_division=0)
f1 = f1_score(y_true_labels, y_pred_labels, average='macro',
zero_division=0)
return {'f1': f1, 'auc': auc, 'precision': precision, 'recall':
recall}
```

Wyniki metryk są systematycznie zapisywane do plików logów.

Callback `TrainingProgressCallback` generuje po każdej epoce wpis w pliku CSV `history_*.csv` zawierający m.in. pola `train_accuracy`, `train_precision`, `train_recall`, `train_f1` oraz analogiczne pola `val_accuracy`, `val_precision`, `val_recall`, `val_f1`. Po zakończeniu treningu ostateczne metryki dla zbioru testowego są zapisywane do pliku postępu w formacie JSON, dzięki czemu wyniki są udokumentowane po zakończeniu uczenia modelu.

5.5 Moduł monitorowania w czasie rzeczywistym

Monitoring w czasie rzeczywistym stanowi obok modułu treningowego najważniejszy komponent całego systemu diagnostycznego. Jego zadaniem jest ciągła analiza sygnału akustycznego zbieranego z mikrofonu, przetwarzanie go, klasyfikacja w czasie rzeczywistym oraz wizualizacja i archiwizacja wyników diagnostycznych.

Moduł ten działa w sposób ciągły i cykliczny, z ustalonym interwałem czasowym próbkowania, której długość definiowana jest przez użytkownika w formularzu konfiguracji. Każda próbka podlega natychmiastowej transformacji – najpierw poprzez ekstrakcję cech akustycznych, a następnie poprzez klasyfikację przy użyciu wcześniej wytrenowanego modelu CNN. Wyniki klasyfikacji wykorzystywane są do określenia bieżącego statusu pracy maszyny, który może przyjmować wartości: praca normalna (*normal*), podejrzenie usterki (*warning*) lub usterka (*fault*). System został tak zaprojektowany, aby interpretacja wyników była możliwa bezpośrednio i czytelnie, zarówno w trybie klasyfikacji binarnej, jak i wieloklasowej na podstawie zdefiniowanych progów prawdopodobieństwa.

5.5.1 Obsługa sesji i strumieniowania danych

Aplikacja realizuje połączenie sesyjne między frontendem a backendem za pomocą standardowych mechanizmów Django (sesja HTTP) oraz asynchronicznych zapytań AJAX. W kodzie backendu wykorzystuje się obiekt `request.session`, który pełni rolę tymczasowego repozytorium stanu monitoringu. W sesji przechowywane są m.in. następujące zmienne kontekstowe:

- **full_audio** – pełny bufor próbek audio z mikrofonu, aktualizowany przy każdym wywołaniu funkcji `get_realtime_data`. Dzięki przesuwaniu bufora możliwa jest analiza ciągłego sygnału w ramach całego okna czasowego.

- **DiagnosticConfig** – słownik parametrów konfiguracyjnych monitoringu (m.in. `duration`, `sample_rate`, progi `t_low`, `t_high` oraz inne wartości przetwarzania sygnału).
- **Model** – obiekt wytrenowanego modelu sieci CNN (TensorFlow/Keras) załadowany do sesji podczas uruchomienia monitoringu. Model ten jest wykorzystywany wielokrotnie bez ponownego ładowania, co skraca czas inferencji.
- **classification_type** – typ klasyfikatora wskazujący czy modele rozróżniają stan normalny/usterka czy też dodatkowe klasy usterek.
- **class_indices** – słownik mapujący indeksy wyjść modelu na nazwy klas, wykorzystywany do etykietowania wyników predykcji.

Strumieniowanie danych odbywa się w pętli – frontend okresowo wysyła zapytania do serwera co ustalony interwał czasowy (1 sekunda) za pomocą AJAX. Wywoływana jest wówczas funkcja widoku `get_realtime_data` (z `views_monitoring.py`), która realizuje odczyt kolejnego fragmentu dźwięku, aktualizuje bufor sesji oraz przetwarza nowe dane. W szczególności funkcja ta wykonuje następujące operacje:

- Odczyt nowej próbki audio z mikrofonu (lub z pliku) i dopisanie jej do bufora `full_audio`. Bufor jest przesuwany o starą próbkę przy każdym wywołaniu, co pozwala na ciągłą analizę (ang. *rolling window*) sygnału.
- Po osiągnięciu pełnej długości okna (liczonej jako `duration * sample_rate`), przekazanie zebranej ramki do ekstrakcji cech.
- Klasyfikacja zebranych cech przez model CNN oraz interpretacja wyniku (status maszyny).
- Zapis wyników do pliku CSV, czyli logowanie pomiaru i przygotowanie odpowiedzi JSON dla użytkownika.

Poniżej znajduje się fragment kodu klienta z pliku `monitoring.js`, odpowiedzialny za pobieranie i przetwarzanie danych w czasie rzeczywistym:


```

const startMonitoring = async () => {
  if (!elements.machineSelect.value) {
    alert('Wybierz maszynę z listy!');
    return;
  }

  try {
    isMonitoring = true;
    window.addEventListener('beforeunload', handleBeforeUnload);

    elements.startBtn.disabled = true;

    if (monitoringInterval) {
      clearInterval(monitoringInterval);
      abortController.abort();
      abortController = new AbortController();
    }

    const classificationType =
      document.getElementById('classificationTypeSelect')?.value || 'multiclass';
    classificationType);
    document.getElementById('classificationTypeSelect').value =
      classificationType;

    const formData = new URLSearchParams();
    const selectedAudio = document.getElementById('audioFileSelect')?.value ||
      '';
    formData.append('audio_file', selectedAudio);
    formData.append('machine_name', elements.machineSelect.value);
    formData.append('classification_type', classificationType);

    const response = await fetch(API_ENDPOINTS.start, {
      method: 'POST',
      headers: {
        'X-CSRFToken':
          document.querySelector('[name=csrfmiddlewaretoken]').value,
        'Content-Type': 'application/x-www-form-urlencoded'
      },
      body: formData.toString()
    });
  }

```

W odpowiedzi z serwera, którą frontend parsuje, znajduje się obiekt JSON zawierający zaktualizowane wyniki (np. współczynniki MFCC, status pracy maszyny, komunikaty tekstowe). W ten sposób interfejs jest na bieżąco synchronizowany z backendem, a użytkownik widzi płynnie aktualizujące się wykresy i alerty.

5.5.2 Analiza danych pomiarowych i klasyfikacja

Moduł analizy danych pomiarowych oraz klasyfikacji w czasie rzeczywistym stanowi kluczowy komponent systemu diagnostycznego. Jego głównym celem jest przekształcenie zarejestrowanych sygnałów akustycznych w postać numeryczną możliwą do interpretacji przez wytrenowaną sieć neuronową, a następnie dokonanie

predykcji aktualnego stanu technicznego urządzenia. Cały proces jest realizowany cyklicznie i przebiega w kilku następujących etapach:

- Pobranie bufora `full_audio` z pamięci podręcznej `AUDIO_CACHE`,
- Wyodrębnienie fragmentu sygnału o długości `duration * sample_rate`,
- Przekazanie sygnału do funkcji `librosa.feature.mfcc()` w celu wyznaczenia współczynników MFCC,
- Obliczenie współczynników MFCC – numerycznej reprezentacji spektralnych cech sygnału,
- Wyznaczenie wartości L_{Aeq} przy użyciu funkcji `calculate_lapeak(audio, sample_rate)` – wskaźnika całkowitej energii akustycznej,
- Wyznaczenie wartości L_{Apeak} przez funkcję `calculate_lapeak()` – maksymalnej amplitudy sygnału w filtracji A ,
- Zgrupowanie wszystkich parametrów w słowniku danych pomiarowych i przekazanie ich do dalszej analizy,
- Sprawdzenie typu klasyfikacji i obecności metadanych normalizacji (`mean`, `std`) w pliku `*_config.json`,
- Przeprowadzenie normalizacji MFCC,
- Ujednolicenie skali cech wejściowych oraz dopasowanie długości MFCC do wymaganego `input_shape`.

Na podstawie wektora prawdopodobieństw zwróconego przez model sieci neuronowej, system klasyfikacyjny interpretuje aktualny stan techniczny urządzenia, przypisując jeden z trzech możliwych statusów diagnostycznych. Logika ta zależy od trybu klasyfikacji (`classification_type`) oraz wartości progowych `t_low` i `t_high`, zdefiniowanych w konfiguracji `DiagnosticConfig`.

5.5.2.1 Tryb klasyfikacji: binarny

W trybie binarnym zakłada się istnienie dwóch kategorii: *normal* oraz *abnormal*. Model zwraca wektor prawdopodobieństw:

$$y_{predprobs} = [p^0, p^1, \dots, p_n]$$

gdzie p_i to prawdopodobieństwo przynależności do klasy i . System oblicza sumaryczne prawdopodobieństwo klas innych niż *normal* według wzoru:

$$p_{abnormal} = \sum (p_i) \text{ dla wszystkich } i \neq i_{normal},$$

gdzie i_{normal} to indeks klasy oznaczonej jako *normal*. Status diagnostyczny przypisywany jest według wartości $p_{abnormal}$:

- **Status Praca normalna:** jeśli $p_{abnormal} \leq t_{low}$
- **Status Podejrzenie usterki:** jeśli $t_{low} < p_{abnormal} \leq t_{high}$
- **Status Usterka:** jeśli $p_{abnormal} > t_{high}$

```
fault_prob = float(np.sum(np.delete(prediction, normal_idx)))
```

5.5.2.2 Tryb klasyfikacji: wieloklasowy

W tym trybie system wybiera klasę o najwyższym prawdopodobieństwie. Jeżeli przewidywana klasa to *normal*, wówczas brana jest pod uwagę wartość prawdopodobieństwa tej klasy.

Status określany jest według progów t_{low} i t_{high} :

- **Status Praca normalna:** jeśli $p_{normal} > t_{high}$
- **Status Podejrzenie usterki:** jeśli $t_{low} < p_{normal} \leq t_{high}$
- **Status Usterka:** jeśli $p_{normal} \leq t_{low}$

```
predicted_class = class_names[predicted_index]
predicted_prob = float(prediction[predicted_index])
```

Jeśli natomiast wybrana klasa nie jest klasą *normal*, to niezależnie od wartości liczbowej przypisywany jest status Usterka, a użytkownik otrzymuje dodatkowo informację o konkretnej nazwie przewidywanej klasy usterki.

Po wykonaniu klasyfikacji system zapisuje pełny zestaw wyników pomiaru do pliku CSV.

Operacja ta realizowana jest za pomocą funkcji `save_monitoring_result`, która dopisuje nowy wiersz zawierający:

- czas pomiaru (`Time`),
- nazwę monitorowanego urządzenia (`Device`),
- tryb klasyfikacji (`ClassificationType`),
- status diagnostyczny (`Status`), określany na podstawie wartości progowych jako *normal*, *warning* lub *fault*,
- nazwę przewidywanej klasy (`PredictedClass`),
- prawdopodobieństwo przypisane do tej klasy (`PredictedProbability`),
- szczytowy poziom dźwięku A (L_{Apeak}),
- równoważny poziom dźwięku A (L_{Aeq}).

Zapisane dane służą do późniejszej analizy działania systemu diagnostycznego, monitorowania zmian akustycznych oraz oceny skuteczności modelu klasyfikacyjnego w rzeczywistych warunkach pracy urządzenia.

Funkcja `get_realtime_data` kończy swoje działanie poprzez zwrócenie obiektu `JsonResponse` z aktualnym statusem (`status`), wartością wyświetlaną (`display_status`), wartościami liczbowymi (np. `predicted_probability`, `lapeak`, `leq`) oraz tablicą `mfcc_array`. Interfejs użytkownika aktualizuje dane w czasie rzeczywistym, dzięki czemu użytkownik może natychmiast śledzić zmiany akustyczne zachodzące w pracy maszyny.

5.5.3 System powiadomień i alertów

W celu zapewnienia pełnej funkcjonalności systemu diagnostycznego z perspektywy obsługi przez wielu użytkowników, zdecydowano się uzupełnić system o powiadomienia o stanie maszyny. Głównym założeniem było umożliwienie użytkownikowi otrzymywania natychmiastowych informacji o wykrytych

nieprawidłowościach, także w sytuacjach, gdy interfejs aplikacji nie jest aktywnie obserwowany.

System powiadomień został zaprojektowany tak, aby spełniać następujące wymagania:

- informowanie użytkownika w czasie rzeczywistym o wykrytej usterce,
- niezależność od przeglądarki internetowej – powiadomienia mogą być odbierane na urządzeniach mobilnych i komputerach,
- prostota konfiguracji – użytkownik może włączyć lub wyłączyć powiadomienia jednym parametrem,
- niezawodność i szybkość działania dzięki integracji z gotowym API.

W konfiguratorze systemu znajduje się funkcja `send_notification` z pliku `views_notification.py`, który stanowi główny przełącznik aktywujący system powiadomień. Wartość tego parametru jest zapisywana w obiekcie konfiguracji (`DiagnosticConfig`) i odczytywana podczas działania modułu monitoringu. Użytkownik może dodatkowo aktywować parametr `notify_on_warning`, który umożliwia wysyłanie alertów również w przypadku stanu ostrzegawczego.

Jeśli `send_notification` jest ustawiony na `True`, system wysyła powiadomienie za każdym razem, gdy wykryty status to usterka lub jej podejrzenie (zgodnie z konfiguracją progową i typem klasyfikacji).

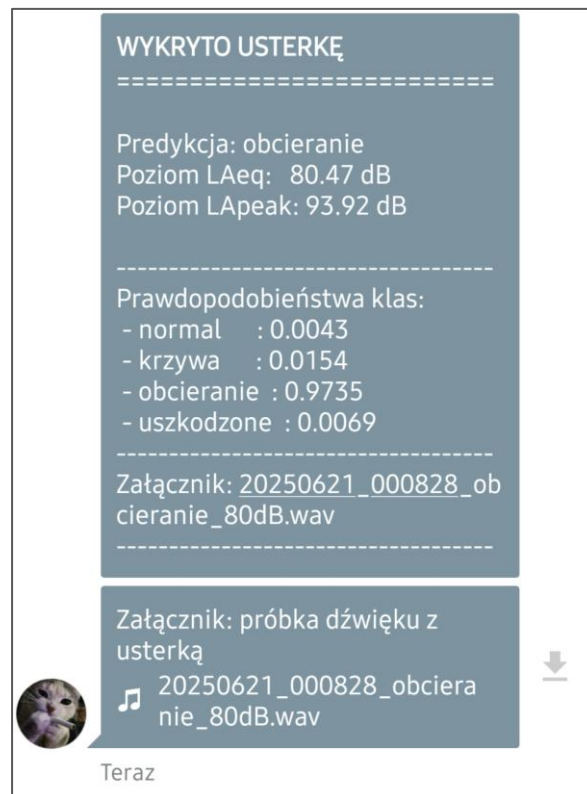
Do realizacji systemu powiadomień wykorzystano usługę `Pushbullet`, która umożliwia przesyłanie komunikatów `push` na urządzenia użytkownika. Funkcja przyjmuje trzy argumenty:

- `title` – tytuł powiadomienia,
- `body` – treść komunikatu diagnostycznego,
- `file_path` – ścieżka do pliku `WAV` z nagraniem próbki dźwiękowej.

Treść wiadomości generowana jest dynamicznie i zawiera szczegółowe informacje diagnostyczne.

W zależności od typu klasyfikacji przyjmuje jedną z poniższych form:

```
if classification_type == "binary":
    body = (
        "=====\n\n"
        f"Prawdopodobieństwo usterki: {predicted_prob:.4f}\n"
        f"Poziom LAeq: {current_leq:.2f} dB\n"
        f"Poziom LApeak: {lapeak:.2f} dB\n\n"
        "-----\n"
        f"Załącznik: {attachment_id}\n"
        "-----"
    )
else:
    probs_text = "\n".join(
        f" - {label:<12}: {float(prob):.4f}" for label, prob in
zip(class_names, prediction)
    )
    body = (
        "=====\n\n"
        f"Predykcja: {predicted_class}\n"
        f"Poziom LAeq: {current_leq:.2f} dB\n"
        f"Poziom LApeak: {lapeak:.2f} dB\n\n"
        "-----\n"
        "Prawdopodobieństwa klas:\n"
        f"{probs_text}\n"
        "-----\n"
        f"Załącznik: {attachment_id}\n"
        "-----"
    )
)
```



Rys. 3. Przykładowe powiadomienie wysłane przez system za pomocą API Pushbullet

Zawartość powiadomienia zawiera zatem nie tylko decyzję diagnostyczną, ale również szczegółowe dane oraz pełen rozkład prawdopodobieństw dla wszystkich klas (w trybie wieloklasowym). Pozwala to użytkownikowi samodzielnie ocenić ryzyko i charakter usterki.

Wywołanie funkcji odbywa się w widoku `get_realtime_data`, pod warunkiem spełnienia warunku:

```
if config.send_notification and (status == "fault" or (status == "warning"
and config.notify_on_warning)):
    send_notification(title, body, file_path=save_path)
```

Funkcja `get_realtime_data` zwraca także odpowiedź w formacie JSON z aktualnymi wynikami klasyfikacji. Odbierana jest ona przez kod *monitoring.js*, który na bieżąco aktualizuje kolory i komunikaty interfejsu, wyświetlając ostrzeżenia i zmieniając status. W przypadku przywrócenia prawidłowego działania maszyny, system samoczynnie resetuje alarm.

5.6 Interfejs użytkownika i konfiguracja systemu

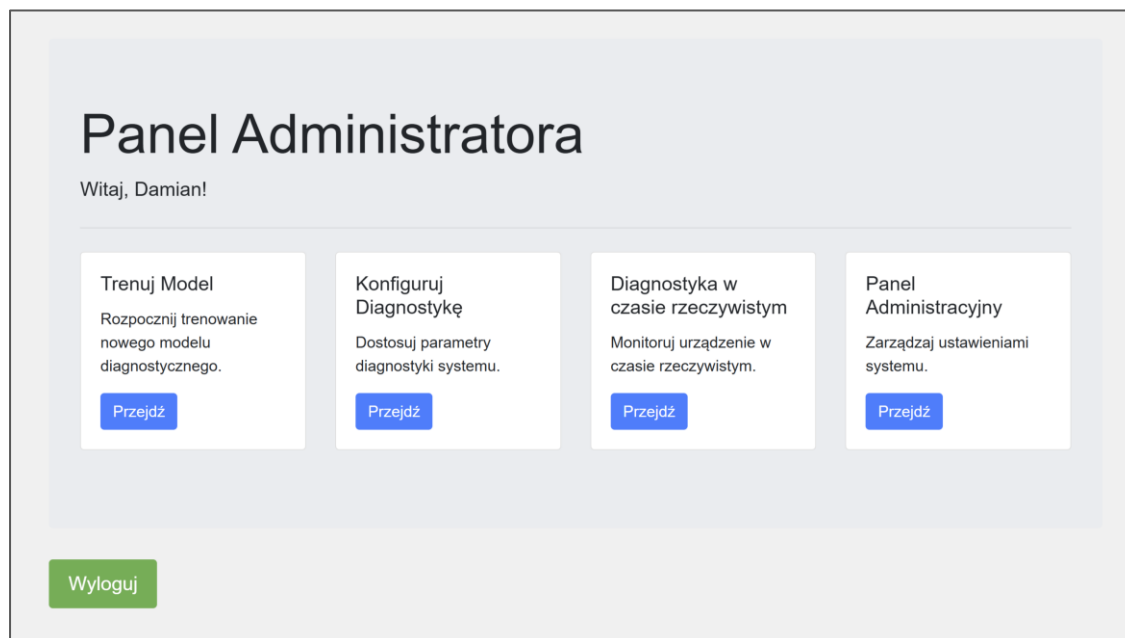
W projekcie wzięto po uwagę również aspekt wyglądu projektowanego systemu diagnostyki akustycznej maszyn w celu zapewnienia przejrzystego i funkcjonalnego interfejsu użytkownika, który umożliwiałby zarówno łatwą konfigurację działania systemu, jak i kontrolę nad jego pracą w czasie rzeczywistym. Interfejs stanowi połączenie warstwy prezentacji a logiki aplikacji i realizuje większość interakcji użytkownika z systemem.

Interfejs zbudowany został z wykorzystaniem szablonów HTML oraz dynamicznych komponentów JavaScript. Obsługuje różne role użytkowników (m.in. administratora i pracownika) oraz oferuje dwa podstawowe panele funkcjonalne: Konfigurację diagnostyki i Monitoring. Każdy z nich został zaprojektowany z myślą o ergonomii i czytelności, a podział na zakładki ułatwia orientację w dużej liczbie dostępnych parametrów.

W niniejszej części opisano zarówno funkcje dostępne dla użytkownika, jak i strukturę formularza konfiguracyjnego. Szczególną uwagę poświęcono mechanizmowi renderowania pól formularza, logice walidacji danych wejściowych oraz integracji konfiguracji z backendem systemu.

5.6.1 Panel użytkownika i jego funkcje

Panel główny jest punktem startowym dla użytkownika. Zawiera zakładki prowadzące do kluczowych modułów.



Rys. 4. Główny panel administratora systemu diagnostycznego

Fragment kodu HTML definiującego je wygląda następująco – przykład dla zakładki treningu:

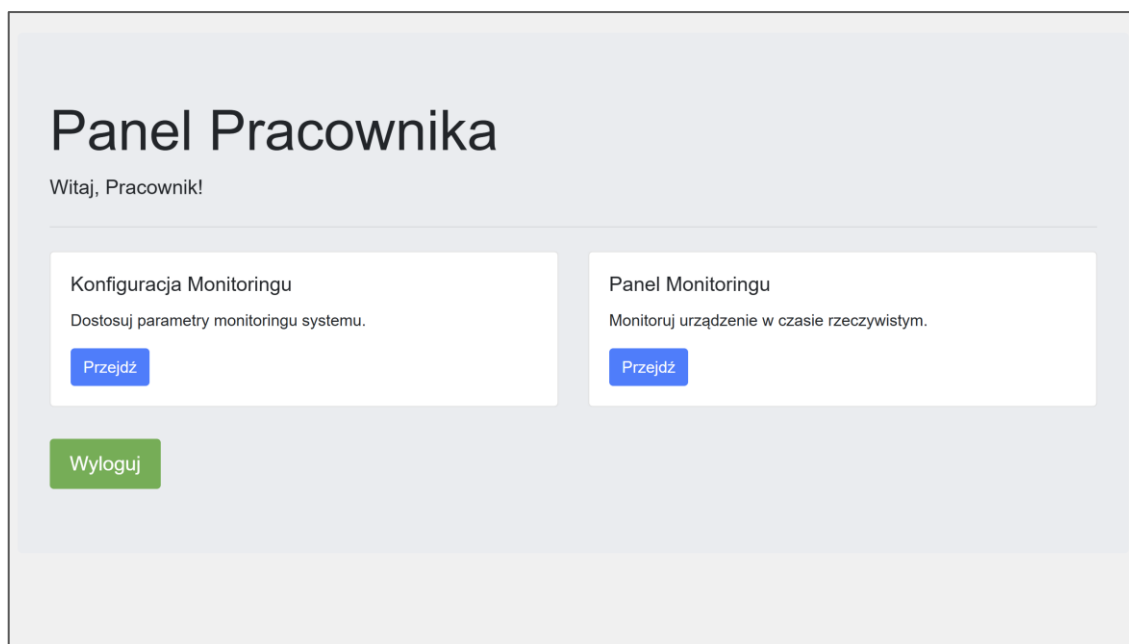
```
<div class="card">
  <div class="card-body">
    <h5 class="card-title">Trenuj Model</h5>
    <p class="card-text">Rozpocznij trenowanie nowego
modelu diagnostycznego.</p>
    <a href="{% url 'diagnostics:train' %}" class="btn
btn-primary">Przejdź</a>
  </div>
</div>
```

Analogicznie zdefiniowane są pozostałe karty. W systemie zaimplementowano również uproszczony panel użytkownika typu *pracownik*, dostępny pod szablonem *home_pracownik.HTML*. W tym widoku dostęp do części funkcjonalności systemu został celowo ograniczony.

Zgodnie z założeniami aplikacji, pracownik nie posiada dostępu do:

- modułu trenowania modelu,
- pełnej konfiguracji diagnostyki,
- panelu administracyjnego Django.

Dzięki temu pracownicy mogą dostosowywać i uruchamiać monitoring, ale nie mają możliwości modyfikacji krytycznych ustawień systemu ani prowadzenia treningu nowych modeli, co zabezpiecza system przed nieautoryzowanymi zmianami i błędami użytkownika. Uprawnienia wynikają z przypisania użytkownika do odpowiedniej grupy lub typu konta, a wybór widoku głównego (*home_admin.HTML* vs *home_pracownik.HTML*) dokonywany jest dynamicznie na podstawie informacji o użytkowniku zalogowanym do sesji.

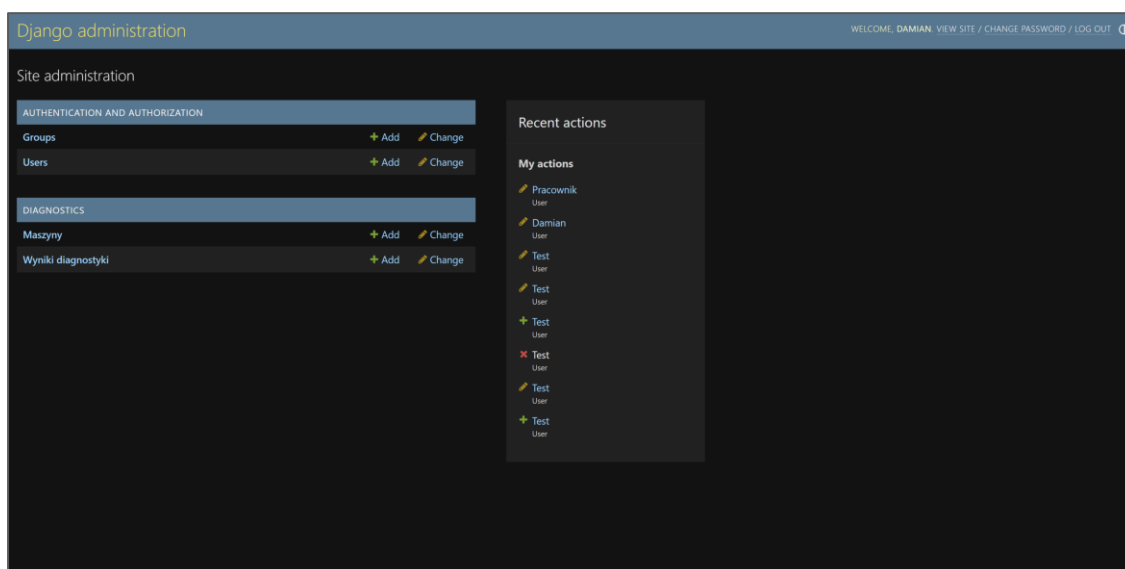


Rys. 5. Główny panel pracownika systemu diagnostycznego

System diagnostyczny korzysta również z wbudowanego panelu administracyjnego Django. Panel ten umożliwia uprawnionym użytkownikom pełne zarządzanie obiektami bazy danych w sposób graficzny i bezpośredni. W panelu tym dostępne są zakładki:

- **Users i Groups** – umożliwiające zarządzanie kontami użytkowników i ich przynależnością do grup (np. pracownicy, administratorzy),
- **Maszyny** – reprezentujące foldery danych używane do treningu i monitoringu,
- **Wyniki diagnostyki** – zawierające zarejestrowane pomiary lub alerty wygenerowane przez system.

Przykładowy widok panelu administracyjnego Django przedstawia dostęp do zakładek oraz historię ostatnich akcji (np. dodanie nowego użytkownika lub modyfikacja konfiguracji). Panel ten jest dostępny tylko dla użytkowników posiadających odpowiednie uprawnienia i nie jest widoczny dla kont pracowniczych.



Rys. 6. Widok panelu administracyjnego Django używanego w systemie diagnostycznym

Zakładka *Diagnostyka* w czasie rzeczywistym udostępnia użytkownikowi opcje uruchamiania i zatrzymywania monitoringu akustycznego oraz podgląd jego wyników.

Elementy interfejsu są opisane w szablonie monitoring.html i obejmują m.in.:

- **Wybór maszyny i pliku audio** – umożliwia wskazanie maszyny (ścieżki folderu z danymi) oraz ewentualnego pliku audio. Te pola są wypełniane dynamicznie przez skrypt: funkcja `loadMachines()` pobiera listę maszyn z backendu i generuje opcje w `<select id="machineSelect">`, a `loadAudioFiles()` analogicznie wypełnia listę dostępnych plików audio.

- **Sterowanie monitoringiem** – przyciski *Start monitoring* i *Stop monitoring* (schowane początkowo) pozwalają włączyć lub wyłączyć zbieranie danych. Ich widoczność i funkcjonalność jest sterowana skryptem *monitoring.js*.
- **Status pracy maszyny** – pole `div id="status"` z klasą `status-badge` wyświetla bieżący stan (np. *Praca normalna*, *Usterka* itp.). Skrypt JavaScript dynamicznie zmienia kolor i tekst tego pola: funkcja `updateStatus()` przypisuje odpowiednią klasę (`status-normal`, `status-warning`, `status-danger`) oraz polski opis stanu.
- **Prezentacja wyników** – Wyniki kolejnych pomiarów są rejestrowane w tabeli (`<table id="resultTable">`) w stopce panelu. W każdej iteracji monitoringu nowy wiersz tabeli jest dodawany za pomocą JavaScript. Przykładowo, fragment z `updateUI()` tworzy wiersz z czasem, statusem i wartościami poziomów dźwięku:

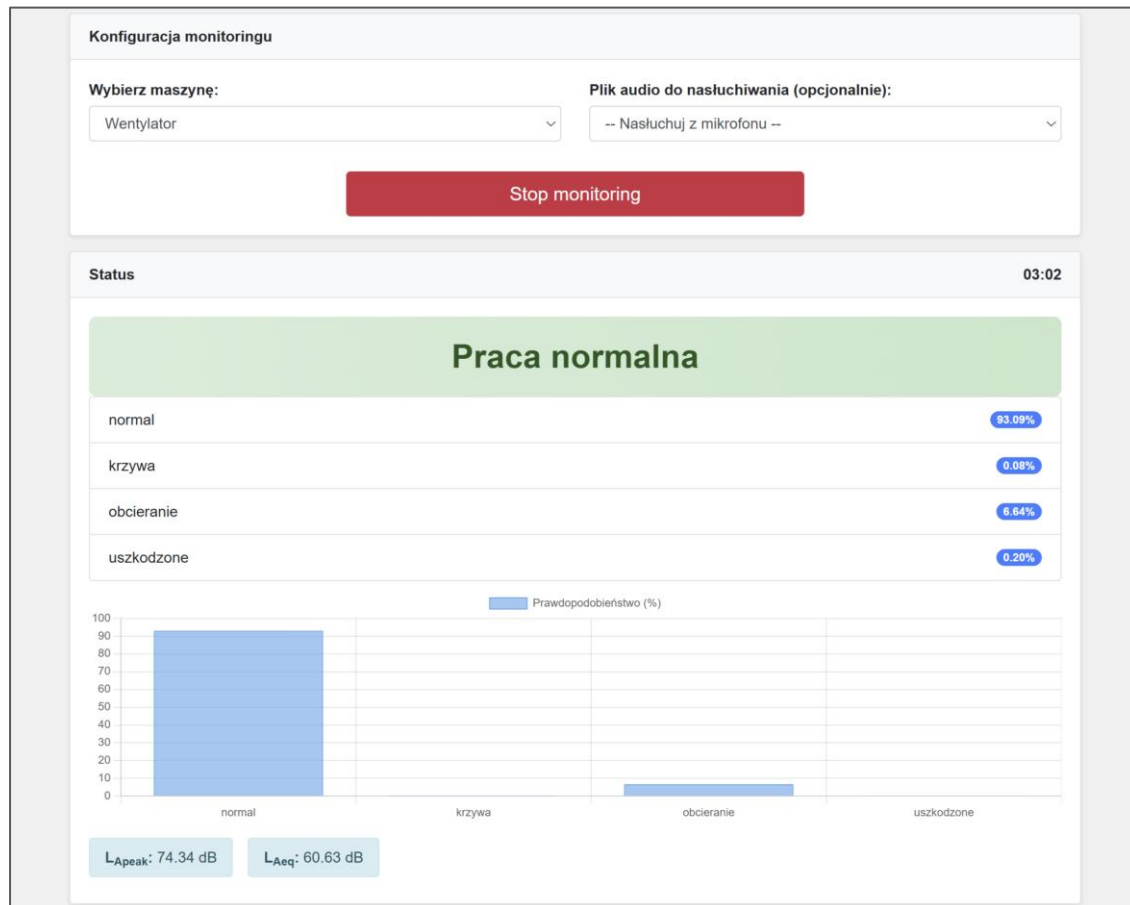
```
const newRow = elements.resultTable.insertRow(0);
newRow.innerHTML = `
  <td>${new Date().toLocaleTimeString()}</td>
  <td>${displayStatus}</td>
  <td>${probPercent}%</td>
  <td>${typeof data.lapeak === 'number' ? data.lapeak.toFixed(2) :
  '-'} dB</td>
  <td>${typeof data.leq === 'number' ? data.leq.toFixed(2) : '-'}
  dB</td>
`;
```

Dzięki temu użytkownik widzi historyczne logi pomiarów w czasie rzeczywistym.

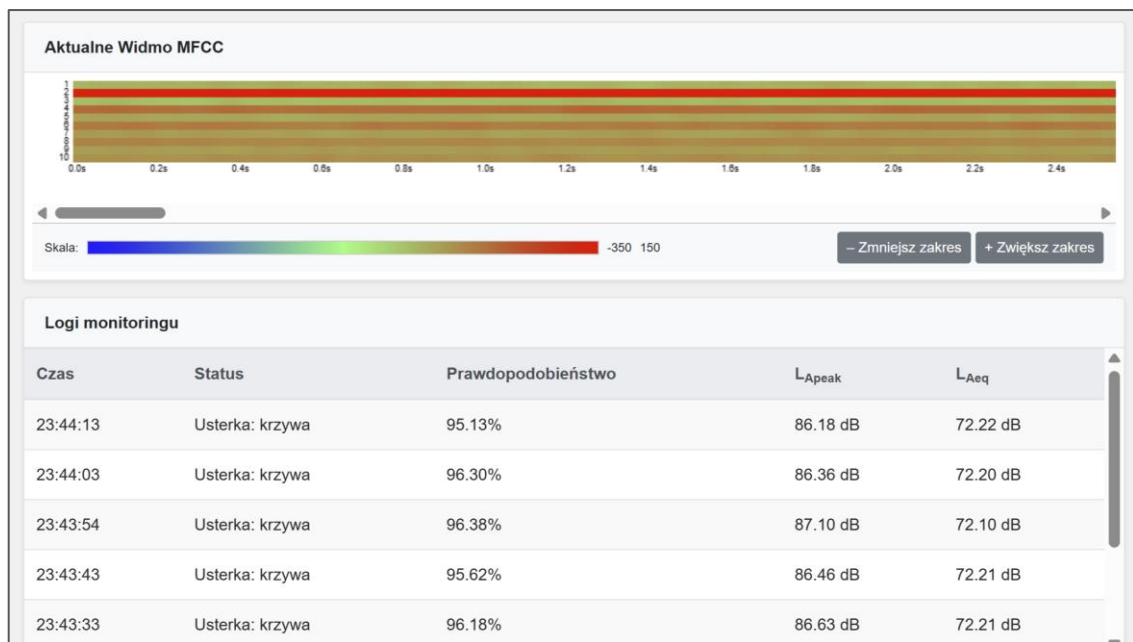
- **Wykres MFCC** – Grafika widma MFCC jest wyświetlana na elemencie `<canvas id="mfccCanvas">` z interaktywną skalą. Po każdej próbie akustycznej skrypt `drawMFCC(data.mfcc_array)` rysuje nowe widmo na kanwie, co pozwala obserwować aktualne cechy dźwięku na żywo. Kontrolki do zmiany skali (przyciski *Zmniejsz zakres* / *Zwiększ zakres*) również sterowane są w JavaScript (`adjustScale()`).

Po naciśnięciu *Start monitoring* JavaScript wysyła za pomocą AJAX dane formatu (wybrana maszyna, plik, typ klasyfikacji) do `/diagnostics/start_monitoring/`.

Backend zwraca konfigurację, którą skrypt wykorzystuje do okresowego pobierania danych z `/diagnostics/get_realtime_data/` i aktualizuje UI. Widać to w metodzie `startMonitoring()`, gdzie po otrzymaniu odpowiedzi `config = await response.json()` pobierany jest czas interwału, a następnie uruchamiana jest pętla `setInterval` wywołująca `updateUI(data)`. Gdy użytkownik kliknie *Stop monitoring*, wysyłane jest żądanie POST do `/diagnostics/stop_monitoring/`, które kończy sesję monitoringu i przywraca interfejs do stanu wyjściowego.



Rys. 7. Interfejs monitoringu akustycznego w czasie rzeczywistym - sekcja 1 z 2



Rys. 8. Interfejs monitoringu akustycznego w czasie rzeczywistym - sekcja 2 z 2

Panel *Trening Modelu* umożliwia wybór danych do treningu i uruchomienie procesu uczenia. Po wybraniu folderów z danymi oraz nazwy modelu, rozpoczęcie treningu inicjuje wywołanie `/diagnostics/start_training/`. Podczas treningu interfejs wyświetla dynamicznie wykresy metryk dla zbioru treningowego i walidacyjnego. Skrypty w *training.js* inicjalizują wykresy liniowe i cyklicznie pobierają dane z `/diagnostics/get_training_progress/`. Otrzymane wartości aktualizują szereg punktów na wykresach oraz pasek postępu. Funkcja `updateCharts(data)` dla każdej metryki wstawia nowy punkt i wywołuje `chart.update()`. Przycisk *Przerwij trening* wysyła POST na `/diagnostics/stop_training/` i umożliwia zatrzymanie procesu przed zakończeniem epoki. Dzięki temu panelowi użytkownik na bieżąco obserwuje proces uczenia – podobnie jak w monitoringu, dane są prezentowane dynamicznie i aktualizowane w czasie rzeczywistym za pomocą mechanizmów AJAX oraz grafiki.

Trening Modelu

Nazwa maszyny:

Wentylator

Dane treningowe:

INNE
wentylator_10m_10s
wentylator_10m_30s
wentylator_10m_3s
wentylator_30m_10s
wentylator_30m_30s
wentylator_30m_3s

Przytrzymaj Ctrl/Cmd do wyboru wielu folderów

Rozpocznij trening

Rys. 9. Formularz uruchamiania procesu treningu modelu

Trening Modelu

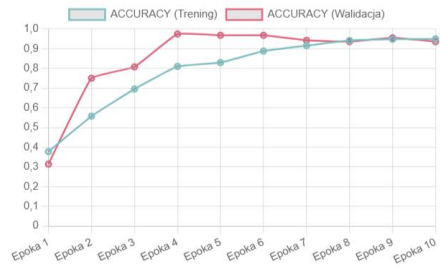
Postęp treningu

33.33%

Epoka 10/30

Przerwij trening

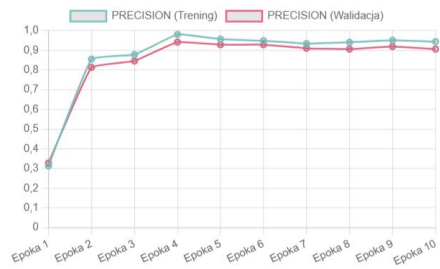
Accuracy



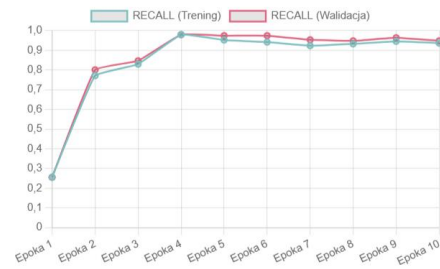
Loss



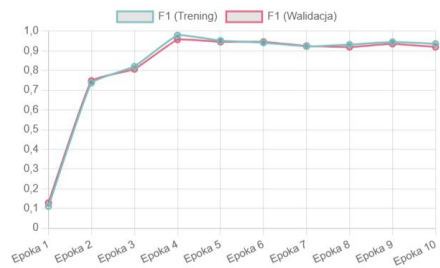
Precision



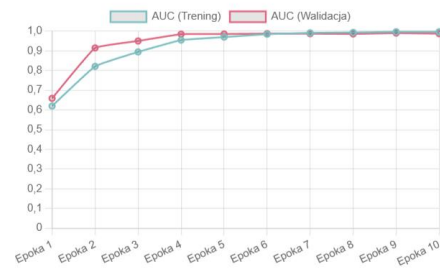
Recall



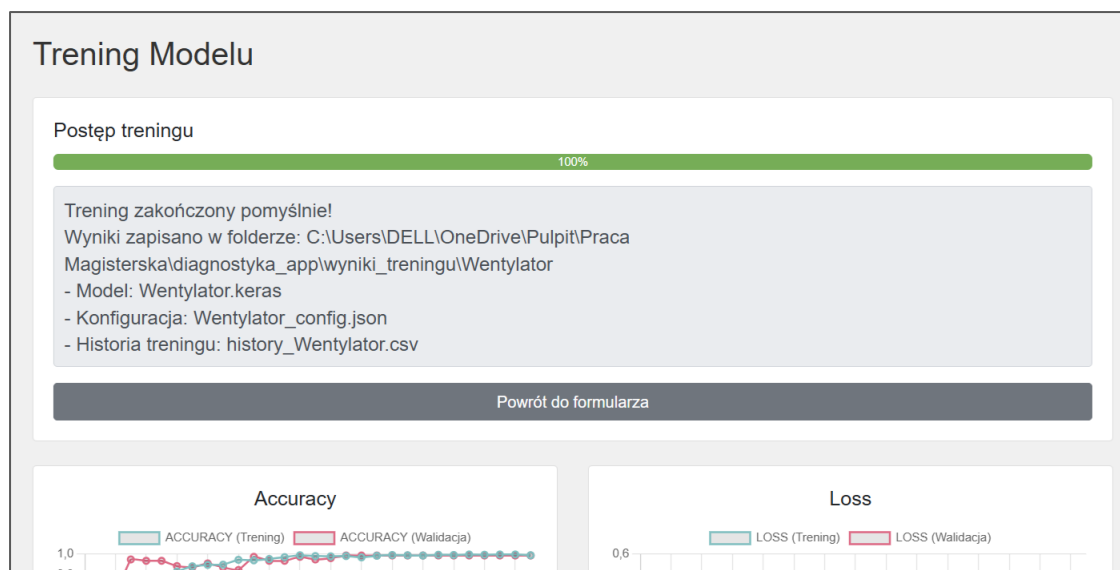
F1



Auc



Rys. 10. Widok postępu treningu z wykresami metryk



Rys. 11. Zakończenie procesu treningu i zapis wyników

5.6.2 Formularz konfiguracji diagnostyki

Formularz konfiguracji diagnostyki stanowi centralny element interfejsu użytkownika w zakresie parametryzacji systemu do treningu czy monitoringu. Został zaprojektowany w sposób umożliwiający elastyczne i szczegółowe dostosowanie zachowania systemu, zarówno pod względem warunków pomiarowych, jak i progów klasyfikacyjnych czy sposobu interpretacji wyników. Za jego wyświetlanie, walidację oraz przetwarzanie odpowiedzialne są klasy formularzy zdefiniowane w pliku *forms.py*, powiązane bezpośrednio z modelem *DiagnosticConfig*.

5.6.2.1 Działanie i zapis konfiguracji

Formularz składa się z wielu sekcji podzielonych na zakładki, które użytkownik wybiera za pomocą poziomej listy nav-tabs.

Przykładowy fragment HTML definiujący je to:

```
<ul class="nav nav-tabs" id="configTabs" role="tablist">
<li class="nav-item">
<a class="nav-link active" id="monitoring-tab" data-toggle="tab"
href="#monitoring" role="tab">Monitoring</a>
</li>
<!-- kolejne zakładki: Trening, Model, Augmentacja, Podział danych -->
</ul>
```

W każdej zakładce znajdują się pola formularza (`{{config_form.t_low}}`),

{{config_form.sample_rate}} itd.). Dla przykładu, sekcja monitoringu zawiera pola progów diagnostycznych oraz parametrów pomiaru, takie jak dolny i górny próg klasyfikacji czy czas trwania pomiaru i częstotliwość próbkowania itd.

Pola te są renderowane w HTML tak jak poniżej (fragment ze szablonu):

```
<div class="form-group">
<label for="id_t_low">Dolny próg klasyfikacji [%]:</label>
  {{ config_form.t_low }}
</div>
<div class="form-group">
<label for="id_t_high">Górny próg klasyfikacji [%]:</label>
  {{ config_form.t_high }}
</div>
...
<div class="form-group">
  {{ config_form.duration.label_tag }}
  {{ config_form.duration }}
</div>
<div class="form-group">
  {{ config_form.sample_rate.label_tag }}
  {{ config_form.sample_rate }}
</div>
```

Analogicznie w innych zakładkach formularza użytkownik może ustawiać parametry sieci (np. liczba warstw konwolucyjnych, funkcje aktywacji), parametry MFCC (`n_fft`, `hop_length`, `n_mfcc`), a także opcje augmentacji czy podziału danych na zbiory treningowe/kwalifikacyjne/testowe. Możliwość konfigurowania tych wszystkich ustawień została zapewniona przez definicję formularza `DiagnosticConfigForm` (oraz w przypadku konta dla pracownika `WorkerDiagnosticForm`) w *forms.py* oraz powiązanie z modelem `DiagnosticConfig`.

Po wysłaniu formularza metodą POST, dane są weryfikowane i zapisywane. W przypadku błędów walidacyjnych, użytkownik otrzymuje zwrot informacji z czytelnym komunikatem przypisanym do konkretnego pola. Po stronie serwera formularz sprawdzany jest pod kątem poprawności. Jeśli walidacja powiedzie się, nowe wartości są zapisywane do bazy (i opcjonalnie do obiektu sesji), a użytkownik informowany jest o pomyślnym zapisie. W wielu miejscach aplikacji (szczególnie w trakcie monitoringu i treningu) aktualna konfiguracja jest odczytywana z bazy danych w czasie rzeczywistym, co zapewnia spójność parametrów systemu bez konieczności restartu serwera.

5.6.2.2 Struktura formularza i przypisane klasy Django

Tak jak wspomniano w powyższej sekcji, w aplikacji występują dwie klasy formularzy:

- **DiagnosticConfigForm** – pełna wersja formularza zawierająca wszystkie sekcje konfiguracyjne, przeznaczona dla administratorów systemu.
- **WorkerDiagnosticForm** – uproszczona wersja ograniczona do parametrów monitoringu, przeznaczona dla pracowników technicznych.

Formularze są instancjonowane dynamicznie w widoku *views_diagnostics.py*, na podstawie typu użytkownika, co zapewnia, że każdy użytkownik widzi tylko te pola, które są dla niego przeznaczone.

Formularz **DiagnosticConfigForm** podzielony jest na kilka głównych sekcji logicznych, które odpowiadają poszczególnym komponentom systemu:

1) Sekcja: Monitoring

- `t_low`, `t_high` – dolny i górny próg klasyfikacji [%],
- `laeq_low`, `laeq_high` – progi dla poziomu L_{Aeq} [dB],
- `duration` – długość próbki [s],
- `sample_rate` – częstotliwość próbkowania [Hz],
- `interval` – interwał monitoringu [s],
- `send_notification` – wysyłanie powiadomień o usterkach (bool),
- `classification_type` – typ klasyfikacji (binary / multiclass).

2) Sekcja: Trening

- `learning_rate` – współczynnik uczenia,
- `batch_size` – rozmiar batcha,
- `epochs` – liczba epok,

- `conv_dropout_rate`, `dense_dropout_rate` – dropout dla warstw konwolucyjnych i gęstych,
- `optimizer` – algorytm optymalizacji,
- `early_stopping`, `patience`, `min_delta` – mechanizm wczesnego zatrzymania,
- `focal_alpha`, `focal_gamma` – parametry funkcji Focal Loss.

3) Sekcja: Model

- `num_conv_layers`, `num_dense_layers` – liczba warstw konwolucyjnych i gęstych,
- `kernel_size` – rozmiar jądra konwolucji,
- `num_pooling_layers`, `pooling_sizes` – liczba i rozmiary warstw poolingowych,
- `activation_function`, `pooling_type`, `output_activation` – funkcje aktywacji, typ pooling, typ aktywacji wyjściowej (np. softmax),
- `kernel_regularizer` – regularyzacja jądra (L2 kernel regularizer).

4) Sekcja: Parametry MFCC

- `n_fft` – rozmiar okna FFT,
- `hop_length` – długość skoku (hop),
- `n_mfcc` – liczba współczynników MFCC,
- `mfcc_length` – długość wektora MFCC (czyli liczba ramek czasowych).

5) Sekcja: Augmentacja

- `augmentation_multiplier` – liczba augmentacji na próbkę klasy *abnormal*,
- `noise_addition_level` – poziom dodawanego szumu,

- `time_stretch_probability` – prawdopodobieństwo rozciągania w czasie,
- `pitch_shift_range` – zakres przesunięcia wysokości dźwięku,
- `volume_gain_range` – zakres wzmocnienia głośności [dB].

6) Sekcja: Podział danych

- `training_split`, `validation_split`, `test_split` – procentowy podział danych na zbiory.

Zaprojektowany formularz konfiguracji stanowi elastyczne i rozbudowane narzędzie pozwalające użytkownikowi na pełną kontrolę nad zachowaniem systemu diagnostycznego. Dzięki ścisłemu powiązaniu z modelem bazy danych, czytelnej strukturze sekcji oraz automatycznej walidacji i obsłudze w czasie rzeczywistym, konfiguracja może być łatwo dostosowywana do potrzeb zarówno analizy eksperymentalnej, jak i rzeczywistego monitoringu maszyn. Całość została zrealizowana zgodnie z zasadami dobrej praktyki inżynierskiej – z naciskiem na ergonomię interfejsu, przejrzystość kodu oraz możliwość przyszłego rozszerzania systemu.

6. Eksperymenty i wyniki

6.1 Metodyka eksperymentów i konfiguracja bazowa

6.1.1 Cel i zakres eksperymentów

Celem przeprowadzonych eksperymentów była analiza wpływu wybranych parametrów konfiguracyjnych systemu diagnostycznego na jego skuteczność, stabilność działania czy podatność na zjawisko nadmiernego dopasowania. Eksperymenty miały charakter parametryczny i obejmowały zarówno komponenty treningowe modelu, jak i aspekty związane z przetwarzaniem cech oraz augmentacją danych wejściowych. Dzięki temu możliwe było zidentyfikowanie, które elementy konfiguracji mają największy wpływ na końcowy wynik klasyfikacji oraz w jakim stopniu zmiany tych parametrów wpływają na ustanowione w aplikacji metryki diagnostyczne.

Zakres badań obejmował wyłącznie te parametry, które użytkownik może bezpośrednio modyfikować za pomocą interfejsu programu. Wykluczono zatem zmienne zależne od warunków zewnętrznych (takie jak np. zmienna akustyczna otoczenia), skupiając się na aspektach technicznych możliwych do powielenia i kontrolowania. Każdy eksperyment polegał na modyfikacji jednego wybranego parametru względem ustalonej konfiguracji bazowej, przy jednoczesnym zachowaniu stałości pozostałych elementów. Tak metodologia umożliwiła precyzyjne wyodrębnienie wpływu pojedynczych czynników na efektywność działania modelu diagnostycznego.

W analizie uwzględniono zarówno klasyczne metryki ewaluacyjne obliczane w czasie treningu (*loss*, *accuracy*, *precision*, *recall*, *AUC*, *F1-score*), jak i dodatkowe wskaźniki opracowane w ramach niniejszej pracy, w tym: wskaźnik nadmiernego dopasowania (*overfitting gap*), rozbieżność precyzji (*pr_diff*), różnicę stabilności predykcji (*stability gap*), lukę AUC (*AUC_gap*), a także zagregowaną ocenę jakości w postaci metryki *composite_score*. Pozwoliło to nie tylko na ocenę skuteczności modelu w klasycznym sensie, ale również na dokładniejszą diagnozę jego zachowania w różnych aspektach jakościowych.

Ekspertyzacje przeprowadzono z użyciem rzeczywistych danych dźwiękowych urządzenia testowego, w tym zarówno sygnałów przedstawiających jego prawidłowe działanie, jak i różne typy usterek. Dzięki temu możliwe było przetestowanie systemu w warunkach zbliżonych do rzeczywistej eksploatacji, a także weryfikacja jego odporności na zjawiska losowe, takie jak zmienność warunków akustycznych czy rozrzut w danych treningowych. W dalszych podrozdziałach zaprezentowano szczegółowe wyniki oraz ich interpretację.

6.1.2 Układ pomiarowy i środowisko testowe

Z uwagi na prototypowy charakter opracowanego systemu diagnostycznego oraz badawczo-eksperymentalny cel niniejszej pracy, w realizacji pomiarów nie zastosowano profesjonalnego toru pomiarowego klasy laboratoryjnej. Z uwagi na ograniczenia budżetowe wykorzystano sprzęt powszechnie dostępny, który pomimo braku certyfikacji metrologicznej oferuje wystarczające parametry techniczne do celów testowych i treningowych systemów opartych na dźwięku. Decyzja ta była uzasadniona również brakiem jednoznacznych, ustandaryzowanych wytycznych dotyczących wymagań pomiarowych przy tworzeniu zbiorów danych dźwiękowych dla algorytmów sztucznej

inteligencji. W efekcie przyjęto podejście zorientowane na praktyczność i odwzorowanie realistycznych warunków pracy potencjalnego systemu wdrożeniowego.



Rys. 12. Komputer HP T630 oraz mikrofon Røde NT-USB Mini tworzące kompletny system

Układ pomiarowy składał się z poniższych komponentów:

- **Mikrofon Rode NT-USB Mini** – podłączony przez interfejs USB,
- **Komputer HP T630** – pełniący rolę urządzenia do akwizycji danych,
- **Monitorowane urządzenie** – wentylator podłogowy 45 cm o mocy 110 W, generujący poziom dźwięku $L_A = 69 \text{ dB}$ deklarowany przez producenta.



Rys. 13. Schemat ideowy układu pomiarowego

Mikrofon Røde NT-USB Mini to urządzenie o charakterystyce kierunkowej typu kardoidalnego, co oznacza, że jest ono najbardziej czułe na dźwięki docierające z przodu kapsuły, a mniej na te z boków i tyłu. Jest to szczególnie korzystne w kontekście przewidywanego wykorzystania systemu diagnostycznego w środowiskach przemysłowych, gdzie obecne są istotne zakłócenia tła i hałas środowiskowy — kierunkowość pozwala na lepszą separację sygnału właściwego od niepożądanych źródeł dźwięku. Mimo, że mikrofon ten nie należy do klasy referencyjnej, jego specyfikacja (mikrofon pojemnościowy, o płaskiej charakterystyce z zakresu $20\text{ Hz} - 20\text{ kHz}$) jest w pełni dostateczna do roli czujnika akustycznego w systemie.

Mikrofon został zamontowany na statywie w sposób stabilny i zgodnie z ogólnymi wytycznymi zawartymi w normach ISO, które choć odnoszą się do różnych form monitorowania stanu technicznego, zawierają wspólne zasady dotyczące sposobu prowadzenia pomiarów: m.in. konieczność zachowania stałej odległości, unikania kontaktu z elementami generującymi drgania mechaniczne oraz eliminacji przypadkowych źródeł zakłóceń [9]. W szczególności, mikrofon ustawiono w osi pracy urządzenia, w odległości 1 m co pozwoliło na rejestrację dominujących komponentów akustycznych generowanych przez nie, przy jednoczesnym ograniczeniu wpływu odbić i zakłóceń tła.



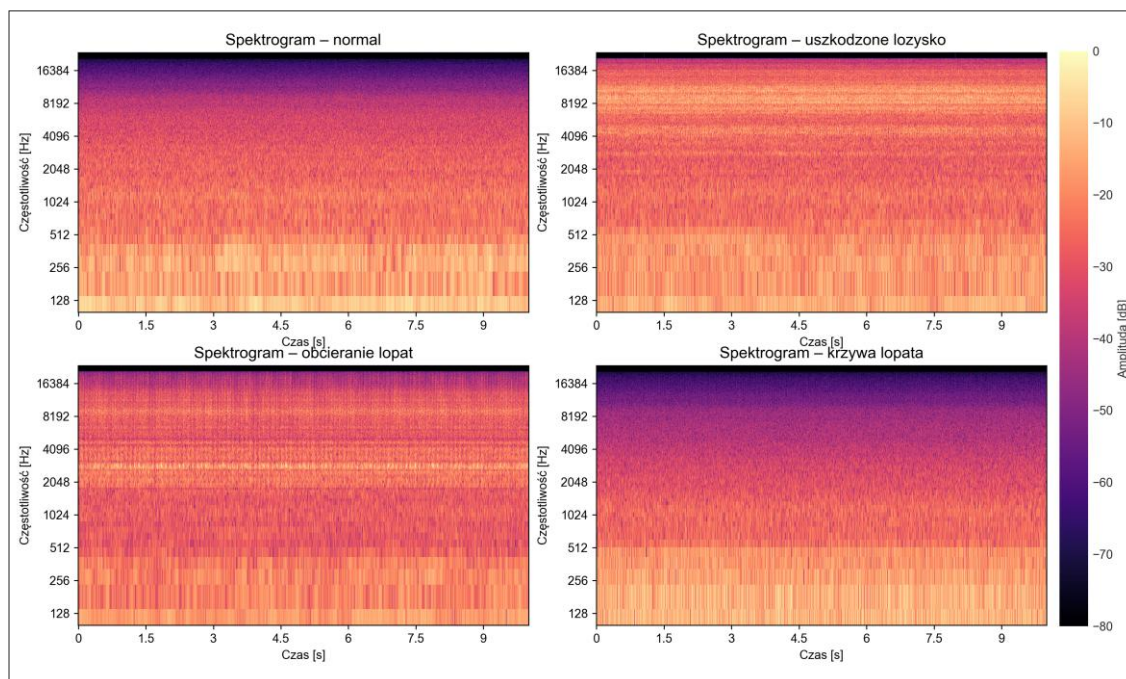
Rys. 14. Układ pomiarowy wykorzystany do stworzenia bazy nagrań oraz testów

Rejestracja sygnału odbywała się w typowym pomieszczeniu mieszkalnym, którego parametry akustyczne nie były specjalnie adaptowane. Czas pogłosu w pomieszczeniu wynosił około 0,4 s co odpowiada typowym warunkom dla średnio umeblowanego wnętrza o powierzchni kilkunastu metrów kwadratowych. Pomimo braku adaptacji akustycznej, poziom tła był wystarczająco niski, by nie wpływać znacząco na jakość zbieranych danych. Średni poziom ciśnienia akustycznego tła wynosił $L_A = 26 \text{ dB}$, co czyniło go znacznie niższym niż poziom dźwięku generowanego przez monitorowane urządzenie. Taka różnica poziomów sygnałów sprawiły, że zarówno szum otoczenia, jak i szum własny mikrofonu nie stanowiły istotnego źródła zakłóceń dla procesu rejestracji.

W ramach akwizycji danych zarejestrowano cztery główne nagrania, każde o długości 1 h:

- **Praca normalna (*normal*)** – bez żadnych defektów,
- **Praca z obcieraniem łopat wentylatora (*obcieranie_lopat*)** – symulacja uszkodzenia mechanicznego,
- **Praca z uszkodzonym łożyskiem (*uszkodzone_lozysko*)** – imitacja zużycia elementu tocznego,
- **Praca z wygiętym śmigłem wentylatora (*krzywa_lopata*)** – symulacja asymetrii masy i zmiany aerodynamiki.

Nagrania te zostały następnie podzielone na odpowiednio krótsze fragmenty i wykorzystane jako zbiór treningowy, walidacyjny i testowy dla modelu.



Rys. 15. Przykładowe spektrogramy sygnałów akustycznych poszczególnych klas

Całość procesu rejestracji została zrealizowana przy użyciu dedykowanego skryptu w języku Python, wykorzystującego dokładnie te same mechanizmy przetwarzania i akwizycji sygnału, które zostały zaimplementowane w interfejsie aplikacji diagnostycznej. Zapewniło to pełną spójność między fazą akwizycji danych a fazą działania systemu. Dzięki temu ograniczono do minimum ryzyko błędów wynikających z różnic w środowisku pomiarowym i zagwarantowano pełną kompatybilność struktury danych wejściowych.

6.1.3 Konfiguracja bazowa i punkt odniesienia

Dane wykorzystywane w treningu i testach systemu diagnostycznego zostały zapisane w formacie WAV PCM *16-bit*, z częstotliwością próbkowania *48 kHz*. Jest to standardowy, bezstratny format kodowania dźwięku, powszechnie stosowany w profesjonalnych zastosowaniach audio i zapewniający wysoką jakość akwizycji. Zastosowanie 16-bitowej głębi umożliwia zapis sygnałów z szerokim zakresem dynamicznym, natomiast wysoka częstotliwość próbkowania gwarantuje precyzyjne odwzorowanie sygnałów w pełnym paśmie dźwięku. Ma duże znaczenie w kontekście

detekcji subtelnych różnic akustycznych charakterystycznych dla stanu technicznego maszyny.

Wszystkie eksperymenty zostały przeprowadzone względem jednej, spójnej konfiguracji bazowej, która stanowiła wspólny punkt odniesienia dla analizy wpływu poszczególnych parametrów.

Wybór tej konfiguracji miał charakter kompromisowy – parametry zostały dobrane tak, aby zapewnić stabilność treningu oraz wystarczającą jakość klasyfikacji, ale jednocześnie pozostawić przestrzeń do dalszej optymalizacji w toku eksperymentów:

1) Parametry treningu

- **Współczynnik uczenia** (`learning_rate`): 0,0003
- **Liczba epok** (`epochs`): 30
- **Rozmiar batcha** (`batch_size`): 64

2) Architektura sieci CNN:

- **Liczba warstw konwolucyjnych i poolingowych** (`num_conv_layers`, `num_pooling_layers`): 1
- **Liczba neuronów w warstwie konwolucyjnej** (`num_conv_filters`): 64
- **Liczba neuronów w warstwie gęstej** (`dense_units`): 64
- **Dropout:**
 - **Warstwy gęste** (`dense_dropout_rate`): 0,5
 - **Warstwy konwolucyjne** (`conv_dropout_rate`): 0,3
- **Regularyzacja jądra L2** (`kernel_regularizer`): 0,0005
- **Rozmiar jądra** (`kernel_size`): 3×3
- **Typ warstwy agregującej** (`pooling_type`): GlobalAveragePooling2D (GAP)
- **Funkcja aktywacji** (`activation_function`): ReLU

3) Ekstrakcja cech audio (MFCC)

- **Liczba współczynników MFCC** (`n_mfcc`): 25
- **Długość MFCC** (`mfcc_length`): 1000
- **Rozmiar FFT** (`n_fft`): 2048
- **Długość skoku** (`hop_length`): 256

4) Augmentacja danych (dla klasy *abnormal*)

- **Poziom dodawanego szumu** (`noise_addition_level`): 0,02
- **Przesunięcie wysokości dźwięku** (`pitch_shift_range`): 2,5
- **Rozciąganie czasowe dźwięku** (`time_stretch_probability`): 0,4
- **Wzmocnienie głośności** (`volume_gain_range`): 3
- **Liczba augmentacji na próbkę** (`augmentations_multiplier`): 3

5) Dane wejściowe i podział zbioru

- **Długość próbki**: 10
- **Podział zbioru**:
 - **Zbiór treningowy**: 0,6
 - **Zbiór walidacyjny**: 0,25
 - **Zbiór testowy**: 0,15

Dzięki tak dobranym wstępnym parametrom możliwa była dokładna analiza wpływu każdego z nich na jakość działania systemu diagnostycznego, co było zasadniczym celem tej części pracy.

6.2 Analiza wpływu parametrów systemu diagnostycznego

W niniejszym rozdziale przedstawiono wyniki serii eksperymentów, których celem było systematyczne badanie wpływu wybranych parametrów konfiguracji systemu diagnostycznego na jakość klasyfikacji i stabilność predykcji. Analizy te przeprowadzono w sposób kontrolowany, z zastosowaniem podejścia eksperymentalnego – każda zmiana dotyczyła wyłącznie jednego parametru, przy zachowaniu pozostałych ustawień na poziomie bazowym.

Aby ułatwić interpretację wyników i lepiej uwidocznić interakcje między komponentami systemu, eksperymenty pogrupowano według kategorii parametrów: trening, architektura modelu, augmentacja danych oraz monitoring w czasie rzeczywistym. Dla każdego parametru testowanego w danym eksperymencie wytrenowano odrębny model przy użyciu tych samych danych wejściowych, tej samej metody ekstrakcji cech oraz identycznych ustawień optymalizatora.

Dla każdej konfiguracji treningowej model był trenowany od podstaw, a jego skuteczność oceniana przy pomocy standardowych metryk walidacyjnych:

- **Accuracy**
- **Loss**
- **Precision**
- **Recall**
- **F1-score**
- **AUC**

Aby umożliwić pełniejszą ocenę działania modelu, uwzględniającą nie tylko końcową skuteczność klasyfikacji, ale także stabilność, równowagę predykcji oraz zdolność generalizacji. Opracowano dodatkowe wskaźniki pomocnicze, liczone po zakończeniu treningu. Każdy z nich został opisany poniżej.

6.2.1.1 Wskaźniki pomocnicze i ich definicje

1) Composite Score

Wskaźnik syntetyczny, uwzględniający ogólną jakość klasyfikatora, który łączy w sobie trzy najważniejsze metryki walidacyjne, nadając im różną wagę:

$$\text{composite_score} = 0.5 \cdot \text{val_f1} + 0.3 \cdot \text{val_AUC} + 0.2 \cdot \text{val_accuracy}$$

Współczynniki te zostały dobrane arbitralnie, przy założeniu, że *F1-score* (równowaga między *precision* i *recall*) jest najważniejszą cechą skutecznego klasyfikatora.

2) Overfitting Gap

Miara przeuczenia modelu. Im większa różnica między skutecznością na danych treningowych i walidacyjnych, tym bardziej model jest dopasowany do danych uczących kosztem generalizacji:

$$\text{overfitting_gap} = \text{train_f1} - \text{val_f1}$$

3) Stability Gap

Miara stabilności predykcji na zbiorze walidacyjnym, obliczana jako wartość bezwzględna różnicy pomiędzy *F1-score* i *accuracy*:

$$\text{stability_gap} = |\text{val_f1} - \text{val_accuracy}|$$

Wskaźnik ten pokazuje, jak bardzo model jest konsekwentny w swoich przewidywaniach. Niska wartość oznacza wysoką spójność.

4) PR Difference

Miara asymetrii klasyfikatora – wykazuje czy model traktuje obie klasy w sposób zrównoważony:

$$\text{pr_diff} = |\text{val_precision} - \text{val_recall}|$$

Zbyt duże rozbieżności świadczą o tym, że model może np. generować zbyt wiele fałszywych alarmów lub pomijać rzeczywiste usterki.

5) AUC Gap

Alternatywna forma pomiaru przeuczenia, szczególnie użyteczna przy niebalansowanych danych.

Obliczana jako różnica między AUC na zbiorze treningowym i walidacyjnym:

$$AUC_gap = |val_recall - val_AUC|$$

6.2.1.2 Etapy analizy i agregacja wyników

Po zakończeniu każdego treningu, z wszystkich epok wybrano tę, która uzyskała najwyższy wynik $composite_score$. Następnie, dla wybranej epoki dodatkowo obliczono przedstawione powyżej wskaźniki pomocnicze.

Z każdego eksperymentu wybrano jedną najlepszą konfigurację (z najlepszą epoką z etapu 1) i obliczono syntetyczny wskaźnik końcowy $final_score$, według wzoru:

$$final_score = 0.6 \cdot composite_score - 0.2 \cdot |overfitting_gap| - 0.1 \cdot pr_diff - 0.05 \cdot stability_gap - 0.05 \cdot AUC_gap$$

Współczynniki wagowe w tym wzorze zostały dobrane arbitralnie, z założeniem:

- Przyznania największej wagi skuteczności klasyfikacji $composite_score$,
- Potraktowania pozostałych wskaźników jako kar – obniżających ocenę w przypadku przeuczenia ($overfitting_gap$), niestabilności ($stability_gap$), nierównowagi między precyzją i czułością (pr_diff) oraz rozbieżności między uczeniem a generalizacją (AUC_gap).

Dobór takiej formy oceny końcowej był uzasadniony potrzebą opracowania miary złożonej, ale jednocześnie intuicyjnej i dostosowanej do wymagań praktycznych systemów diagnostycznych. Warto zaznaczyć, że najwyższy możliwy do uzyskania tym wyznacznikiem wynik wynosi 0,6, a nie jak w większości przypadków 1,0.

Stosowane w klasycznej literaturze pojedyncze metryki często nie uwzględniają złożonej dynamiki działania modelu, w tym:

- różnic między skutecznością na zbiorze treningowym i walidacyjnym,
- niesymetrycznej dokładności detekcji różnych klas (np. przewagi precyzji nad czułością),

- czy też wahań w metrykach w ostatnich epokach, które mogą świadczyć o braku stabilności uczenia.
- Z tego względu opracowano podejście bardziej kompleksowe, które:
- faworyzuje modele nie tylko skuteczne, ale też stabilne i zrównoważone,
- umożliwia porównanie eksperymentów różniących się pojedynczymi parametrami,
- oraz umożliwia podjęcie decyzji na podstawie jednej syntetycznej liczby, bez konieczności wieloaspektowej analizy każdej metryki osobno.

Zaletą zastosowanego wskaźnika *final_score* jest także jego interpretowalność, ponieważ dzięki prostemu wzorowi użytkownik może zrozumieć, dlaczego dany model uzyskał niższą ocenę końcową mimo wysokiego *composite_score* (np. z powodu dużej luki *pr_diff* lub *AUC_gap*).

Taka dwuetapowa procedura, czyli najpierw wybór najlepszej epoki, następnie obliczenie *final_score*, pozwalała nie tylko określić najlepsze wartości testowanych parametrów, ale również lepiej zrozumieć mechanizmy ich wpływu na jakość i zachowanie modelu.

6.2.2 Parametry treningu

6.2.2.1 Współczynnik uczenia (*learning_rate*)

Tab. 2. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika uczenia

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,00005	30	0,2038	0,2034	0,3984	0,2051	0,2074	0,1348	0,3711	0,3571	0,6831	0,7452	0,2608	0,1654
0,0001	25	0,1696	0,1877	0,5234	0,7756	0,8312	0,8673	0,7734	0,8178	0,8085	0,9069	0,7724	0,8091
0,0002	19	0,1451	0,1748	0,5716	0,7821	0,8928	0,8889	0,8359	0,823	0,8544	0,8748	0,8379	0,8256
0,0003	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0,0005	30	0,0593	0,0891	0,8099	0,8782	0,8747	0,9076	0,8685	0,9016	0,9658	0,9352	0,8668	0,901
0,001	8	0,1038	0,1705	0,6602	0,859	0,8842	0,8823	0,8424	0,8855	0,9107	0,9295	0,8322	0,8666

Eksperyment wykazał wyraźny wpływ współczynnika uczenia na efektywność treningu modelu. Najniższa testowana wartość 0,00005 skutkowała nieefektywnym uczeniem – model osiągnął bardzo niską trafność (*val_accuracy* = 0,2051) oraz *val_f1* = 0,1654, co wskazuje na brak konwergencji pomimo pełnych 30 epok.

Dla wartości 0,0001 i 0,0002 odnotowano znaczny wzrost jakości predykcji ($val_f1 = 0,8091$ i $0,8256$), przy wysokich wartościach $val_precision$ i val_recall powyżej 0,8. Oba ustawienia pozwoliły na stosunkowo szybkie osiągnięcie maksimum skuteczności w mniej niż 30 epokach.

Współczynniki 0,0005 oraz 0,001 zapewniły najwyższe wartości metryk walidacyjnych – $val_accuracy = 0,8782$ i $0,859$, a val_f1 osiągnęło poziom odpowiednio 0,901 i 0,8666. Warto zwrócić uwagę, że model przy 0,001 osiągnął ten wynik już po 8 epokach, co sugeruje bardzo szybką konwergencję.

Tab. 3. Wskaźniki złożone dla różnych wartości współczynnika uczenia

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,00005	30	0,3473	0,0954	0,2223	0,0397	0,0621
0,0001	25	0,8317	-0,0367	0,0495	0,0335	0,0984
0,0002	19	0,8317	0,0123	0,0659	0,0435	0,0204
0,0003	30	0,7833	0,0308	0,0718	0,0102	0,0258
0,0005	30	0,9067	-0,0342	0,006	0,0228	0,0306
0,001	8	0,884	-0,0344	0,0032	0,0076	0,0188

Wskaźnik złożony (*composite_score*) osiągnął najwyższy poziom dla wartości 0,0005 (0,9067), wskazując na najlepszy kompromis między skutecznością, równowagą predykcji i stabilnością. Również 0,001 dało bardzo dobry wynik (0,884), przy minimalnym $pr_diff = 0,0032$ i $stability_gap = 0,0076$, co wskazuje na niemal idealną równowagę między precyzją a czułością.

W przypadku 0,0002, mimo stosunkowo wysokiego *composite_score* (0,8317), odnotowano zwiększony pr_diff (0,0659) i $stability_gap$ (0,0435), co może sugerować mniejszą równowagę klasyfikacji. Z kolei 0,00005, oprócz niskiego val_f1 , cechowało się również bardzo niekorzystnym $pr_diff = 0,2223$ i najwyższym $overfitting_gap = 0,0954$.

Podsumowanie

- Najlepsze rezultaty osiągnięto przy współczynniku 0,0005, który zapewnił najwyższy *composite_score* oraz najniższe wartości pr_diff i *AUC_gap*, świadczące o zrównoważonej i stabilnej klasyfikacji.
- Wartość 0,001 wykazała dużą skuteczność, ale bardzo szybka konwergencja w zaledwie 8 epokach może prowadzić do przeuczenia przy dłuższym treningu.

- Zbyt niskie wartości, takie jak 0,00005, są nieefektywne i powodują istotne niedouczenie modelu.
- Wartości pośrednie (0,0001, 0,0002, 0,0003) dają akceptowalne wyniki, ale nie dorównują konfiguracjom o większym `learning_rate`.

6.2.2.2 Liczba epok (epochs)

Tab. 4. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby epok

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
10	10	0,1468	0,2073	0,5964	0,3782	0,4686	0,341	0,5482	0,4984	0,8564	0,7316	0,4769	0,3373
15	15	0,13	0,1842	0,6042	0,6923	0,625	0,6263	0,75	0,75	0,8778	0,9085	0,6667	0,6678
20	13	0,1256	0,1794	0,6042	0,7179	0,8602	0,8768	0,7734	0,7579	0,8787	0,8821	0,7411	0,7127
30	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
40	40	0,0698	0,0809	0,7878	0,7692	0,8946	0,8941	0,8177	0,8125	0,9584	0,9523	0,7898	0,7828
45	44	0,0615	0,062	0,8008	0,9103	0,9247	0,9444	0,8958	0,9271	0,9613	0,9795	0,8914	0,9261
50	41	0,0651	0,0719	0,7995	0,8974	0,9305	0,9385	0,9036	0,9167	0,96	0,9936	0,8999	0,9149

Wraz ze wzrostem liczby epok można zaobserwować systematyczne obniżanie wartości `val_loss` (z 0,2073 dla 10 epok do 0,062 dla 45 epok) oraz wzrost `val_accuracy`, `val_precision`, `val_recall` i `val_f1`, co wskazuje na poprawę skuteczności klasyfikatora. Szczególnie istotny wzrost F1-score – z poziomu 0,3373 dla 10 epok do aż 0,9261 dla 45 epok – jednoznacznie potwierdza, że większa liczba epok umożliwiła modelowi pełniejsze dopasowanie do danych. Przy dalszym wzroście (do 50 epok) jakość predykcji pozostała na bardzo wysokim poziomie, ale nie uległa już istotnej poprawie.

Tab. 5. Wskaźniki złożone dla różnych wartości liczby epok

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
10	10	0,4638	0,1396	0,1574	0,0409	0,1248
15	15	0,7449	-0,0011	0,1237	0,0245	0,0307
20	13	0,7646	0,0284	0,1189	0,0052	0,0034
30	30	0,7833	0,0308	0,0718	0,0102	0,0258
40	40	0,8309	0,007	0,0816	0,0136	0,0061
45	44	0,934	-0,0347	0,0173	0,0158	0,0182
50	41	0,935	-0,015	0,0218	0,0175	0,0336

Wskaźnik `composite_score` rośnie wraz z liczbą epok – od 0,4638 dla 10 epok do 0,939 dla 45 epok. Dalszy wzrost do 50 epok skutkuje jedynie kosmetycznym spadkiem (0,935), co sugeruje, że optymalna liczba epok znajduje się w tym przedziale. Najniższe wartości wskaźnika `overfitting_gap` zaobserwowano również dla modeli trenowanych najdłużej, co potwierdza dobrą generalizację. Wartości `pr_diff`,

stability_gap i *AUC_gap* w najlepszych modelach (45 i 50 epok) były bardzo niskie, co dodatkowo potwierdza ich zrównoważoną i stabilną charakterystykę.

Podsumowanie

- Zbyt krótki trening (10–20 epok) prowadził do niedouczenia modelu i niskiej jakości predykcji.
- Zakres 30–40 epok był bardziej akceptowalny, ale nadal mniej skuteczny od dłuższych treningów.
- Najlepsze wyniki osiągnięto przy 45 epokach – model był zarówno skuteczny, jak i stabilny.
- 50 epok nie przyniosło już istotnych korzyści, a *composite_score* uległ minimalnemu pogorszeniu.

6.2.2.3 Rozmiar batcha (batch_size)

Tab. 6. Zestawienie metryk klasyfikacyjnych dla różnych wartości rozmiaru batcha

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
16	25	0,0606	0,0502	0,7773	0,859	0,9214	0,9225	0,8854	0,8854	0,9592	0,9766	0,8791	0,8798
32	27	0,064	0,062	0,7708	0,8397	0,9184	0,9155	0,8789	0,8698	0,9555	0,9667	0,8714	0,8612
48	30	0,0665	0,0788	0,7695	0,8077	0,9089	0,9051	0,8568	0,8438	0,9568	0,9527	0,844	0,8278
64	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
96	30	0,0877	0,1465	0,7031	0,5705	0,5993	0,5548	0,6862	0,651	0,9282	0,8504	0,6125	0,5595
128	26	0,1143	0,1649	0,6693	0,7244	0,8808	0,8819	0,7878	0,7806	0,9031	0,8857	0,7458	0,7272
256	30	0,1639	0,2197	0,5	0,25	0,3276	0,2711	0,3932	0,394	0,8095	0,6245	0,3088	0,229

Dla małych wartości *batch_size* (16, 32, 48) uzyskano najwyższe wartości metryk walidacyjnych (*val_f1*, *val_AUC*, *val_accuracy*), przy czym najlepszy rezultat osiągnięto przy *batch_size* = 16 – m.in. *val_f1* = 0,8798, *val_AUC* = 0,9766, *val_accuracy* = 0,859. Wraz ze wzrostem batcha do 64 i wyżej następuje stopniowy spadek jakości klasyfikacji widoczny zarówno w dokładności (*val_accuracy* = 0,25 dla *batch_size* = 256), jak i w metrykach *f1*, *AUC* i *precision*.

Zbyt duże wartości (128, 256) prowadziły do znacznego pogorszenia wyników, co może świadczyć o utracie zdolności modelu do efektywnego uogólniania z powodu zmniejszonej liczby kroków aktualizacji wag w jednej epoce.

Tab. 7. Wskaźniki złożone dla różnych wartości rozmiaru batcha

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
16	25	0,9047	-0,0007	0,0371	0,0208	0,0174
32	27	0,8885	0,0102	0,0457	0,0215	0,0112
48	30	0,8613	0,0162	0,0613	0,0201	0,0041
64	30	0,7833	0,0308	0,0718	0,0102	0,0258
96	30	0,649	0,053	0,0962	0,011	0,0778
128	26	0,7742	0,0186	0,1013	0,0028	0,0174
256	30	0,3518	0,0798	0,1229	0,021	0,185

Najlepsze wartości *composite_score* uzyskano dla *batch_size* = 16 (0,9047) i 32 (0,8885). Wskaźnik *overfitting_gap* był bardzo niski (w okolicach zera), co sugeruje równowagę pomiędzy wynikami na zbiorze treningowym i walidacyjnym. Wraz ze wzrostem rozmiaru pogarszały się także pozostałe wskaźniki: rosły wartości *pr_diff* i *AUC_gap*, a *composite_score* dla *batch_size* = 256 wyniósł tylko 0,3518.

Najniższe *AUC_gap* = 0,0041 uzyskano dla *batch_size* = 48, co może wskazywać na dobrą równowagę między stabilnością a skutecznością, jednak nie wystarczyło to do uzyskania najlepszego wyniku końcowego.

Podsumowanie

- Najlepsze rezultaty osiągnięto przy małych wartościach *batch_size* (16-32), co wskazuje na wyższą zdolność modelu do generalizacji przy większej liczbie aktualizacji gradientu.
- *batch_size* = 16 zapewniał najwyższe wyniki klasyfikacyjne i bardzo niskie wskaźniki przeuczenia oraz asymetrii predykcji.
- Zwiększanie *batch_size* powyżej 64 skutkowało szybkim pogorszeniem wyników – zwłaszcza dla wartości 128 i 256.
- Wartość *batch_size* = 256 skutkowałą znacznym spadkiem jakości (*val_f1* = 0,229, *val_accuracy* = 0.25) – prawdopodobnie z powodu niestabilności optymalizacji.

6.2.3 Architektura modelu

6.2.3.1 Liczba warstw konwolucyjnych i poolingowych (num_conv_layers, num_pooling_layers)

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
1	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
2	30	0,0756	0,1141	0,9388	0,7564	0,8652	0,7867	0,8073	0,8021	0,9941	0,9454	0,7641	0,6736
3	30	0,1084	0,1785	0,9362	0,4359	0,508	0,4298	0,5208	0,5417	0,9913	0,8829	0,378	0,3291
4	30	0,1354	0,1552	0,9349	0,8718	0,9024	0,8504	0,8737	0,8958	0,9939	0,9776	0,8704	0,8516
5	30	0,1792	0,1634	0,8711	0,9103	0,9643	0,9444	0,9583	0,9271	0,981	0,997	0,958	0,9261

Zwiększanie liczby warstw konwolucyjnych i poolingowych miało zauważalny wpływ na skuteczność klasyfikacji. Dla jednej warstwy osiągnięto umiarkowaną wartość wskaźnika $val_f1 = 0,7281$, która następnie nieco spadła przy dwóch warstwach ($val_f1 = 0,6736$), mimo wysokiego $train_accuracy$. W przypadku trzech warstw model wyraźnie się załamał – uzyskano bardzo niską skuteczność walidacyjną ($val_f1 = 0,3291$), a metryki precision i recall znacznie spadły. Od czterech warstw wzwyż nastąpiła poprawa zarówno dla 4, jak i 5 warstw model wykazał się wysoką skutecznością (val_f1 odpowiednio 0,8516 i 0,9261), a także bardzo dobrym dopasowaniem ($val_accuracy$ do 0,9103 i val_AUC do 0,997). Jednocześnie nie zaobserwowano istotnego wzrostu val_loss , co potwierdza stabilność treningu.

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
1	30	0,7833	0,0308	0,0718	0,0102	0,0258
2	30	0,7717	0,0905	0,0154	0,0828	0,0487
3	30	0,5166	0,0489	0,1119	0,1068	0,1084
4	30	0,8934	0,0188	0,0454	0,0202	0,0163
5	30	0,9442	0,0319	0,0173	0,0158	0,016

Najwyższy $composite_score$ (0,9442) został osiągnięty dla 5 warstw, co jednoznacznie wskazuje tę konfigurację jako najbardziej efektywną. Wartości wszystkich wskaźników pomocniczych dla tej konfiguracji były bardzo niskie, w tym $pr_diff = 0,0173$, $stability_gap = 0,0158$ oraz $AUC_gap = 0,016$, co potwierdza zrównoważony charakter modelu. Dla trzech warstw odnotowano najgorsze wyniki: $composite_score = 0,5166$, $stability_gap = 0,1068$ oraz $AUC_gap = 0,1084$, co wskazuje na niestabilność procesu uczenia i pogorszenie generalizacji. Wzrost liczby warstw powyżej 3 wyraźnie przyczynił się do poprawy stabilności oraz zmniejszenia rozbieżności pomiędzy metrykami.

Podsumowanie

- Najlepsze wyniki uzyskano dla konfiguracji z 5 warstwami konwolucyjnymi i poolingowymi.
- Model z 3 warstwami wykazywał bardzo niską skuteczność oraz dużą niestabilność, mimo wysokiego dopasowania do danych treningowych.
- Zarówno 4, jak i 5 warstw zapewniły bardzo wysoką jakość predykcji przy zachowaniu dobrej równowagi między precision i recall.
- Przesadnie mała liczba warstw (1-2) ograniczała potencjał klasyfikatora, natomiast konfiguracje 4-5 warstw okazały się optymalne dla analizowanego zestawu danych.

6.2.3.2 Liczba neuronów w warstwie konwolucyjnej (num_conv_filters)

Tab. 8. Zestawienie metryk klasyfikacyjnych dla różnych liczby warstw konwolucyjnych i poolingowych

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
16	23	0,1436	0,1629	0,5378	0,5385	0,7412	0,6814	0,7174	0,6281	0,8281	0,8257	0,7263	0,6461
32	19	0,1353	0,1631	0,5924	0,7564	0,8594	0,8256	0,8151	0,8033	0,8589	0,878	0,8176	0,8079
48	12	0,1522	0,1852	0,5768	0,6859	0,8348	0,8498	0,7148	0,7448	0,8369	0,8822	0,6557	0,7049
64	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
96	30	0,0712	0,1033	0,7878	0,8269	0,9146	0,9082	0,875	0,8419	0,9582	0,9226	0,8675	0,8402
128	12	0,1174	0,1855	0,6875	0,8654	0,8993	0,9067	0,8594	0,8906	0,9096	0,918	0,8523	0,8783
256	30	0,0704	0,1024	0,849	0,859	0,826	0,6576	0,7005	0,6927	0,9755	0,9755	0,637	0,6667

Wraz ze wzrostem liczby neuronów w warstwie konwolucyjnej obserwuje się wyraźną poprawę wyników walidacyjnych aż do wartości 128. Modele z 96 i 128 neuronami osiągnęły najwyższe wartości *val_f1* (0,8402 i 0,8783) oraz *val_AUC* (0,9226 i 0,918), co świadczy o dobrej zdolności generalizacji. Z kolei konfiguracja z 256 neuronami charakteryzowała się najniższą *val_precision* (0,6576) i stosunkowo niskim *val_f1* (0,6667), pomimo wysokiego *val_AUC* (0,9755), co może sugerować nadmierne dopasowanie do klasy dominującej. Modele z mniejszą liczbą neuronów (16-48) notowały słabsze wyniki zarówno pod względem trafności klasyfikacji, jak i wartości strat walidacyjnych.

Tab. 9. Wskaźniki złożone dla różnych liczby warstw konwolucyjnych i poolingowych

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
16	23	0,6785	0,0802	0,0533	0,1076	0,0024
32	19	0,8186	0,0097	0,0223	0,0515	0,0191
48	12	0,7543	-0,0492	0,105	0,019	0,0453
64	30	0,7833	0,0308	0,0718	0,0102	0,0258
96	30	0,8623	0,0273	0,0663	0,0133	0,0356
128	12	0,8876	-0,026	0,0161	0,0129	0,0084
256	30	0,7978	-0,0297	0,0351	0,1923	0

Najwyższy wskaźnik *composite_score* (0,8876) uzyskano przy 128 neuronach, przy jednoczesnym zachowaniu bardzo niskich wartości *pr_diff* (0,0161) i *AUC_gap* (0,0084), co wskazuje na wysoką stabilność i zrównoważenie klasyfikatora. Również wariant 96-neuronowy uzyskał wysoki wynik (0,8623), jednak z nieco wyższymi wartościami *pr_diff* i *AUC_gap*. Niezadowalające wyniki odnotowano przy skrajnie niskich i wysokich wartościach: 16 neuronów charakteryzowało się wysokim *overfitting_gap* (0,0802) i *stability_gap* (0,1076), zaś wariant 256-neuronowy – choć miał zerowy *AUC_gap*, to cechował się największą niestabilnością (*stability_gap* = 0,1923), co znacząco obniżyło jego ogólną ocenę.

Podsumowanie

- Najlepszy wynik osiągnięto dla 128 neuronów w warstwie konwolucyjnej (najwyższy *composite_score*, bardzo niska niestabilność, niski *pr_diff*), co czyni ten wariant najbardziej efektywnym.
- Konfiguracje 96 i 64 neuronów również osiągnęły dobre wyniki i mogą być alternatywą w przypadku ograniczeń obliczeniowych.
- Modele z mniej niż 64 neuronami wykazują wyraźnie słabszą skuteczność klasyfikacji.
- Zbyt duża liczba neuronów (256) prowadzi do utraty równowagi pomiędzy precyzją a czułością oraz dużej niestabilności predykcji.

6.2.3.3 Liczba neuronów w warstwie gęstej (dense_units)

Tab. 10. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby neuronów w warstwie gęstej

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
16	30	0,0965	0,1193	0,6562	0,8205	0,9068	0,8986	0,8841	0,8543	0,8974	0,8752	0,8798	0,8435
32	30	0,1025	0,1331	0,7357	0,6859	0,6237	0,6212	0,7487	0,7273	0,9321	0,9427	0,6654	0,6533
48	30	0,0767	0,1068	0,7422	0,8205	0,8971	0,9091	0,8451	0,8542	0,9407	0,8939	0,8364	0,8463
64	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
96	15	0,1328	0,1803	0,6185	0,8013	0,8723	0,8686	0,8112	0,8385	0,8935	0,8829	0,8112	0,8219
128	13	0,1319	0,1871	0,6354	0,7885	0,8637	0,8515	0,8073	0,8281	0,8859	0,88	0,8054	0,8065
256	13	0,1216	0,1786	0,6602	0,7885	0,8946	0,8994	0,8177	0,8281	0,9061	0,8975	0,8181	0,8303

Zmiana liczby neuronów w warstwie gęstej w zakresie od 16 do 256 przyniosła zróżnicowane rezultaty. Najwyższe wartości *val_accuracy* (0,8205) i *val_f1* (0,8463) zaobserwowano dla 48 neuronów, co wiązało się również z bardzo niskimi wartościami *val_loss* (0,1068) i dużą spójnością *val_precision* i *val_recall* (odpowiednio 0,9091 i 0,8542). Wartość 64 neuronów – przyjęta jako bazowa – również zapewniła dobre wyniki (*val_f1* = 0,7281), choć nieco niższe niż w przypadku 48. Większe liczby neuronów (128, 256) utrzymały relatywnie wysoką skuteczność (*val_f1* $\approx$ 0,83), ale kosztem wzrostu *val_loss* oraz nieco niższego *val_precision* dla 256 neuronów. Z kolei wartość 32 neuronów była wyraźnie mniej skuteczna (*val_f1* = 0,6533), mimo wysokiej *val_AUC*.

Tab. 11. Wskaźniki złożone dla różnych wartości liczby neuronów w warstwie gęstej

wartość	epoka epoch	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
16	30	0,8484	0,0363	0,0443	0,023	0,0222
32	30	0,7466	0,0121	0,1061	0,0326	0,0106
48	30	0,8554	-0,0099	0,0549	0,0258	0,0468
64	30	0,7833	0,0308	0,0718	0,0102	0,0258
96	15	0,8361	-0,0107	0,0301	0,0206	0,0106
128	13	0,825	-0,0011	0,0234	0,018	0,0059
256	13	0,8421	-0,0122	0,0713	0,0418	0,0086

Wartości *composite_score* dla większości konfiguracji przekroczyły 0,83, co świadczy o wysokiej ogólnej jakości predykcji. Najwyższy wynik (0,8554) uzyskała konfiguracja z 48 neuronami, dzięki bardzo niskim wartościom *overfitting_gap*, *pr_diff* i umiarkowanej *stability_gap*. Zbliżony wynik (0,8484) uzyskano przy 16 neuronach, lecz przy wyraźnie wyższym wskaźniku przeuczenia. Najniższy wynik dotyczył 32 neuronów (0,7466), gdzie *pr_diff* wyniósł aż 0,1061 – świadcząc o dużej asymetrii klasyfikacji. Warto także zwrócić uwagę na bardzo niskie

wartości *AUC_gap* (poniżej 0,01) dla neuronów 128 i 256, co wskazuje na wysoką spójność klasyfikatora między treningiem a walidacją.

Podsumowanie

- Najlepsze rezultaty osiągnięto dla 48 neuronów w warstwie gęstej (*val_f1* = 0,8463, *composite_score* = 0,8554).
- Konfiguracje 128 i 256 neuronów były również skuteczne, lecz wykazały oznaki niestabilności lub asymetrii (np. wzrost *pr_diff*).
- Zbyt mała liczba neuronów (np. 32) skutkowała wyraźnym spadkiem skuteczności modelu oraz wzrostem wskaźnika asymetrii.

6.2.3.4 Dropout – warstwy konwolucyjne (*conv_dropout_rate*)

Tab. 12. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika dropout w warstwach konwolucyjnych

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	29	0,0549	0,0835	0,8906	0,8526	0,9155	0,9201	0,8724	0,8802	0,9852	0,9768	0,8635	0,8737
0,1	30	0,061	0,0894	0,8164	0,7051	0,8837	0,8789	0,7826	0,7604	0,9695	0,929	0,7318	0,6901
0,2	30	0,0707	0,0996	0,7904	0,7051	0,8766	0,8789	0,7565	0,7604	0,9585	0,9191	0,6808	0,6901
0,3	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0,4	30	0,0813	0,1113	0,724	0,7885	0,8132	0,8496	0,7747	0,7606	0,9328	0,9071	0,7667	0,7796
0,5	15	0,1654	0,1941	0,5547	0,7692	0,825	0,8371	0,7917	0,8128	0,8075	0,907	0,7656	0,787
0,6	30	0,1278	0,1559	0,569	0,609	0,7778	0,8399	0,6302	0,6122	0,863	0,8366	0,6107	0,5921

Zmiana współczynnika dropout dla warstw konwolucyjnych miała istotny wpływ na jakość klasyfikacji. Najlepsze wyniki pod względem metryk walidacyjnych osiągnięto przy braku dropout (0,0), gdzie model uzyskał najwyższą dokładność walidacyjną (0,8526), precyzję (0,9201), recall (0,8802), AUC (0,9768) oraz F1-score (0,8737). Wraz ze wzrostem wartości dropout obserwowano systematyczne pogorszenie jakości predykcji. Przy wartościach 0,5 i 0,6, mimo wysokiej liczby epok, metryki takie jak *val_f1* (odpowiednio 0,787 oraz 0,5921) znacząco spadły, co świadczy o trudności modelu w generalizacji przy zbyt dużym poziomie regularyzacji.

Tab. 13. Wskaźniki złożone dla różnych wartości współczynnika dropout w warstwach konwolucyjnych

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,0	29	0,9004	-0,0102	0,0399	0,0211	0,0084
0,1	30	0,7648	0,0417	0,1185	0,015	0,0405
0,2	30	0,7618	-0,0093	0,1185	0,015	0,0394
0,3	30	0,7833	0,0308	0,0718	0,0102	0,0258
0,4	30	0,8196	-0,0129	0,089	0,0089	0,0257
0,5	15	0,8194	-0,0214	0,0243	0,0178	0,0995
0,6	30	0,6688	0,0186	0,2277	0,0169	0,0264

Wartość *composite_score* była najwyższa dla dropout = 0,0 (0,9004), co potwierdza skuteczność modeli bez regularizacji konwolucyjnej. Wartości dropout w zakresie 0,1-0,3 wiązały się z umiarkowanym *composite_score* (ok. 0,76–0,78), przy zauważalnie większych różnicach pomiędzy precision a recall (*pr_diff* = 0,1185). Najwyższe wartości *pr_diff* (0,2277 dla dropout = 0,6) oraz wzrost *overfitting_gap* przy tej wartości sugerują utratę równowagi modelu i jego spadek zdolności generalizacji, a *stability_gap* pozostawał niski i stabilny w całym zakresie.

Podsumowanie

- Najlepsze wyniki klasyfikacyjne i najwyższy *composite_score* uzyskano przy braku dropout (0,0).
- Dropout w zakresie 0,1-0,3 poprawia odporność na przeuczenie, ale kosztem spadku jakości predykcji.
- Wartości dropout powyżej 0-4 prowadzą do istotnego pogorszenia wyników i powodują znaczną asymetrię klasyfikatora (wzrost *pr_diff*).

6.2.3.5 Dropout – warstwy gęste (*dense_dropout_rate*)

Tab. 14. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika dropout w warstwach gęstych

wartość	epoka <i>epoch</i>	strata tr. <i>train_loss</i>	strata wal. <i>val_loss</i>	dokł. tr. <i>train_accuracy</i>	dokł. wal. <i>val_accuracy</i>	precyzja tr. <i>train_precision</i>	precyzja wal. <i>val_precision</i>	czułość tr. <i>train_recall</i>	czułość wal. <i>val_recall</i>	AUC tr. <i>train_AUC</i>	AUC wal. <i>val_AUC</i>	F1 tr. <i>train_f1</i>	F1 wal. <i>val_f1</i>
0,0	15	0,1021	0,1566	0,6992	0,7308	0,8458	0,7673	0,7747	0,7815	0,9186	0,8545	0,7689	0,7226
0,1	15	0,1018	0,1593	0,7044	0,7051	0,8619	0,792	0,7604	0,7604	0,9206	0,9025	0,746	0,7065
0,2	13	0,118	0,174	0,681	0,6987	0,8364	0,7622	0,7109	0,7552	0,912	0,8848	0,6877	0,6791
0,3	15	0,124	0,1752	0,6693	0,641	0,6252	0,7013	0,6224	0,678	0,8979	0,8345	0,573	0,6514
0,4	30	0,0717	0,1007	0,7591	0,7949	0,8941	0,9012	0,8164	0,8333	0,9472	0,9101	0,8167	0,837
0,5	18	0,1298	0,1661	0,6042	0,6795	0,7709	0,7627	0,7526	0,7426	0,8745	0,8509	0,7544	0,7458
0,6	11	0,1602	0,2014	0,5716	0,6538	0,802	0,7525	0,7331	0,719	0,8203	0,8622	0,689	0,6758

Wyniki wskazują, że najlepszą skuteczność walidacyjną osiągnięto przy wartości dropout dla warstw gęstych równej 0,4. Dla tej konfiguracji *val_accuracy* wyniosła 0,7949, *val_f1* 0,8370, a *val_AUC* 0,9101 – przy bardzo dobrych wynikach precyzji i czułości. Wartości dropout mniejsze niż 0,4 dawały przeciętne wyniki (np. dla 0,0 – *val_accuracy* = 0,7308), a wartości wyższe (0,5-0,6) powodowały wyraźny spadek skuteczności klasyfikacji (*val_accuracy* do 0,6538, *val_f1* do 0,6758). Warto również zauważyć, że przy braku dropout (0,0) dokładność uczenia (*train_accuracy* = 0,6992) była niska w porównaniu z jakością predykcji walidacyjnych, co może wskazywać na niestabilność modelu bez regularyzacji.

Tab. 15. Wskaźniki złożone dla różnych wartości współczynnika dropout w warstwach gęstych

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,0	15	0,7638	0,0463	0,0142	0,0082	0,0641
0,1	15	0,765	0,0395	0,0316	0,0014	0,0181
0,2	13	0,7447	0,0086	0,007	0,0196	0,0272
0,3	15	0,7042	-0,0784	0,0233	0,0104	0,0634
0,4	30	0,8505	-0,0203	0,0679	0,0421	0,0371
0,5	18	0,7641	0,0086	0,0201	0,0663	0,0236
0,6	11	0,7273	0,0132	0,0335	0,022	0,0419

Zestawienie composite score oraz różnic stabilności i overfittingu jednoznacznie wskazuje na wartość 0,4 jako najefektywniejszą. Osiągnięto najwyższy composite score (0,8505), relatywnie niski overfitting gap (-0,0203), umiarkowaną różnicę precyzji (0,0679) i niski *AUC_gap* (0,0371). Pozostałe konfiguracje wypadały słabiej – np. dropout 0,0 miał co prawda relatywnie dobre wyniki walidacyjne, ale wysoki overfitting (0,0463) i większy *AUC_gap* (0,0641), co pogarsza jego końcową ocenę. Dropout 0,3 z kolei miał najniższy *composite_score* (0,7042) oraz największy *stability_gap* (0,0104) i jedną z najgorszych dokładności walidacyjnych.

Podsumowanie

- Najlepszy wynik osiągnięto przy wartości dropout = 0,4 dla warstw gęstych.
- Brak dropout (0,0) powoduje pogorszenie stabilności oraz większe ryzyko niestabilnych predykcji mimo dobrych metryk walidacyjnych.
- Zbyt wysoka wartość dropout (0,5-0,6) obniża skuteczność klasyfikacji – szczególnie precyzję i f1-score.

- Dropout powyżej 0,3 może prowadzić do nadmiernej regularizacji i spadku wydajności modelu.

6.2.3.6 Regularyzacja jądra L2 (`kernel_regularizer`)

Tab. 16. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika regularyzacji L2

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	16	0,0859	0,1349	0,625	0,6346	0,6143	0,6156	0,7031	0,7031	0,8934	0,8916	0,631	0,6322
0,0001	16	0,0987	0,142	0,6237	0,7436	0,8687	0,8684	0,7891	0,7917	0,8897	0,8668	0,7851	0,781
0,0003	14	0,1209	0,1683	0,6094	0,7692	0,8754	0,8168	0,8008	0,8125	0,8753	0,8664	0,798	0,7675
0,0005	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0,0007	19	0,1301	0,1718	0,6823	0,8205	0,8976	0,9091	0,8359	0,8542	0,9086	0,9436	0,8366	0,8548
0,001	13	0,1697	0,2137	0,6055	0,7821	0,8899	0,8976	0,8112	0,8229	0,8696	0,8635	0,8113	0,8181
0,002	28	0,1456	0,1827	0,7279	0,6218	0,5971	0,6134	0,6823	0,6927	0,9266	0,8755	0,6117	0,6237

Dla niskich wartości dropout (0,0-0,2) obserwujemy stosunkowo zrównoważone metryki: zarówno *val_accuracy*, jak i *val_f1* utrzymują się powyżej 0,70, a najlepsze rezultaty osiąga wartość 0,4 z *val_accuracy* = 0,7949 i *val_f1* = 0,837. Dalsze zwiększanie wartości do 0,5-0,6 skutkuje wyraźnym pogorszeniem skuteczności – spadają wszystkie kluczowe wskaźniki, w tym dokładność walidacyjna i f1. Warto również zwrócić uwagę, że wartość 0,3 doprowadziła do znacznego spadku skuteczności (*val_f1* = 0,6514), mimo że nie była skrajnie wysoka.

Tab. 17. Wskaźniki złożone dla różnych wartości współczynnika regularyzacji L2

wartość	epoka epoch	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,0	16	0,7105	-0,0012	0,0875	0,0024	0,0018
0,0001	16	0,7993	0,0041	0,0767	0,0374	0,0229
0,0003	14	0,7975	0,0305	0,0043	0,0017	0,0089
0,0005	30	0,7833	0,0308	0,0718	0,0102	0,0258
0,0007	19	0,8746	-0,0182	0,0549	0,0343	0,035
0,001	13	0,8245	-0,0068	0,0747	0,036	0,0061
0,002	28	0,6989	-0,012	0,0793	0,0019	0,0511

Najwyższy wynik *composite_score* = 0,8505 osiągnięto przy wartości dropout 0,4. Wartości parametru w zakresie 0,0-0,2 również dawały stosunkowo dobre wyniki (powyżej 0,74), jednak ich *stability_gap* i *AUC_gap* były bardziej zróżnicowane. Zbyt wysoki dropout (0,6) skutkuje istotnym spadkiem jakości (najniższy *composite_score* = 0,7273). Pozytywną cechą ustawienia 0,4 był niski *overfitting_gap* = -0,0203 oraz zrównoważone pozostałe miary.

Podsumowanie

- Optymalny poziom dropout w warstwach gęstych to 0,4 – zapewnia najwyższe *val_f1*, *val_accuracy* oraz *composite_score*.
- Dropout powyżej 0,5 prowadzi do znacznego pogorszenia skuteczności klasyfikacji, co wskazuje na nadmierne ograniczenie przepływu informacji.
- Dropout 0,0 i 0,1 zapewniają umiarkowaną jakość, ale gorszą ogólną stabilność i większe ryzyko przeuczenia.
- Dropout 0,3 przyniósł niewytłumaczalnie słaby wynik, co sugeruje wrażliwość modelu na małe zmiany w tej okolicy.

6.2.3.7 Typ warstwy agregującej (*pooling_type*)

Tab. 18. Zestawienie metryk klasyfikacyjnych dla różnych typów warstwy agregującej

wartość	epoka <i>epoch</i>	strata tr. <i>train_loss</i>	strata wal. <i>val_loss</i>	dokł. tr. <i>train_accuracy</i>	dokł. wal. <i>val_accuracy</i>	precyzja tr. <i>train_precision</i>	precyzja wal. <i>val_precision</i>	czułość tr. <i>train_recall</i>	czułość wal. <i>val_recall</i>	AUC tr. <i>train_AUC</i>	AUC wal. <i>val_AUC</i>	F1 tr. <i>train_f1</i>	F1 wal. <i>val_f1</i>
Flatten	24	0,6765	0,1649	0,8659	1	1	1	1	1	0,9139	1	1	1
GAP2D	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
GMP2D	30	0,1351	0,1087	0,6393	0,9936	0,9898	0,9949	0,9896	0,9949	0,8755	0,9503	0,9896	0,9948

Dla typu Flatten zaobserwowano maksymalne możliwe wartości wszystkich metryk, co wskazuje na przetrenowanie – zarówno dokładność, jak i precyzja, recall oraz f1 osiągnęły wartość 1,0. W przypadku GAP2D, wyniki były bardziej zrównoważone – zarówno metryki treningowe, jak i walidacyjne utrzymywały się na umiarkowanym poziomie, sugerując poprawne uogólnienie modelu. Dla GMP2D, metryki walidacyjne osiągały bardzo wysokie wartości, natomiast metryki treningowe były niższe, co może świadczyć o wyjątkowo dobrej generalizacji lub zbyt małej liczbie epok, by wystąpiło przetrenowanie.

Tab. 19. Wskaźniki złożone dla różnych typów warstwy agregującej

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
Flatten	24	1	0	0	0	0,0861
GAP2D	30	0,7833	0,0308	0,0718	0,0102	0,0258
GMP2D	30	0,9812	-0,0052	0	0,0012	0,0748

Composite_score dla Flatten wyniósł 1,0, co jednoznacznie potwierdza przetrenowanie. Wartości *overfitting_gap*, *pr_diff*, *stability_gap* równe były 0. Dla GAP2D *composite_score* wyniósł 0,7833, a wartości wskaźników rozbieżności były

niskie, z wyjątkiem *pr_diff* (0,0718). GMP2D osiągnął bardzo wysoki *composite_score* (0,9812) i najniższy *stability_gap* (0,0012), co może sugerować skuteczne dopasowanie modelu bez oznak przetrenowania.

Podsumowanie

- Flatten spowodował skrajne przetrenowanie – model osiągnął 100% we wszystkich metrykach, co jest nienaturalne i niepożądane.
- GAP2D zapewnił najbardziej zrównoważoną wydajność – rozsądne metryki i brak oznak przetrenowania.
- GMP2D osiągnął bardzo wysokie metryki walidacyjne oraz stabilność, ale jego stosunkowo niskie metryki treningowe mogą wskazywać na potencjalne niedotrenowanie lub zbyt niski learning rate.
- GMP2D wydaje się być najlepszym wyborem, jeśli celem jest maksymalizacja jakości predykcji bez przetrenowania, przy zachowaniu bardzo dobrej stabilności i uogólnienia.

6.2.3.8 Funkcja aktywacji (*activation_function*)

Tab. 20. Zestawienie metryk klasyfikacyjnych dla różnych funkcji aktywacji

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
ELU	11	0,1095	0,1724	0,651	0,8397	0,9044	0,9155	0,8698	0,8698	0,8943	0,9135	0,8637	0,8612
ReLU	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
Sigmoid	16	0,2816	0,2159	0,2526	0,3654	0,1507	0,1949	0,3021	0,2936	0,5216	0,6865	0,2009	0,2295
Tanh	27	0,0699	0,098	0,7643	0,7949	0,7832	0,8355	0,7669	0,7998	0,9467	0,9436	0,7646	0,8151

Najlepsze wyniki osiągnięto dla funkcji ELU, która przy jedynie 11 epokach treningu uzyskała *val_accuracy* = 0,8397 oraz *val_f1* = 0,8612, zachowując przy tym bardzo wysokie wartości precyzji (*val_precision* = 0,9155) i czułości (*val_recall* = 0,8698). Równie dobre rezultaty dała funkcja Tanh, osiągając *val_f1* = 0,8151 oraz najniższą *val_loss* = 0,098, co świadczy o jej dużej skuteczności predykcyjnej.

Z kolei funkcja ReLU dała przeciętne rezultaty – *val_accuracy* = 0,7179, *val_f1* = 0,7281, a także niższą precyzję i czułość niż ELU i Tanh. Najslabiej wypadła funkcja Sigmoid, której *val_accuracy* = 0,3654 i *val_f1* = 0,2295 sugerują poważne problemy z rozróżnianiem klas i bardzo niską jakość predykcji.

Tab. 21. Wskaźniki złożone dla różnych funkcji aktywacji

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
ELU	11	0,8726	0,0025	0,0457	0,0215	0,0192
ReLU	30	0,7833	0,0308	0,0718	0,0102	0,0258
Sigmoid	16	0,3938	-0,0286	0,0987	0,1359	0,1649
Tanh	27	0,8496	-0,0505	0,0357	0,0202	0,0031

Pod względem wskaźnika złożonego *composite_score* najlepszy wynik uzyskała funkcja ELU (0,8726), co potwierdza jej skuteczność i równowagę pomiędzy jakością klasyfikacji a stabilnością modelu. Na drugim miejscu znalazła się funkcja Tanh (0,8496), która charakteryzowała się najniższą wartością *AUC_gap* = 0,0031 oraz bardzo dobrą stabilnością (*stability_gap* = 0,0202). Funkcja ReLU wypadła umiarkowanie dobrze (*composite_score* = 0,7833), jednak wykazywała większe różnice między wynikami treningowymi a walidacyjnymi.

Najgorszy wynik osiągnęła funkcja Sigmoid (*composite_score* = 0,3938), przy jednocześnie najwyższym *stability_gap* = 0,1359 oraz *AUC_gap* = 0,1649, co wskazuje na dużą niestabilność i brak uogólnienia modelu.

Podsumowanie

- ELU okazała się najskuteczniejszą funkcją aktywacji – najlepszy *composite_score*, wysokie metryki klasyfikacji, szybki czas trenowania (11 epok).
- Tanh była również bardzo skuteczna – niski *val_loss*, dobra stabilność, najniższy *AUC_gap*.
- ReLU dała umiarkowane wyniki – poprawne, ale słabsze od ELU i Tanh, szczególnie w zakresie precyzji i stabilności.
- Sigmoid wypadła zdecydowanie najgorzej – niska skuteczność klasyfikacji, wysoki *val_loss* i *stability_gap*, brak uogólnienia.

6.2.4 Parametry ekstrakcji cech

6.2.4.1 Liczba współczynników MFCC (n_{mfcc})

Tab. 22. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby współczynników MFCC

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
10	30	0,0636	0,091	0,8438	0,8782	0,9245	0,9301	0,8919	0,901	0,9728	0,9659	0,8866	0,8977
15	14	0,0997	0,1472	0,6888	0,8526	0,8958	0,9201	0,8672	0,8802	0,9202	0,9161	0,861	0,8737
25	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
30	13	0,1498	0,1918	0,5872	0,7115	0,78	0,7682	0,7799	0,7682	0,8536	0,9183	0,7799	0,7679
40	20	0,0996	0,1502	0,7018	0,7628	0,8933	0,8832	0,8138	0,8074	0,924	0,8746	0,8148	0,8102
50	18	0,1351	0,1729	0,6458	0,7628	0,8393	0,8205	0,8281	0,8103	0,8886	0,8706	0,8226	0,8025
60	15	0,1632	0,1927	0,5573	0,8077	0,8095	0,8625	0,7656	0,8092	0,8268	0,8994	0,7576	0,8198

Najlepsze wyniki pod względem skuteczności klasyfikacji osiągnięto dla wartości 10 i 15, przy czym dla *tested_value* = 10 uzyskano najwyższą dokładność walidacyjną (0,8782), precyzję (0,9301), czułość (0,9010), AUC (0,9659) oraz miarę F1 (0,8977). Z kolei wartość 25 (czyli domyślna wartość stosowana w innych eksperymentach) charakteryzuje się przeciętnymi wynikami. Wraz ze wzrostem liczby współczynników do 30-60, obserwuje się spadek skuteczności, przy czym wartość 60, mimo wysokiego *val_accuracy* = 0,8077, osiąga niższe precyzje i F1 niż optymalne wartości niższe.

Tab. 23. Wskaźniki złożone dla różnych wartości liczby współczynników MFCC

wartość	epoka epoch	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
10	30	0,9143	-0,0111	0,0291	0,0195	0,0069
15	14	0,8822	-0,0127	0,0399	0,0211	0,0041
25	30	0,7833	0,0308	0,0718	0,0102	0,0258
30	13	0,8017	0,012	0	0,0564	0,0647
40	20	0,82	0,0046	0,0758	0,0474	0,0494
50	18	0,815	0,0201	0,0102	0,0397	0,018
60	15	0,8413	-0,0622	0,0533	0,0121	0,0726

Z drugiej tabeli wynika, że najwyższy *composite_score* uzyskano ponownie dla *tested_value* = 10 (0,9143) oraz 15 (0,8822). Te ustawienia charakteryzują się bardzo niskimi wskaźnikami overfittingu, *pr_diff* oraz *AUC_gap*. Natomiast dla wartości 60 zaobserwowano znaczny ujemny *overfitting_gap* = -0,0622, co sugeruje zbyt silne dopasowanie do zbioru walidacyjnego lub nietypowe zachowanie modelu. Dla wartości 30 zaobserwowano zerowy *pr_diff*, jednak wskaźniki *stability_gap* i *AUC_gap* są najwyższe w całej tabeli, co czyni to ustawienie niestabilnym.

Podsumowanie

- Najlepsze wyniki jakościowe i stabilności modelu osiągnięto przy liczbie współczynników MFCC równej 10.
- Wartość 15 również zapewnia wysoką skuteczność i niski poziom przeuczenia.
- Wartość domyślna 25 zapewnia umiarkowane wyniki, ale ustępuje niższym wartościom.
- Wysokie wartości (40-60) powodują zwiększenie *val_accuracy*, ale kosztem stabilności i precyzji predykcji.
- Liczba MFCC 60 prowadzi do ujemnego wskaźnika *overfitting_gap* oraz pogorszenia miary F1 mimo pozornie dobrej dokładności walidacyjnej.

6.2.4.2 Długość MFCC (*mfcc_length*)

Tab. 24. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości MFCC

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
250	30	0,0809	0,1118	0,7279	0,6795	0,5902	0,6163	0,7096	0,7045	0,9416	0,9294	0,626	0,6377
500	28	0,0808	0,1126	0,7214	0,8462	0,9208	0,9178	0,8841	0,875	0,9388	0,9086	0,8775	0,8675
750	12	0,1447	0,1913	0,5951	0,891	0,8726	0,9129	0,8724	0,9125	0,8641	0,9015	0,8723	0,9123
1000	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
1250	17	0,1282	0,1735	0,6432	0,7436	0,8581	0,869	0,7904	0,7919	0,8816	0,9165	0,7861	0,7949
1500	26	0,0908	0,1242	0,681	0,7692	0,8916	0,8941	0,8086	0,8125	0,9177	0,899	0,7757	0,7828
2000	23	0,09	0,1351	0,6875	0,8654	0,9172	0,9181	0,8763	0,8928	0,9229	0,893	0,8682	0,8878

Najkrótszy czas (250) skutkował wyraźnie niższymi wynikami dla *val_accuracy* = 0,6795 i *val_f1* = 0,6377, co może wynikać z niedostatecznego kontekstu akustycznego. Wydłużenie długości do 500 przyniosło znaczny wzrost skuteczności (*val_accuracy* = 0,8462, *val_f1* = 0,8675), a dalsze zwiększanie długości do 750 dało najwyższy wynik F1 na zbiorze walidacyjnym (*val_f1* = 0,9123) oraz doskonałą precyzję i recall.

Zauważalny jest jednak spadek stabilności wyników przy ekstremalnych wartościach. Dla długości 1250, 1500 i 2000 metryki wciąż są wysokie (*val_accuracy* powyżej 0,86), ale pojawia się wzrost strat walidacyjnych (*val_loss* = 0,1735 dla 1250) oraz większe rozbieżności.

Tab. 25. Wskaźniki złożone dla różnych wartości długości MFCC

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
250	30	0,7336	-0,0117	0,0882	0,0418	0,0122
500	28	0,8756	0,01	0,0428	0,0213	0,0302
750	12	0,9048	-0,04	0,0004	0,0213	0,0374
1000	30	0,7833	0,0308	0,0718	0,0102	0,0258
1250	17	0,8211	-0,0088	0,0771	0,0513	0,0349
1500	26	0,8149	-0,0071	0,0816	0,0136	0,0187
2000	23	0,8849	-0,0196	0,0253	0,0224	0,0299

Druga tabela pozwala dostrzec, że najlepszy wynik *composite_score* (0,9048) osiągnięto przy długości 750, przy jednocześnie najniższym *pr_diff* = 0,0004 i umiarkowanym *overfitting_gap* = -0,04. Bardzo dobry rezultat dał również czas 2000 (*composite_score* = 0,8849) i 500 (0,8756), z zachowaniem niskiej nadmiarowej dopasowalności.

Dłuższe sekwencje MFCC generalnie skutkowały poprawą stabilności predykcji i AUC (średni *AUC_gap* poniżej 0,03), co oznacza większą spójność między zbiorami. Jednak zbyt krótki kontekst (250) nie pozwalał modelowi na uchwycenie kluczowych różnic w sygnale, czego dowodem jest niska precyzja i F1.

Podsumowanie

- Najlepszy wynik klasyfikacji uzyskano dla długości MFCC = 750 (*val_f1* = 0,9123, *composite_score* = 0,9048).
- Długości od 500 do 2000 zapewniają wysoką skuteczność klasyfikacji i stabilność predykcji.
- Zbyt krótka długość (250) prowadzi do znacznego pogorszenia jakości predykcji.
- Zbyt długa długość (np. 2000) może prowadzić do wzrostu czasu obliczeń bez znacznej poprawy wyników.

6.2.4.3 Długość FFT (n_{fft})

Tab. 26. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości FFT

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
256	29	0,0957	0,1275	0,6471	0,7372	0,8856	0,8861	0,7891	0,7865	0,8923	0,8721	0,7434	0,7399
512	30	0,0908	0,1174	0,7201	0,9295	0,625	0,7034	0,75	0,75	0,9282	0,9831	0,6667	0,7243
1024	15	0,146	0,1877	0,5755	0,8397	0,9237	0,8732	0,9089	0,87	0,8582	0,8432	0,9067	0,8535
2048	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
3072	10	0,1656	0,2023	0,5208	0,6795	0,7819	0,7638	0,7539	0,7413	0,8051	0,8647	0,758	0,7444
4096	30	0,082	0,1089	0,7812	0,8077	0,8955	0,9051	0,8229	0,8438	0,9493	0,9463	0,7981	0,8278

Rozkład metryk klasyfikacyjnych dla różnych wartości FFT ujawnia kilka istotnych zależności. Najwyższy wynik dokładności walidacyjnej ($val_accuracy = 0,9295$) osiągnięto przy $FFT = 512$, jednak towarzyszy mu bardzo niska precyzja treningowa ($0,625$) oraz umiarkowana $f1$ -score, co może wskazywać na przypadkowe dopasowanie lub problemy ze stabilnością. $FFT = 4096$ zapewnia bardzo dobre wyniki ogólne – wysoka $val_accuracy = 0,8077$, najlepsze $val_f1 = 0,8278$ oraz wyrównany stosunek precyzji i recall, co wskazuje na skuteczność i równowagę modelu. Z kolei $FFT = 1024$ charakteryzuje się najwyższym $train_f1 = 0,9067$ i bardzo dobrym $val_f1 = 0,8535$, ale wskazuje na silne przetrenowanie (duży rozdźwięk między metrykami treningowymi a walidacyjnymi). $FFT = 3072$ wypada najslabiej pod względem dokładności ($train_accuracy = 0,5208$) oraz skuteczności klasyfikacji ($val_f1 = 0,7444$), co sugeruje trudności z nauką przy tej wielkości FFT.

Tab. 27. Wskaźniki złożone dla różnych wartości długości FFT

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
256	29	0,779	0,0035	0,0996	0,0027	0,0202
512	30	0,843	-0,0576	0,0466	0,2052	0,0549
1024	15	0,8477	0,0532	0,0032	0,0138	0,015
2048	30	0,7833	0,0308	0,0718	0,0102	0,0258
3072	10	0,7675	0,0136	0,0225	0,0649	0,0596
4096	30	0,8593	-0,0297	0,0613	0,0201	0,003

Złożony wskaźnik skuteczności ($composite_score$) potwierdza przewagę niektórych wartości FFT nad innymi. Najwyższy wynik uzyskano dla $FFT = 4096$ ($composite_score = 0,8593$), przy bardzo niskim $AUC_gap = 0,003$ i niewielkim pr_diff oraz $stability_gap$, co wskazuje na spójność i skuteczność działania modelu. Tuż za nim plasuje się $FFT = 1024$ ($composite_score = 0,8477$), który jednak cierpi na przeuczenie ($overfitting_gap = 0,0532$), choć równoważy to bardzo mały $pr_diff = 0,0032$. $FFT = 512$

osiąga dobry wynik (0,843), ale wyróżnia się ekstremalnie wysokim *stability_gap* = 0,2052, co czyni go niestabilnym mimo wysokiej skuteczności na walidacji. Bazowa wartość FFT = 2048 osiąga umiarkowany rezultat (*composite_score* = 0,7833), pozostając w tyle za czołówką.

Podsumowanie

- FFT = 4096 to najbardziej zrównoważona i skuteczna konfiguracja: najwyższy *val_f1*, bardzo dobry *composite_score*, niski *AUC_gap* i stabilna struktura metryk.
- FFT = 1024 oferuje znakomite metryki f1 i precision, ale cierpi na silny overfitting – wskazane jest jego użycie z dodatkowymi mechanizmami regularyzacji.
- FFT = 512 daje świetne wyniki walidacyjne, lecz jego ekstremalnie wysoki *stability_gap* sugeruje niestabilność i potencjalnie losowy charakter skuteczności.
- FFT = 3072 prezentuje się zdecydowanie najgorzej – słabe wyniki klasyfikacji i najwyższy *AUC_gap* wskazują na nieefektywność tej konfiguracji.

6.2.4.4 Długość skoku (hop_length)

Tab. 28. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości skoku

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
128	14	0,1314	0,1796	0,6211	0,7244	0,8709	0,8832	0,7448	0,776	0,8831	0,8953	0,7253	0,7637
192	29	0,0823	0,1154	0,707	0,7821	0,8799	0,8628	0,8021	0,8229	0,9292	0,899	0,7785	0,7957
256	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
384	19	0,1082	0,1456	0,6432	0,8654	0,9133	0,925	0,8672	0,8906	0,9027	0,901	0,8571	0,8859
512	30	0,0814	0,1098	0,7435	0,6923	0,625	0,6263	0,75	0,75	0,9398	0,9099	0,6667	0,6678
1024	18	0,149	0,1769	0,6003	0,75	0,8379	0,8348	0,7904	0,7971	0,8632	0,8729	0,791	0,7804

W analizie podstawowych metryk klasyfikacyjnych długości skoku MFCC (hop_length), wartość 384 próbek wyróżnia się wyraźnie najlepszymi wynikami. Model osiągnął wysoką dokładność walidacyjną (0,8654), precyzję (0,925), czułość (0,8906), F1-score (0,8859) i AUC (0,901), co świadczy o bardzo dobrej jakości klasyfikacji i zrównoważonych proporcjach pomiędzy trafnościami i wykrywalnością. Z kolei najniższe wyniki uzyskano dla wartości 512, gdzie precyzja, czułość i F1-score drastycznie spadają, zwłaszcza po stronie walidacyjnej – F1 zaledwie 0,6678. Dla wartości 256 i 192 uzyskano średni poziom metryk, przy czym 192 przewyższa 256 pod względem ogólnej dokładności, precyzji oraz F1-score na walidacji.

Tab. 29. Wskaźniki złożone dla różnych wartości długości skoku

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
128	14	0,7953	-0,0384	0,1072	0,0393	0,0122
192	29	0,824	-0,0172	0,0399	0,0136	0,0302
256	30	0,7833	0,0308	0,0718	0,0102	0,0258
384	19	0,8863	-0,0288	0,0344	0,0205	0,0017
512	30	0,7453	-0,0011	0,1237	0,0245	0,0299
1024	18	0,8021	0,0106	0,0377	0,0304	0,0097

Najwyższy wynik composite score uzyskano przy długości skoku 384 (0,8863), co potwierdza wyniki z metryk klasyfikacyjnych. Niski overfitting gap (-0,0288) oraz niewielkie różnice w precision-recall (0,0344) i AUC gap (0,0017) wskazują na bardzo stabilny i dobrze generalizujący model. Wysokie wartości stability gap oraz *pr_diff* dla skoku 128 i 512 mogą wskazywać na niestabilność modelu lub nierówną jakość predykcji pomiędzy zbiorem treningowym a walidacyjnym. Długość 256 ma wyraźnie wyższy overfitting gap (0,0308), co sugeruje lekkie przeuczenie przy tej konfiguracji.

Podsumowanie

- Najlepsze rezultaty uzyskano dla długości skoku 384, która zapewnia zarówno wysoką jakość klasyfikacji, jak i stabilność modelu.
- Wartość 192 również zapewnia dobre wyniki, zwłaszcza pod względem balansu precyzja–czułość i umiarkowanego przeuczenia.
- Długość 256 uzyskała przeciętne wyniki – stabilne, ale nieoptymalne, z oznakami lekkiego przeuczenia.
- Skok 512 okazał się zdecydowanie najgorszy – mimo niskiej straty model uzyskuje najgorsze metryki klasyfikacyjne, co wskazuje na brak trafnej reprezentacji cech.

6.2.5 Augmentacja danych

6.2.5.1 Poziom dodawanego szumu (noise_addition_level)

Tab. 30. Zestawienie metryk klasyfikacyjnych dla różnych wartości poziomu dodawanego szumu

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	15	0,1293	0,174	0,6081	0,8846	0,95	0,9	0,9427	0,9062	0,8734	0,9387	0,9423	0,8888
0,01	17	0,118	0,1688	0,668	0,6538	0,7273	0,7623	0,6953	0,6854	0,9045	0,8617	0,7049	0,712
0,02	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0,03	30	0,0787	0,1105	0,7344	0,7372	0,8801	0,8861	0,7695	0,7865	0,9402	0,8983	0,7074	0,7399
0,05	12	0,1433	0,1915	0,5755	0,7885	0,8757	0,8561	0,8021	0,8282	0,8583	0,8859	0,8029	0,8124
0,1	13	0,1398	0,1874	0,5664	0,7436	0,8762	0,8123	0,8086	0,7917	0,854	0,8631	0,8046	0,7513
0,2	13	0,1549	0,1934	0,5013	0,6282	0,7499	0,7226	0,7214	0,701	0,8166	0,8556	0,7313	0,7091

W analizie wpływu poziomu szumu w augmentacji na skuteczność klasyfikacji, najwyższe wartości dokładności walidacyjnej (*val_accuracy*) oraz metryk takich jak *val_f1* i *val_AUC* uzyskano przy braku augmentacji (0,0) – odpowiednio 0,8846, 0,8888 i 0,9387. Wartość *train_f1* również osiągnęła tam maksimum (0,9423), wskazując na bardzo dobrą spójność klasyfikatora na zbiorze treningowym. Dla wartości 0,02 i 0,03 zauważalne są umiarkowane wyniki – *val_f1* wynosi odpowiednio 0,7281 i 0,7399 – z niewielką stratą dokładności względem modelu bazowego, ale wciąż z zachowaniem dobrej równowagi między precision i recall. W przypadku szumu na poziomie 0,1 i 0,2 następuje pogorszenie niemal wszystkich metryk walidacyjnych, szczególnie przy 0,2 – *val_f1* spada do 0,7091, co wskazuje na negatywny wpływ zbyt intensywnej augmentacji szumem.

Tab. 31. Wskaźniki złożone dla różnych wartości poziomu dodawanego szumu

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
0,0	15	0,9029	0,0535	0,0062	0,0042	0,0653
0,01	17	0,7453	-0,0071	0,0769	0,0582	0,0428
0,02	30	0,7833	0,0308	0,0718	0,0102	0,0258
0,03	30	0,7869	-0,0325	0,0996	0,0027	0,0419
0,05	12	0,8297	-0,0095	0,0279	0,0239	0,0276
0,1	13	0,7833	0,0533	0,0206	0,0077	0,0091
0,2	13	0,7369	0,0222	0,0216	0,0809	0,039

Najlepszy wynik złożony (*composite_score*) osiągnięto dla wartości 0,0 – aż 0,9029 co oznacza, że model bez dodanego szumu działa najstabilniej i najdokładniej. Warto zauważyć, że poziomy 0,05 i 0,03 również uzyskały dobre wyniki (odpowiednio 0,8297 i 0,7869), choć przy nieco większych *pr_diff* i *stability_gap*,

co wskazuje na większą niestabilność modelu w prognozach mimo dobrej ogólnej skuteczności. Wartości *overfitting_gap* są minimalne lub wręcz ujemne przy poziomach 0,01, 0,03 i 0,05, co sugeruje, że umiarkowany szum może nieznacznie redukować przeuczenie. Jednak dla wartości 0,2 pojawia się wyraźny wzrost *stability_gap* (0,0809), co sygnalizuje pogorszenie spójności predykcji przy zwiększaniu poziomu zakłóceń.

Podsumowanie

- Brak augmentacji (0,0) daje najlepsze wyniki metryk klasyfikacyjnych i najwyższy wynik złożony, lecz wiąże się z lekkim przeuczeniem (*overfitting_gap* = 0,0535).
- Poziomy szumu 0,02 i 0,03 zapewniają dobry kompromis między stabilnością a skutecznością — przy zachowaniu wysokiej precyzji i relatywnie niskim *AUC_gap*.
- Dodanie zbyt dużego poziomu szumu (0,1-0,2) pogarsza wyniki klasyfikacji – *val_f1* i *val_accuracy* wyraźnie spadają, a *stability_gap* wzrasta.
- Umiarkowany szum (do 0,05) może być korzystny w zapobieganiu przeuczeniu bez dużych strat w dokładności.
- Optymalny zakres poziomu szumu mieści się w przedziale 0,02-0,05 dając dobre wyniki metryk, niskie luki stabilności i umiarkowane przeuczenie.

6.2.5.2 Przesunięcie wysokości dźwięku (*pitch_shift_range*)

Tab. 32. Zestawienie metryk klasyfikacyjnych dla różnych wartości przesunięcia wysokości dźwięku

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	14	0,1397	0,1863	0,6432	0,8654	0,8831	0,8918	0,8828	0,8918	0,8815	0,9471	0,8827	0,8918
1,0	17	0,1125	0,1669	0,7201	0,8526	0,9064	0,8865	0,8724	0,8802	0,92	0,9238	0,8671	0,8597
1,5	19	0,1125	0,1556	0,6172	0,6923	0,7369	0,7908	0,681	0,6809	0,8854	0,8543	0,6826	0,7117
2,5	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
3,5	15	0,1317	0,1806	0,6224	0,641	0,6129	0,6154	0,6966	0,7135	0,8837	0,8652	0,6257	0,6382
4,0	15	0,1236	0,1688	0,6237	0,641	0,6114	0,6167	0,6888	0,7083	0,8873	0,854	0,6192	0,6364
5,0	16	0,1145	0,153	0,6615	0,75	0,8585	0,8405	0,7305	0,7969	0,9005	0,8745	0,7065	0,7633

Wartość *pitch_shift* równa 0,0 daje zaskakująco dobre rezultaty – mimo braku augmentacji, model osiąga najwyższy wynik *val_accuracy* (0,8654) i *val_f1* (0,8918). Dodanie przesunięcia o 1,0 utrzymuje wysoką skuteczność, choć lekko obniża *val_f1* do 0,8597. Wartości od 1,5 do 5,0 skutkują spadkiem dokładności – np. przy 3,5 i 4,0 obserwujemy zauważalny spadek skuteczności (*val_accuracy* $\approx$ 0,641, *val_f1* $\approx$ 0,638).

Tylko przy 5,0 następuje częściowe odbudowanie skuteczności ($val_f1 = 0,7633$). Najlepszy kompromis między skutecznością a generalizacją daje wartość 1,0 lub 2,5.

Tab. 33. Wskaźniki złożone dla różnych wartości przesunięcia wysokości dźwięku

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,0	14	0,9031	-0,0091	0	0,0264	0,0656
1,0	17	0,8775	0,0074	0,0063	0,0071	0,0038
1,5	19	0,7506	-0,0291	0,1099	0,0194	0,0311
2,5	30	0,7833	0,0308	0,0718	0,0102	0,0258
3,5	15	0,7069	-0,0125	0,0981	0,0028	0,0185
4,0	15	0,7026	-0,0172	0,0916	0,0046	0,0333
5,0	16	0,794	-0,0568	0,0436	0,0133	0,026

Najwyższy *composite_score* osiąga 0,0 (0,9031), potwierdzając, że brak przesunięcia może być korzystny w tym przypadku. Wartości 1,0 (0,8775) i 2,5 (0,7833) również wypadają dobrze. Z kolei 3,5 i 4,0 mają najniższe wyniki (ok. 0,70), co wynika z dużych różnic precision oraz lekkiego pogorszenia *AUC_gap*. Dodatkowo, 1,5 wykazuje znaczny *pr_diff* (0,1099), co oznacza dużą nierównowagę między precision a recall. Wskaźnik *overfitting_gap* jest najgorszy dla 5,0 (-0,0568), co wskazuje na znaczną różnicę między skutecznością na zbiorze treningowym i walidacyjnym.

Podsumowanie

- Wartość 0,0 (czyli brak *pitch_shift*) daje najlepsze wyniki pod względem *val_accuracy*, *val_f1* i *composite_score*.
- Niewielkie przesunięcie (1,0) może wspomóc generalizację bez istotnego pogorszenia wyników.
- Zbyt duże wartości *pitch_shift* (3,5-4,0) obniżają skuteczność klasyfikatora.
- Wartość 2,5 jest rozsądnym kompromisem między skutecznością a stabilnością modelu.
- Wartość 1,5 powoduje wyraźne zaburzenie równowagi między precision i recall (*pr_diff* = 0,1099), co może wpływać negatywnie na interpretowalność predykcji.

6.2.5.3 Rozciąganie czasowe dźwięku (time_stretch_probability)

Tab. 34. Zestawienie metryk klasyfikacyjnych dla różnych wartości rozciągania czasowego dźwięku

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	13	0,1446	0,1874	0,5664	0,7564	0,7239	0,8385	0,668	0,7677	0,8547	0,9033	0,6743	0,786
0,2	29	0,085	0,1209	0,7331	0,7885	0,8898	0,8919	0,8177	0,8282	0,9346	0,916	0,8178	0,8332
0,3	17	0,1016	0,1551	0,6523	0,859	0,9071	0,915	0,875	0,8855	0,9039	0,9023	0,8685	0,881
0,4	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0,5	12	0,1469	0,1896	0,5833	0,7372	0,8564	0,7504	0,8138	0,7867	0,8511	0,8962	0,8076	0,7045
0,6	30	0,0738	0,1044	0,7513	0,859	0,9207	0,9225	0,888	0,8854	0,9454	0,9069	0,8824	0,8798
0,8	14	0,1464	0,1908	0,5964	0,7628	0,8933	0,8924	0,8138	0,8073	0,862	0,8624	0,8122	0,8077

W analizie wpływu poziomu rozciągania czasowego dźwięku na skuteczność modelu zauważamy kilka wyraźnych trendów. Najwyższe wartości metryk klasyfikacyjnych, takich jak *val_accuracy* (0,859), *val_precision* (0,915), *val_recall* (0,8855) i *val_f1* (0,881), uzyskano przy *tested_value* = 0,3. Równie dobre wyniki, z nieco wyższym *train_accuracy* i *train_f1*, ale odrobinę niższym *val_AUC*, osiągnięto przy *tested_value* = 0,6. Natomiast zastosowanie wartości granicznych, takich jak 0,0, 0,5 i 0,8, skutkowało wyraźnym pogorszeniem wyników walidacyjnych – zwłaszcza w przypadku *val_f1*, które spadło odpowiednio do 0,786, 0,7045 i 0,8077. Jest to szczególnie widoczne przy braku augmentacji (0,0), gdzie choć *val_accuracy* wyniosło 0,7564, to *train_accuracy* i *train_f1* były wyraźnie niższe. Takie wyniki sugerują, że umiarkowane rozciąganie (0,3-0,6) skutecznie poprawia generalizację modelu, natomiast jego brak lub zbyt duży poziom prowadzi do osłabienia zdolności predykcyjnych.

Tab. 35. Wskaźniki złożone dla różnych wartości rozciągania czasowego dźwięku

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
0,0	13	0,8153	-0,1117	0,0708	0,0296	0,0486
0,2	29	0,8491	-0,0154	0,0637	0,0447	0,0186
0,3	17	0,883	-0,0125	0,0295	0,022	0,0016
0,4	30	0,7833	0,0308	0,0718	0,0102	0,0258
0,5	12	0,7686	0,1031	0,0363	0,0327	0,0451
0,6	30	0,8838	0,0026	0,0371	0,0208	0,0385
0,8	14	0,8151	0,0045	0,0851	0,0449	0,0004

W kontekście wskaźników złożonych *composite_score* oraz różnic stabilności, najlepszy rezultat uzyskano przy *tested_value* = 0,6, gdzie *composite_score* osiągnął 0,8838, a jednocześnie wartości wszystkich wskaźników rozbieżności były niskie. Podobnie dobre zachowanie wykazał model przy *tested_value* = 0,3 (*composite_score* =

0,883, $AUC_gap = 0,0016$), co potwierdza jego stabilność i równowagę pomiędzy precyzją a recall. Z kolei wartości 0,0 i 0,8 cechowały się odpowiednio wysoką i bardzo wysoką niestabilnością ($overfitting_gap = -0,1117$ oraz $pr_diff = 0,0851$). Model przy $tested_value = 0,5$ wyróżniał się największą nadmierną specjalizacją ($overfitting_gap = 0,1031$) i spadkiem skuteczności walidacyjnej mimo wysokiego $train_f1$.

Podsumowanie

- Najlepsze rezultaty klasyfikacyjne i stabilnościowe uzyskano przy $tested_value = 0,3$ oraz $0,6$.
- Zbyt małe (0,0) lub zbyt duże (0,8) rozciąganie skutkowało pogorszeniem stabilności i jakości klasyfikacji.
- $tested_value = 0,6$ cechuje się najlepszym kompromisem pomiędzy skutecznością ($val_f1 = 0,8798$) a niskim pr_diff i AUC_gap .
- Wartości 0,5 i 0,8 powinny być unikane – generują niestabilne modele z dużym $overfitting_gap$.
- Wprowadzenie umiarkowanego rozciągania czasowego znacznie poprawia zdolność generalizacji modelu.

6.2.5.4 Wzmocnienie głośności (volume_gain_range)

Tab. 36. Zestawienie metryk klasyfikacyjnych dla różnych wartości wzmocnienia głośności

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0,0	14	0,138	0,1863	0,6055	0,7821	0,8463	0,8067	0,8372	0,8238	0,8722	0,8713	0,8342	0,8037
1,5	29	0,0763	0,1072	0,7617	0,7885	0,8617	0,8961	0,7969	0,8106	0,9456	0,9249	0,7645	0,799
2,0	17	0,1168	0,167	0,6432	0,7372	0,8157	0,8142	0,7956	0,7901	0,8886	0,8743	0,7865	0,7716
3,0	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
4,0	17	0,1222	0,1677	0,6055	0,6731	0,8513	0,8008	0,737	0,7344	0,8777	0,8719	0,7205	0,6829
6,0	19	0,1106	0,1475	0,6471	0,75	0,7988	0,8337	0,763	0,7802	0,8962	0,8727	0,7682	0,7946
9,0	21	0,1018	0,1495	0,694	0,75	0,8902	0,8791	0,8268	0,797	0,9167	0,8812	0,8268	0,7983

W analizie metryk klasyfikacyjnych dla różnych poziomów wzmocnienia sygnału zauważalna jest pewna stabilność w zakresie wartości dokładności walidacyjnej, oscylującej wokół 0,75-0,79. Najlepsze wyniki model uzyskał przy wartości 1,5, osiągając $val_precision = 0,8961$, $val_recall = 0,8106$ i $val_f1 = 0,799$, przy bardzo niskim $val_loss = 0,1072$. Warto jednak zwrócić uwagę, że przy tej samej wartości pojawia się również jedna z największych różnic precyzji między zbiorem treningowym

a walidacyjnym ($pr_diff = 0,0855$). Dla $gain = 3,0$, mimo nieco słabszych metryk walidacyjnych, obserwujemy najmniejszy $val_loss = 0,1064$, co wskazuje na dobrą stabilność modelu. Wartości $gain$ większe niż 4 prowadzą do nieco bardziej rozproszonych wyników, ale bez gwałtownego pogorszenia skuteczności.

Tab. 37. Wskaźniki złożone dla różnych wartości wzmocnienia głośności

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0,0	14	0,8197	0,0305	0,0171	0,0216	0,0009
1,5	29	0,8347	-0,0345	0,0855	0,0105	0,0207
2,0	17	0,7955	0,0149	0,0241	0,0344	0,0143
3,0	30	0,7833	0,0308	0,0718	0,0102	0,0258
4,0	17	0,7376	0,0376	0,0664	0,0098	0,0058
6,0	19	0,8091	-0,0264	0,0535	0,0446	0,0235
9,0	21	0,8135	0,0285	0,0821	0,0483	0,0355

Wyniki *composite_score* są względnie wysokie w przedziale 0,78-0,83, z wyjątkiem $gain = 4,0$, gdzie spada do 0,7376. Wskaźniki *overfitting_gap* przyjmują wartości zarówno dodatnie, jak i ujemne, ale nie są szczególnie skrajne, co świadczy o umiarkowanej stabilności generalizacyjnej modelu. Szczególnie dobre zrównoważenie obserwujemy przy $gain = 6,0$, gdzie $overfitting_gap = -0,0264$, $pr_diff = 0,0535$, a $stability_gap = 0,0446$, co daje jeden z najlepszych bilansów między stabilnością a dokładnością. Z kolei najniższy $AUC_gap = 0,0009$ występuje przy $gain = 0,0$, co wskazuje na wysoką spójność powierzchni ROC mimo umiarkowanej skuteczności klasyfikacyjnej.

Podsumowanie

- Najwyższe wartości val_f1 i precyzji osiągnięto przy $gain = 1,5$, ale kosztem większego rozrzutu między zbiorami (pr_diff).
- Parametr $gain = 3,0$ zapewnia najniższą stratę walidacyjną, dobrą stabilność i przyzwoite metryki klasyfikacji.
- $gain = 6,0$ oferuje najlepszy kompromis między *composite_score*, stabilnością i różnicą precyzji, co czyni go bardzo konkurencyjnym.
- Wartości $gain$ powyżej 4 nie pogarszają dramatycznie wyników, ale nie oferują już zauważalnych korzyści.

6.2.5.5 Liczba augmentacji na próbkę (`augmentation_multiplier`)

Tab. 38. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby augmentacji na próbkę

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0	30	0,1946	0,2141	0,3929	0,4583	0,3247	0,3333	0,4133	0,4375	0,7178	0,8313	0,3126	0,3393
1	28	0,1315	0,1821	0,6136	0,6905	0,613	0,6173	0,697	0,6979	0,8807	0,8966	0,626	0,6307
2	30	0,0886	0,1394	0,7317	0,6667	0,6136	0,6218	0,7	0,6919	0,9348	0,8934	0,6285	0,6309
3	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
4	30	0,0751	0,0857	0,7437	0,8021	0,8567	0,8638	0,8417	0,8468	0,9409	0,9392	0,8359	0,8408
5	29	0,0652	0,0654	0,7703	0,7982	0,9038	0,8734	0,8788	0,8371	0,9555	0,9688	0,8737	0,8196
6	28	0,0617	0,0603	0,7633	0,8826	0,9154	0,9225	0,8928	0,9104	0,9557	0,9877	0,889	0,907

Wraz ze wzrostem liczby augmentacji zauważalna jest wyraźna poprawa niemal wszystkich podstawowych metryk. Dla modelu trenowanego bez augmentacji (0) uzyskano bardzo niską skuteczność ($val_accuracy = 0,4583$, $val_f1 = 0,3393$), co sugeruje, że sieć nie była w stanie nauczyć się odpowiedniego rozróżniania klas na podstawie ograniczonego zbioru danych. Już przy jednej augmentacji ($tested_value = 1$) następuje znaczna poprawa wyników, a kolejne augmentacje stopniowo zwiększają jakość klasyfikacji. Dla wartości 6 uzyskano najwyższe wartości: $val_accuracy = 0,8826$, $val_AUC = 0,9877$ oraz $val_f1 = 0,907$, co potwierdza skuteczność strategii rozszerzania zbioru danych.

Tab. 39. Wskaźniki złożone dla różnych wartości liczby augmentacji na próbkę

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
0	30	0,5107	-0,0267	0,1042	0,119	0,1135
1	28	0,7224	-0,0047	0,0806	0,0598	0,0159
2	30	0,7168	-0,0024	0,0701	0,0358	0,0414
3	30	0,7833	0,0308	0,0718	0,0102	0,0258
4	30	0,8626	-0,0049	0,017	0,0387	0,0017
5	29	0,8601	0,0541	0,0363	0,0214	0,0133
6	28	0,9263	-0,018	0,0121	0,0244	0,032

Wartość wskaźnika złożonego *composite_score* rośnie systematycznie od 0,51 (dla braku augmentacji) do 0,9263 przy sześciu augmentacjach, co stanowi najwyższy wynik w całym zestawie eksperymentów. Towarzyszy temu wyraźne zmniejszenie *stability_gap* (z 0,119 do 0,0244) oraz *pr_diff* (z 0,1042 do 0,0121), co sugeruje wzrost spójności klasyfikatora i zmniejszenie różnicy między precyzją a czułością. Choć *overfitting_gap* i *AUC_gap* nie są monotoniczne, ich wartości pozostają w akceptowalnych granicach i nie wskazują na nadmierne przeuczenie modelu.

Podsumowanie

- Brak augmentacji skutkuje znacznym obniżeniem jakości klasyfikacji, zarówno pod względem trafności predykcji, jak i spójności modelu.
- Stopniowe zwiększanie liczby augmentacji poprawia wszystkie kluczowe metryki, co świadczy o skuteczności tej techniki w kontekście ograniczonego zbioru danych.
- Najlepsze wyniki uzyskano przy sześciu augmentacjach – zarówno pod względem wskaźników klasyfikacyjnych, jak i złożonych, co czyni tę konfigurację optymalną.

6.2.6 Dane wejściowe i podział zbioru

6.2.6.1 Długość próbki

Tab. 40. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości próbki

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
3s	30	0,0443	0,0412	0,8401	0,8635	0,9159	0,9206	0,8826	0,8917	0,9778	0,9929	0,8764	0,8856
10s	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
30s	30	0,1474	0,1926	0,5933	0,5962	0,7271	0,6419	0,6978	0,6594	0,8525	0,8585	0,6931	0,6225

Długość próbki dźwiękowej miała wyraźny wpływ na skuteczność klasyfikacji. Najlepsze rezultaty uzyskano dla krótkich próbek o długości 3 sekund, gdzie osiągnięto najwyższą dokładność walidacyjną ($val_accuracy = 0,8635$), najniższą wartość błędu walidacyjnego ($val_loss = 0,0412$) oraz najwyższy współczynnik AUC ($val_AUC = 0,9929$). Wraz ze wzrostem długości próbki do 10 i 30 sekund obserwowano stopniowy spadek wszystkich kluczowych metryk, co szczególnie widoczne było dla wariantu 30-sekundowego ($val_accuracy = 0,5962$, $val_f1 = 0,6225$). Wskazuje to na znaczne pogorszenie skuteczności modelu w przypadku zbyt długich okien czasowych.

Tab. 41. Wskaźniki złożone dla różnych wartości długości próbki

wartość	epoka epoch	wskaźnik zł. composite_score	luka przeucz. overfitting_gap	różn. prec. pr_diff	luka stab. stability_gap	różn. AUC AUC_gap
3s	30	0,9134	-0,0092	0,0289	0,0221	0,0151
10s	30	0,7833	0,0308	0,0718	0,0102	0,0258
30s	30	0,688	0,0706	0,0175	0,0263	0,006

Wartość składowa *composite_score* również potwierdza przewagę krótkich próbek: wartość 0,9134 dla 3 sekund była najwyższa spośród wszystkich testowanych,

podczas gdy wynik dla 30-sekundowych próbek wyniósł jedynie 0,688. Parametr *overfitting_gap* dla 3 sekund wynosił -0,0092, co oznacza bardzo dobrą równowagę między zbiorem treningowym a walidacyjnym. Warto też zauważyć najniższy *AUC_gap* dla 30 sekund (0,006), co przy bardzo niskich metrykach ogólnych sugeruje raczej uśrednione dopasowanie niż dobrą generalizację. Najniższy *stability_gap* osiągnięto dla 10 sekund (0,0102), jednak nie rekompensuje to ogólnego spadku jakości predykcji.

Podsumowanie

- Najlepsze rezultaty osiągnięto przy długości próbki 3 sekundy, zarówno pod względem dokładności, F1-score, jak i *composite_score*.
- Wydłużenie próbki do 10 sekund nieco pogorszyło wyniki, choć model wciąż pozostawał skuteczny.
- Próbkę o długości 30 sekund znacząco obniżyły jakość klasyfikacji, co może być skutkiem większego zróżnicowania zawartości dźwiękowej lub rozmycia sygnału diagnostycznego.

6.2.6.2 Podział zbioru

Tab. 42. Zestawienie metryk klasyfikacyjnych dla różnych proporcji podziału zbioru danych

wartość	epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
0.4/0.3/0.3	18	0,116	0,1631	0,6784	0,7885	0,8883	0,8907	0,8333	0,8282	0,9087	0,8753	0,8371	0,8303
0.5/0.3/0.2	13	0,1342	0,1849	0,6172	0,8333	0,8509	0,8621	0,8385	0,8651	0,8763	0,8722	0,839	0,8585
0.6/0.25/0.15	30	0,0768	0,1064	0,7357	0,7179	0,7711	0,7746	0,7604	0,7028	0,9446	0,9188	0,7589	0,7281
0.6/0.2/0.2	30	0,0783	0,1105	0,7331	0,6795	0,8797	0,8737	0,7682	0,7396	0,9415	0,894	0,7095	0,6677
0.75/0.15/0.1	30	0,0779	0,1112	0,7513	0,7564	0,8924	0,8908	0,8112	0,8021	0,941	0,9088	0,7798	0,7664
0.7/0.2/0.1	16	0,128	0,181	0,6641	0,5897	0,6119	0,6084	0,6914	0,6667	0,8966	0,8531	0,6214	0,6012
0.8/0.1/0.1	14	0,1344	0,1884	0,6055	0,6923	0,6187	0,875	0,7435	0,7551	0,8742	0,8913	0,6602	0,6767

W eksperymencie zbadano wpływ różnych proporcji podziału danych treningowych, walidacyjnych i testowych na jakość klasyfikacji. Najlepszy wynik ogólny osiągnięto dla wariantu 0.5/0.3/0.2, który uzyskał wysoką dokładność walidacyjną (0,8333), bardzo dobrą precyzję (0,8621), czułość (0,8651) oraz wskaźnik F1 (0,8585). Pomimo krótszego treningu (13 epok), model ten wykazał dużą skuteczność i dobrą ogólną równowagę między metrykami. Nieco gorsze, ale wciąż stabilne wyniki uzyskał wariant 0.4/0.3/0.3, który charakteryzował się zrównoważoną precyzją, czułością i AUC, a także wysokim F1 na zbiorze walidacyjnym (0,8303). Z kolei

warianty o mniejszych rozmiarach zbioru testowego (0.6/0.25/0.15 i 0.75/0.15/0.1) osiągnęły przeciętne wyniki, natomiast skrajne proporcje (np. 0.8/0.1/0.1) negatywnie wpłynęły zarówno na jakość klasyfikacji, jak i na stabilność predykcji.

Tab. 43. Wskaźniki złożone dla różnych proporcji podziału zbioru danych

wartość	epoka <i>epoch</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	różn. AUC <i>AUC_gap</i>
0.4/0.3/0.3	18	0,8354	0,0068	0,0625	0,0418	0,0334
0.5/0.3/0.2	13	0,8576	-0,0195	0,003	0,0252	0,0041
0.6/0.25/0.15	30	0,7833	0,0308	0,0718	0,0102	0,0258
0.6/0.2/0.2	30	0,7379	0,0418	0,1341	0,0118	0,0475
0.75/0.15/0.1	30	0,8071	0,0134	0,0887	0,01	0,0322
0.7/0.2/0.1	16	0,6745	0,0202	0,0583	0,0115	0,0435
0.8/0.1/0.1	14	0,7442	-0,0165	0,1199	0,0156	0,0171

Najwyższy *composite_score* (0,8576) zanotowano przy podziale 0.5/0.3/0.2, co potwierdza jego znakomitą ogólną jakość – niski *overfitting_gap* (-0,0195), bardzo mała różnica precyzji (0,003) i minimalne luki *stability_gap* (0,0252) oraz *AUC_gap* (0,0041). Dobre wyniki uzyskał także wariant 0.4/0.3/0.3 (0,8354), mimo nieco większych luk stabilności i AUC. Konfiguracja 0.6/0.25/0.15 (standardowa) cechowała się wyższym *overfitting_gap* (0,0308) i przeciętnym *composite_score* (0,7833). Bardzo wysoki *pr_diff* (0,1341) w wariancie 0.6/0.2/0.2 oraz 0,1199 przy 0.8/0.1/0.1 wskazują na brak równowagi między precyzją a czułością, co skutkowało niższą jakością ogólną.

Podsumowanie

- Najlepszą jakość klasyfikacji osiągnięto dla proporcji zbioru 0.5/0.3/0.2, która zapewniła:
 - wysoką jakość predykcji na zbiorze walidacyjnym,
 - bardzo niski poziom nadmiernego dopasowania i niestabilności,
 - najwyższy wskaźnik *composite_score*.
- Wariant 0.4/0.3/0.3 również okazał się skuteczny, choć mniej stabilny.
- Zwiększenie rozmiaru zbioru treningowego kosztem testowego (np. 0.8/0.1/0.1) prowadziło do spadku skuteczności modelu i wyższych różnic precyzji.

6.2.7 Dobór optymalnej konfiguracji parametrów modelu

W celu wyznaczenia optymalnej konfiguracji modelu, dokonano zestawienia wyników wszystkich dotychczasowych eksperymentów w jednej tabeli porównawczej. Dla każdego parametru wybrano tę wartość, która osiągnęła najwyższy wynik *final_score* metrykę podsumowującą skuteczność klasyfikacji z uwzględnieniem pozostałych wskaźników jakości. Sposób jej wyznaczenia został przedstawiony w rozdziale 6.2. Tabela zawiera również wartość złożoną *composite_score*, a także wskaźniki pomocnicze: *overfitting_gap*, *pr_diff*, *stability_gap* i *AUC_gap* – na potrzeby pełnej oceny wpływu danego parametru na stabilność i generalizację modelu.

Tab. 44. Zestawienie najlepszych wyników dla wszystkich testowanych parametrów modelu

parametr	wartość	epoka epoch	<i>final_score</i>	wskaźnik zł. <i>composite_score</i>	luka przeucz. <i>overfitting_gap</i>	różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>	luka AUC <i>AUC_gap</i>
Dropout – warstwy gęste	0,4	30	0,4955	0,8505	-0,0203	0,0679	0,0421	0,0371
Dropout – warstwy konwolucyjne	0,0	29	0,5327	0,9004	-0,0102	0,0399	0,0211	0,0084
Długość FFT	4096	30	0,5024	0,8593	-0,0297	0,0613	0,0201	0,003
Długość MFCC	750	12	0,5319	0,9048	-0,04	0,0004	0,0213	0,0374
Długość próbek	3s	30	0,5415	0,9134	-0,0092	0,0289	0,0221	0,0151
Długość skoku	384	19	0,5215	0,8863	-0,0288	0,0344	0,0205	0,0017
Funkcja aktywacji	ELU	11	0,5165	0,8726	0,0025	0,0457	0,0215	0,0192
Liczba augmentacji na próbkę	6	28	0,5481	0,9263	-0,018	0,0121	0,0244	0,032
Liczba epok	50	41	0,5533	0,935	-0,015	0,0218	0,0175	0,0336
Liczba współczynników MFCC	10	30	0,5421	0,9143	-0,0111	0,0291	0,0195	0,0069
Liczba warstw konwolucyjnych i poolingowych	5	30	0,5568	0,9442	0,0319	0,0173	0,0158	0,016
Liczba neuronów w warstwie gęstej	48	30	0,5021	0,8554	-0,0099	0,0549	0,0258	0,0468
Liczba neuronów w warstwie konwolucyjnej	128	12	0,5247	0,8876	-0,026	0,0161	0,0129	0,0084
Podział zbioru	0.5/0.3/0.2	13	0,5089	0,8576	-0,0195	0,003	0,0252	0,0041
Poziom dodawanego szumu	0,0	15	0,5269	0,9029	0,0535	0,0062	0,0042	0,0653
Rozciąganie czasowe dźwięku	0,3	17	0,5232	0,883	-0,0125	0,0295	0,022	0,0016
Przesunięcie wysokości dźwięku	0,0	14	0,5354	0,9031	-0,0091	0	0,0264	0,0656
Regularyzacja jądra L2	0,0007	19	0,5122	0,8746	-0,0182	0,0549	0,0343	0,035
Typ warstwy agregującej	Flatten	24	0,5957	1	0	0	0	0,0861
Rozmiar batcha	16	25	0,5371	0,9047	-0,0007	0,0371	0,0208	0,0174
Współczynnik uczenia	0,0005	30	0,5339	0,9067	-0,0342	0,006	0,0228	0,0306
Wzmocnienie głośności	1,5	29	0,4838	0,8347	-0,0345	0,0855	0,0105	0,0207

Każdy wiersz tabeli odpowiada jednej wartości danego parametru przy pozostałych parametrach domyślnych.

Najlepsze wartości *final_score* osiągnięto dla następujących ustawień:

- **Typ warstwy agregującej: Flatten** – jedyny przypadek osiągnięcia idealnego *composite_score* = 1 oraz zerowych wartości luk.
- **Liczba warstw konwolucyjnych i poolingowych: 5** – najwyższy wynik wśród parametrów architektury modelu, wskazujący na korzystny kompromis między złożonością a generalizacją.
- **Liczba epok: 50** – dłuższy czas treningu prowadził do poprawy skuteczności przy jednoczesnym utrzymaniu niskiego *overfitting_gap* i wysokiego AUC.
- **Liczba augmentacji: 6** – świadczy o pozytywnym wpływie intensywnego wzbogacania danych na jakość klasyfikacji.
- **Długość próbki: 3 sekundy** – krótsze fragmenty okazały się bardziej reprezentatywne w kontekście detekcji usterek.

Warto zauważyć, że *composite_score* sam w sobie nie wystarczał do trafnej oceny modelu – np. dla wartości dropout w warstwach konwolucyjnych *composite_score* = 0,9004, ale dopiero relatywnie niski *pr_diff* = 0,0399 i małe gapy przełożyły się na wysoki *final_score* = 0,5327. Odwrotnie, parametr wzmocnienie głośności osiągnął względnie dobry *composite_score* = 0,8347, lecz przez bardzo wysoki *pr_diff* = 0,0855 oraz inne nierównowagi wynik końcowy został zdegradowany do najniższego miejsca w rankingu. Powyższe zestawienie umożliwiło wybór najbardziej zrównoważonego zestawu parametrów, które zostały wykorzystane do budowy modelu finalnego.

W oparciu o to zestawienie opracowano końcową konfigurację modelu, łącząc wszystkie parametry, które indywidualnie osiągnęły najwyższe rezultaty. Tak dobrany zestaw wartości został następnie użyty do przeprowadzenia treningu modelu podsumowującego. Trening ten przeprowadzono z użyciem wszystkich danych oraz ustalonego podziału zbioru, bez dalszych modyfikacji architektury.

Tab. 45. Zestawienie wyników końcowej konfiguracji modelu

epoka epoch	strata tr. train_loss	strata wal. val_loss	dokł. tr. train_accuracy	dokł. wal. val_accuracy	precyzja tr. train_precision	precyzja wal. val_precision	czułość tr. train_recall	czułość wal. val_recall	AUC tr. train_AUC	AUC wal. val_AUC	F1 tr. train_f1	F1 wal. val_f1
2	0,2956	0,2604	0,9978	1	1	1	1	1	0,9998	1	1	1
epoka epoch	<i>final_score</i>		wskaźnik zł. <i>composite_score</i>		luka przeucz. <i>overfitting_gap</i>		różn. prec. <i>pr_diff</i>	luka stab. <i>stability_gap</i>		luka AUC <i>AUC_gap</i>		
2	0,6		1		0		0	0		0,0002		

Ostateczny model wykazał wyjątkowo wysoką skuteczność klasyfikacyjną już w początkowych fazach treningu. W szczególności, w 2. epoce osiągnięto idealne

wartości wszystkich metryk walidacyjnych – dokładności, precyzji, czułości, F1-score i AUC – każda z nich wyniosła dokładnie 1,0, co oznaczało całkowitą zgodność prognoz modelu z rzeczywistymi etykietami, a dodatkowo przyjęty najważniejszy do oceny modelu wskaźnik *final_score* = 0,6 (przy czym maksymalna możliwa wartość tej metryki to 1,0).

Co istotne, tak wysokie wyniki pojawiały się już w pierwszych kilku epokach treningu. Od epoki 5. wzwyż metryki utrzymywały się na poziomie bliskim 1,0, a ich niewielkie wahania świadczyły wyłącznie o naturalnej dynamice procesu uczenia. W późniejszych epokach, takich jak 21., 41. i 46. odnotowano przejściowy wzrost straty walidacyjnej, jednak nie towarzyszyło temu pogorszenie kluczowych metryk klasyfikacyjnych, co potwierdza odporność modelu na lokalne zakłócenia. Uzyskany model może być uznany za wzorcowy – jego wyniki na zbiorze walidacyjnym odpowiadają niemal idealnemu klasyfikatorowi.

6.3 Testy działania systemu w środowisku rzeczywistym

Końcowym i zarazem najistotniejszym etapem pracy było przeprowadzenie testów systemu diagnostyki w warunkach rzeczywistych. To właśnie tutaj weryfikowana była nie tylko skuteczność wytrenowanego modelu, lecz także ogólna użyteczność i odporność całej aplikacji na zmienność sygnałów, niepewność danych i ogólnie pojęte warunki środowiskowe. W przeciwieństwie do wcześniejszych eksperymentów, które bazowały na danych znanych systemowi, testy te miały na celu sprawdzenie, jak dobrze system radzi sobie z nowymi, nieznanymi danymi oraz jak stabilnie reaguje na zmianę warunków operacyjnych.

Przeprowadzona analiza pozwala odpowiedzieć na kluczowe pytania dotyczące praktycznej skuteczności, doboru optymalnych parametrów monitoringu oraz ryzyka błędnych diagnoz. Wyniki tego rozdziału stanowią zatem bezpośrednią odpowiedź na cele pracy oraz potwierdzenie, że opracowany system może być z powodzeniem wdrażany w rzeczywistych scenariuszach diagnostycznych.

6.3.1 Konfiguracja monitoringu

W celu rzetelnego przetestowania działania systemu diagnostycznego w warunkach zbliżonych do rzeczywistych przygotowano zestaw eksperymentów opartych na nowych, nieznanymi wcześniej nagraniach dźwiękowych. Dane testowe

obejmowały łącznie 40 minut materiału audio, pochodzącego z czterech różnych klas: pracy prawidłowej (*normal*) oraz trzech odmiennych rodzajów usterek: *obcieranie_lopat*, *uszkodzone_lozysko* i *krzywa_lopata*. Nagranie każdej kategorii trwało 10 minut i nie zostało użyte na żadnym etapie treningu ani walidacji modelu, choć rejestrowano je w tym samym środowisku i przy użyciu tego samego sprzętu, co dane treningowe. Dzięki temu możliwe było uzyskanie naturalnej reprezentacji pracy maszyny, przy jednoczesnym zachowaniu pełnej neutralności danych względem systemu.

Z przyczyn technicznych oraz dla zwiększenia powtarzalności pomiarów, testy nie były wykonywane w czasie rzeczywistym za pomocą mikrofonu, lecz poprzez moduł monitoringu, który analizował wcześniej nagrane pliki audio. Mimo to, cały mechanizm przetwarzania i klasyfikacji dźwięku działał w identyczny sposób, jak w przypadku transmisji na żywo.

Przy testach uwzględniono następujące czynniki:

1) Długość próbki (duration) – czas trwania pojedynczego fragmentu dźwięku analizowanego przez model:

- 3 sekundy (krótka analiza, szybka reakcja),
- 10 sekund (domyślna wartość),
- 30 sekund (większa stabilność, kosztem opóźnień).

2) Częstotliwość próbkowania (sample_rate) – parametr determinujący rozdzielczość czasową sygnału:

- 8 *kHz* (niska jakość, mniejsze zasoby),
- 16 *kHz* (kompromisowa jakość),
- 48 *kHz* (najwyższa jakość i szczegółowość).

3) Rodzaj klasyfikacji (classification_type):

- Binarna (binary) – klasyfikacja typu *Praca normalna* vs *Usterka* (prawdopodobieństwa klas usterek są sumowane),

- Wieloklasowa (multiclass) – klasyfikacja rozdzielająca każdą klasę usterki indywidualnie.

4) Progi klasyfikacyjne (t_{low} , t_{high}) – zestawy progów prawdopodobieństwa usterki, determinujące sposób interpretacji wyników:

- $t_{low} = 20\%$, $t_{high} = 80\%$
- $t_{low} = 30\%$, $t_{high} = 70\%$
- $t_{low} = 40\%$, $t_{high} = 60\%$

Dobór wartości progów oparto na probabilistycznych założeniach związanych z liczbą klas i oczekiwanym poziomem pewności modelu. Ponieważ w trybie wieloklasowym model musi rozróżniać pomiędzy czterema klasami, jego wartość graniczna prawdopodobieństwa wynosi ok. 25%. Ustawienie progów poniżej i powyżej tej wartości umożliwia kontrolę poziomu zaufania – im wyższy próg t_{high} , tym bardziej konserwatywna klasyfikacja i mniejsze ryzyko błędów typu fałszywie pozytywnego.

6.3.2 Skuteczność klasyfikacji w warunkach rzeczywistych

6.3.2.1 Długość próbki

Tab. 46. Zestawienie skuteczności klasyfikacji dla różnych długości próbki sygnału

wartość	dokł. całk. <i>overall_acc</i>	dokł. normal <i>normal_acc</i>	dokł. krzywa <i>krzywa_acc</i>	dokł. obcieranie <i>obcieranie_acc</i>	dokł. łożysko <i>lozysko_acc</i>
3s	0,9738	1	1	1	0,895
10s	0,9958	0,9833	1	1	1
30s	0,9625	0,9048	0,95	1	1

Najlepszą ogólną skuteczność (0,9958) uzyskano dla długości 10 sekund, co potwierdza, że przy takim czasie próbki model otrzymuje wystarczająco dużo informacji, by rozróżniać klasy, jednocześnie nie wprowadzając zbyt wielu zmiennych zakłócających. W tej konfiguracji udało się osiągnąć idealną detekcję wszystkich klas usterek oraz bardzo wysoką skuteczność dla klasy *normal* (0,9833).

Dla najkrótszej próbki (3 s) dokładność ogólna wyniosła 0,9738, czyli nadal bardzo wysoki poziom. System poprawnie identyfikował wszystkie przypadki *normal*, *krzywa_lopata* i *obcieranie_lopat*, jednak nieco gorzej radził sobie z rozpoznawaniem klasy *uszkodzone_lozysko* (0,895). Sugeruje to, że pewne cechy tej klasy mogą wymagać dłuższego czasu analizy, by być jednoznacznie wykrywalne.

W przypadku najdłuższej próbki (30 s) zaobserwowano obniżenie skuteczności (0,9625) – przede wszystkim spadła skuteczność klasyfikacji dźwięków pracy normalnej (do 0,9048) oraz klasy *krzywa_lopata* (0,95). Wydłużenie próbki mogło doprowadzić do większej zmienności w obrębie jednego fragmentu sygnału, co mogło osłabić jednoznaczność reprezentacji cech.

Podsumowanie

- 10-sekundowa próbka stanowi najbardziej wyważoną i skuteczną długość okna analizy w testowanych warunkach.
- Zbyt krótkie próbki (3 s) mogą nie rejestrować w pełni cech istotnych dla bardziej subtelnych usterek (np. łożyska).
- Zbyt długie próbki (30 s) narażone są na heterogeniczność dźwięku, co może prowadzić do rozmycia reprezentacji cech.

6.3.2.2 Rodzaj klasyfikacji

Tab. 47. Zestawienie skuteczności klasyfikacji dla różnych trybów klasyfikacji

wartość	dokł. całk. <i>overall_acc</i>	dokł. normal <i>normal_acc</i>	dokł. krzywa <i>krzywa_acc</i>	dokł. obcieranie <i>obcieranie_acc</i>	dokł. łożysko <i>lozysko_acc</i>
wieloklasowa	0,9958	0,9833	1	1	1
binarna	1	-	-	-	-

W przeprowadzonych testach klasyfikacja binarna osiągnęła 100% poprawnych decyzji (1,0) na całym zestawie danych testowych. Tak wysoka skuteczność sugeruje, że model z bardzo dużą pewnością potrafi odróżnić stany prawidłowe od dowolnej postaci zakłócenia pracy maszyny, niezależnie od jego konkretnego typu.

Z kolei klasyfikacja wieloklasowa, choć nieco trudniejsza zadaniowo, również wykazała się znakomitą skutecznością. Ogólna dokładność wyniosła 0,9958, a skuteczność rozpoznawania usterek osiągnęła 100% dla każdej z klas *krzywa_lopata*, *obcieranie_lopat* i *uszkodzone_lozysko*. Jedynym zauważalnym odchyleniem był niewielki spadek dokładności dla klasy *normal* (0,9833), co może wynikać z bardziej konserwatywnego działania modelu przy rozróżnianiu subtelnych odchyleń od pracy nominalnej.

Podsumowanie

- Klasyfikacja binarna zapewnia absolutną skuteczność wykrywania zakłóceń, ale nie dostarcza informacji o ich przyczynie.
- Klasyfikacja wieloklasowa umożliwia identyfikację konkretnego rodzaju usterki przy zachowaniu bardzo wysokiej dokładności.

6.3.2.3 Progi klasyfikacji

Tab. 48. Zestawienie skuteczności klasyfikacji dla różnych progów decyzyjnych

wartość	dokł. całk. <i>overall_acc</i>	dokł. normal <i>normal_acc</i>	dokł. krzywa <i>krzywa_acc</i>	dokł. obcieranie <i>obcieranie_acc</i>	dokł. łożysko <i>łożysko_acc</i>
30%/70%	0,9958	0,9833	1	1	1
20%/80%	0,9958	0,9833	1	1	1
40%/60%	0,9833	0,9672	0,9833	0,9833	1

Zarówno próg 20%/80%, jak i 30%/70% pozwoliły uzyskać niemal perfekcyjną klasyfikację. Dokładność ogólna wyniosła 0,9958, a wszystkie klasy usterek zostały wykryte ze 100% skutecznością. Jedynie klasa *normal* miała nieznacznie niższą dokładność 0,9833 co może wynikać z ostrożniejszego podejścia modelu przy klasyfikacji obiektów granicznych.

Przy zastosowaniu bardziej zawężonego przedziału niepewności (40%/60%) zaobserwowano spadek dokładności ogólnej do 0,9833 oraz pogorszenie wyników. Może to świadczyć o zbyt małej tolerancji modelu na niepewność predykcji w tym ustawieniu, skutkującej bardziej arbitralną klasyfikacją przypadków granicznych.

Podsumowanie

- Ustawienie progów klasyfikacyjnych ma wpływ na czułość systemu i stabilność rozróżniania klas.
- Najlepsze wyniki uzyskano przy progach 20%/80% i 30%/70% – oferują one kompromis między czułością a precyzją klasyfikacji.
- Zbyt wąski zakres niepewności (40%/60%) może prowadzić do wzrostu liczby błędnych klasyfikacji w przypadkach trudnych.

6.3.2.4 Częstotliwość próbkowania

Tab. 49. Zestawienie skuteczności klasyfikacji dla różnych częstotliwości próbkowania sygnału audio

wartość	dokł. całk. <i>overall_acc</i>	dokł. normal <i>normal_acc</i>	dokł. krzywa <i>krzywa_acc</i>	dokł. obcieranie <i>obcieranie_acc</i>	dokł. łożysko <i>łożysko_acc</i>
48000	0,9958	0,9833	1	1	1
16000	0,9833	0,9672	0,9833	0,9833	1
8000	0,9833	0,9672	0,9833	0,9833	1

Najwyższą skuteczność uzyskano dla częstotliwości próbkowania 48 kHz. Dokładność ogólna wyniosła 0,9958, a wszystkie klasy usterek zostały poprawnie rozpoznane. Obniżenie częstotliwości do 16 kHz i 8 kHz skutkowało niewielkim spadkiem dokładności ogólnej do 0,9833, z zauważalnym, choć niewielkim pogorszeniem wykrywalności klasy *normal*. Co istotne, wyniki dla klas usterek pozostały bardzo wysokie, co oznacza, że kluczowe cechy anomalii są nadal zachowywane nawet przy ograniczonej rozdzielczości czasowej.

Podsumowanie

- Najwyższa skuteczność klasyfikacji została osiągnięta przy pełnym próbkowaniu 48 kHz.
- Redukcja częstotliwości do 16 kHz lub 8 kHz może być rozważana jako kompromis między skutecznością a wydajnością.
- Klasy usterek są rozpoznawalne nawet przy niższej częstotliwości próbkowania.

6.3.3 Dobór optymalnej konfiguracji parametrów monitoringu

Na podstawie przeprowadzonych eksperymentów można jednoznacznie wskazać, że najlepsze wyniki klasyfikacji w warunkach rzeczywistych uzyskano przy następującej konfiguracji parametrów:

- **Długość próbki:** 10 sekund
- **Rodzaj klasyfikacji:** binarna
- **Progi klasyfikacji:** 30%/70%
- **Częstotliwość próbkowania:** 48 kHz

Z użyciem powyższej konfiguracji model był w stanie osiągnąć ogólną dokładność przy rzeczywistym monitoringu na poziomie dokładnie **100%**.

Oznacza to, że wszystkie próbki testowe zostały poprawnie sklasyfikowane jako reprezentujące stan prawidłowy lub usterkę. Co istotne, nawet przy przejściu na klasyfikację wieloklasową skuteczność pozostawała bardzo wysoka, jednak tylko konfiguracja binarna dała wynik bezbłędny. Duża częstotliwość próbkowania (48 kHz) oraz umiarkowana długość próbki (10 s) zapewniły bogaty materiał akustyczny, który umożliwił dokładną analizę i detekcję kluczowych cech sygnału. Progi klasyfikacyjne 30%/70% zapewniły dodatkowo optymalny balans między czułością a stabilnością predykcji. Konfiguracja ta może być rekomendowana jako bazowy wariant do zastosowań praktycznych.

Działanie systemu w czasie rzeczywistym było również testowane z użyciem mikrofonu, natomiast ze względu na mniejszą powtarzalność takich pomiarów (np. różnice w ustawieniu mikrofonu czy poziomie tła akustycznego), nie zostały one uwzględnione w ścisłych porównaniach eksperymentalnych. Niemniej jednak, zarówno w klasyfikacji binarnej, jak i wieloklasowej system również osiągał bardzo dobre wyniki. Jedynymi zaobserwowanymi odchyłkami od pełnej skuteczności były sytuacje, w których np. podczas testu obcierania śmigieł o przedmioty, elementy zakłócające nie były jeszcze całkowicie włożone, co skutkowało słabszym poziomem dźwięku usterki. W takich przypadkach model w niektórych sytuacjach klasyfikował próbkę jako *podejrzanie usterki* zamiast *usterka*, co jednak mieści się w akceptowalnym zakresie błędu i potwierdza skuteczność systemu nawet w warunkach nieoptymalnych.

6.4 Podsumowanie i wnioski z eksperymentów

System diagnostyki akustycznej opracowany w ramach niniejszej pracy wykazał bardzo wysoką skuteczność zarówno na etapie treningu, jak i monitorowania w czasie rzeczywistym. Kluczową obserwacją jest odporność modelu na modyfikacje większości parametrów oraz jego stabilność działania w różnych konfiguracjach. W najlepszych przypadkach dokładność klasyfikacji osiągnęła 99,6%, co przekłada się na praktyczną niezawodność wykrywania usterek w warunkach przemysłowych.

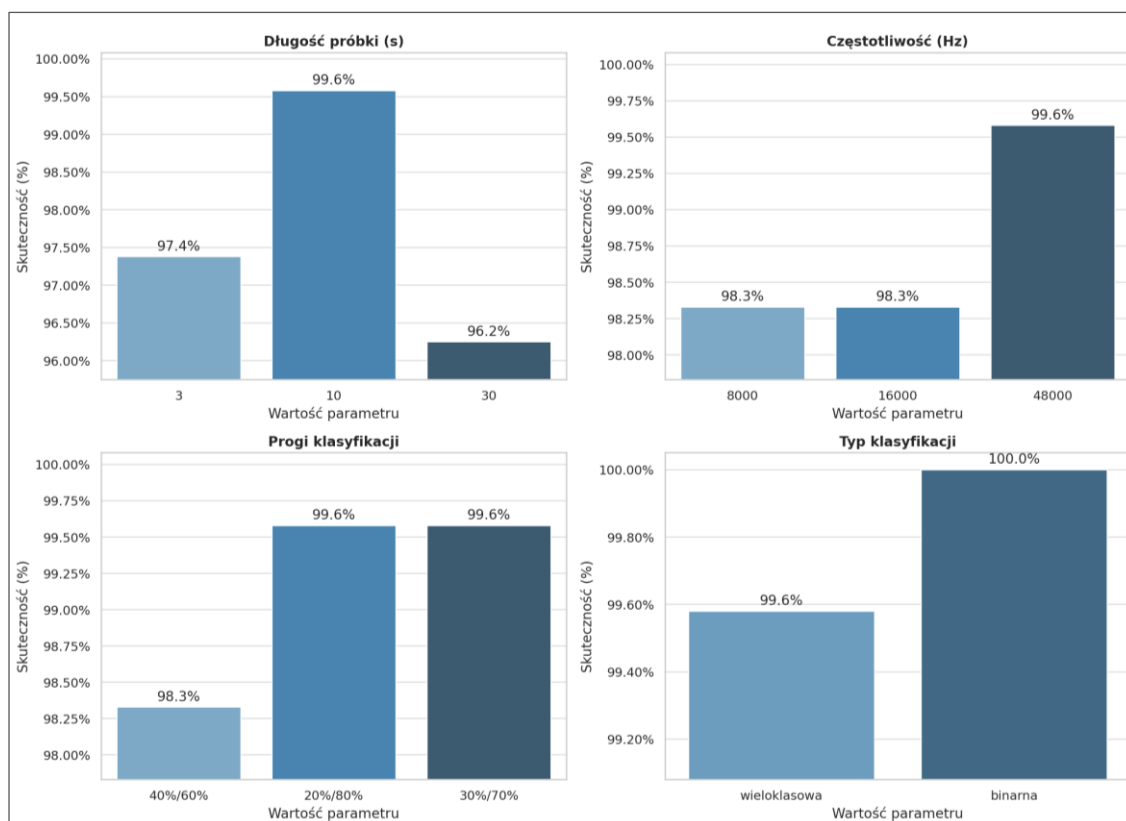
Architektura sieci neuronowej oparta na dwóch warstwach konwolucyjnych i dwóch gęstych okazała się wystarczająco uniwersalna – zmiany liczby filtrów,

neuronów czy wartości dropout miały minimalny wpływ na wyniki, o ile wartości mieściły się w rozsądnych zakresach. Takie zachowanie świadczy o dużej tolerancji modelu na parametry strukturalne i umożliwia jego łatwą adaptację do platform o ograniczonej mocy obliczeniowej, takich jak mikrokontrolery.

Jednym z najistotniejszych odkryć była zależność skuteczności wykrywania stanu od długości analizowanej próbki dźwiękowej. Próbki o długości 3 sekund zapewniały bardzo wysoką skuteczność klasyfikacji (97,4%), porównywalną z wynikami dla 30 sekund (98,3%) i tylko nieznacznie niższą niż dla optymalnych 10 sekund (99,6%). Co szczególnie istotne, nawet najkrótsza próbka umożliwiała skuteczną detekcję wszystkich klas usterek, co wskazuje, że charakterystyczne anomalie akustyczne występowały już na krótkich fragmentach sygnału.

Z praktycznego punktu widzenia krótsze próbki oznaczają szybsze działanie systemu i mniejsze wymagania sprzętowe. Przykładowo, w aplikacjach przemysłowych, gdzie kluczowe jest szybkie reagowanie (np. w trybie alertowym), zastosowanie 3–5-sekundowego okna analizy może pozwolić na niską latencję działania bez utraty jakości detekcji.

Jednocześnie zauważono, że dłuższe próbki (30 s) częściej generowały błędne klasyfikacje stanu *normal*, co można tłumaczyć większym zróżnicowaniem dźwięków w długim sygnale – zawierającym zmiany trybu pracy, hałas tła lub chwilowe zakłócenia, które mogą zacierać wzorce charakterystyczne dla klasy.



Rys. 16. Wpływ wybranych ustawień monitoringu na skuteczność klasyfikacji

Eksperymenty dotyczące liczby współczynników MFCC wykazały zaskakujący efekt: najlepsze wyniki osiągnięto przy zastosowaniu zaledwie 10 współczynników, co przełożyło się na najwyższą skuteczność klasyfikacji – 99,58%. Stopniowe zwiększanie liczby współczynników do 15, 20, 40 czy 50 nie poprawiło wyników, a wręcz je pogarszało. To odkrycie ma duże znaczenie praktyczne – sugeruje, że najbardziej istotne informacje diagnostyczne dla analizowanej maszyny są zakodowane w najniższych pasmach częstotliwości.

W klasycznych systemach dźwiękowych przyjmuje się zazwyczaj zakres 13-20 współczynników MFCC, ale niniejsze wyniki wskazują, że dla konkretnego zastosowania – diagnostyki usterek – mniej może znaczyć więcej. Uproszczenie parametrów może przynieść realne korzyści w systemach o ograniczonej mocy obliczeniowej, np. wbudowanych systemach na mikrokontrolerach.

Równie ciekawe obserwacje przyniosły testy związane z częstotliwością próbkowania. Choć najwyższą skuteczność uzyskano przy 48 kHz (pełne pasmo), to obniżenie do 16 kHz i nawet 8 kHz skutkowało tylko niewielkim spadkiem skuteczności (do 98,33%). Oznacza to, że kluczowe informacje akustyczne potrzebne do

detekcji usterek są zawarte w dolnym zakresie pasma, co ponownie potwierdza użyteczność systemu w warunkach przemysłowych – nawet z tanimi przetwornikami i mikrofonami. Zastosowanie takich uproszczeń pozwala obniżyć koszt sprzętu i zwiększyć energooszczędność systemu, co ma szczególne znaczenie w przypadku mobilnych urządzeń diagnostycznych lub wdrożeń w warunkach ograniczonego zasilania.

Testy związane z ustawieniem progów klasyfikacji pokazały, że system wykazuje wysoką odporność na zmiany poziomu czułości. Konfiguracje z progami 20%/80% i 30%/70% dawały identycznie wysoką skuteczność klasyfikacji (99,58%), natomiast dopiero znaczne zawężenie strefy niepewności do 40%/60% powodowało spadek dokładności (do 98,33%). Wnioski płynące z tej obserwacji są jednoznaczne – system działa skutecznie, gdy zachowuje się wyraźną strefę niepewności. Pozwala to użytkownikowi na elastyczne dostosowanie poziomu wrażliwości, np. w aplikacjach wymagających dużej precyzji można zawęzić progi, a w aplikacjach alarmowych – rozluźnić je. Oznacza to, że operator może dopasować zachowanie systemu do konkretnego scenariusza.

W eksperymentach porównano również dwa tryby klasyfikacji. Tryb binarny, który informuje tylko o tym czy wystąpiła usterka, osiągnął 100% czułości dla przypadków uszkodzeń, ale nie pozwalał zidentyfikować ich rodzaju. Tryb wieloklasowy rozróżniał konkretne typy usterek z dokładnością 99,58%, co pozwala na znacznie bardziej precyzyjne działania serwisowe.

Wybór trybu powinien być uzależniony od aplikacji:

- W zastosowaniach alarmowych (np. linie produkcyjne) tryb binarny jest wystarczający i bardziej niezawodny.
- W zastosowaniach serwisowych lub predykcyjnych tryb wieloklasowy oferuje istotne informacje diagnostyczne przy niemal tej samej skuteczności.
- System umożliwia płynne przełączanie między tymi trybami, co znacznie zwiększa jego uniwersalność i zastosowanie w różnych środowiskach przemysłowych.

Jednym z ważniejszych aspektów badań była analiza wpływu parametrów architektury modelu na skuteczność klasyfikacji. Eksperymenty wykazały, że system cechuje się wysoką odpornością na zmiany architektoniczne, co jest istotne w kontekście wdrożeń w środowiskach o ograniczonych zasobach sprzętowych.

Zmiany liczby filtrów w warstwach konwolucyjnych (np. 16 i 32), liczby neuronów w warstwach gęstych (np. 32 i 16), typu warstwy agregującej (Flatten vs GAP), zastosowanej funkcji aktywacji (ReLU, ELU) czy nawet stopnia dropout (np. 0.3/0.3) nie miały istotnego wpływu na końcowy wynik klasyfikacji, dopóki mieściły się w rozsądnych granicach. To oznacza, że model można łatwo przenieść na inne platformy, takie jak mikrokomputery typu Raspberry Pi czy mikrokontrolery ARM, bez konieczności długiego strojenia.

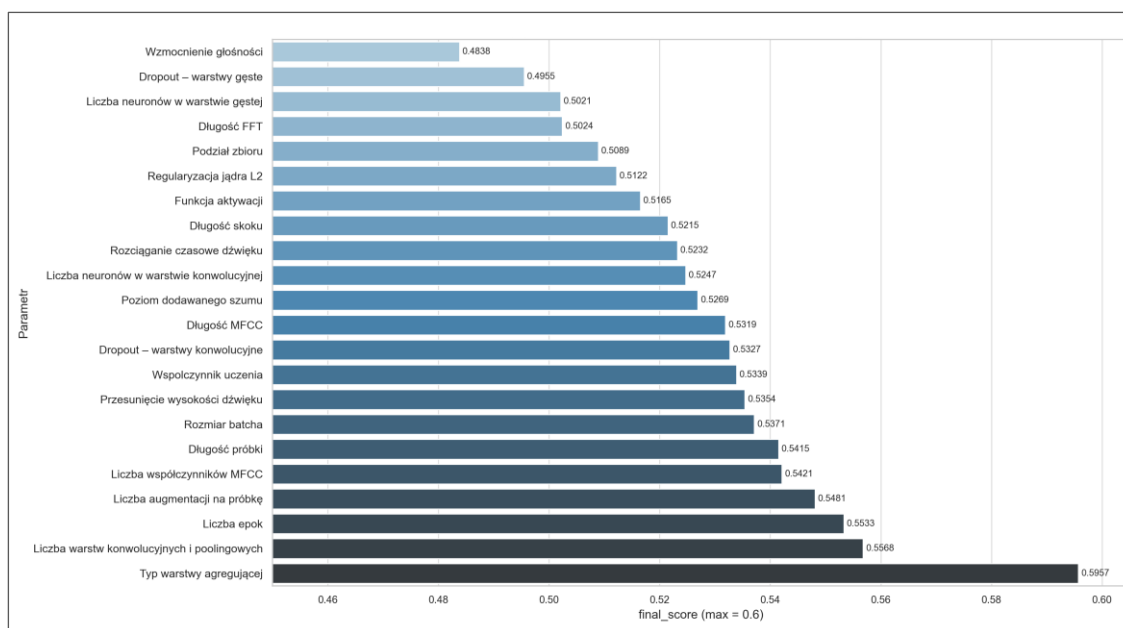
Z punktu widzenia praktycznego wdrożenia oznacza to również, że system można łatwo optymalizować pod kątem wydajności, np. poprzez redukcję złożoności modelu kosztem minimalnych strat dokładności.

Kolejnym kluczowym zagadnieniem była analiza stabilności systemu oraz odporności na fałszywe alarmy.

Testy wykazały, że:

- System praktycznie nie generuje fałszywych pozytywów w klasie *normal* – nawet przy zwiększonej czułości
- Błędne klasyfikacje dotyczyły głównie trudniejszych do uchwycenia usterek lub nietypowych warunków akustycznych,
- Większość błędów miała charakter łagodnego przesunięcia między klasami usterek, co nie wpływało negatywnie na decyzję o interwencji.

Taka charakterystyka oznacza, że system nie przeciąża operatora niepotrzebnymi alertami, co jest ogromną przewagą nad wieloma istniejącymi rozwiązaniami, które cierpią na tzw. "fatygę alarmową". W aplikacjach przemysłowych to właśnie nadmiar błędnych ostrzeżeń często prowadzi do ignorowania rzeczywistych zagrożeń.



Rys. 17. Wpływ poszczególnych parametrów na końcowy wynik jakości modelu (*final_score*)

Choć testy przeprowadzono na przykładzie wentylatora, należy podkreślić, że zaprojektowana architektura i metody ekstrakcji cech nie są ściśle zależne od konkretnego typu maszyny. System można łatwo przystosować do innych urządzeń przemysłowych emitujących sygnały akustyczne, takich jak pompy, silniki, sprężarki czy łożyska liniowe. Wystarczy jedynie odpowiednie nagranie danych treningowych, bez konieczności modyfikacji architektury modelu czy kodu aplikacji. Przeprowadzone eksperymenty wykazały, że zaprojektowany system diagnostyki akustycznej spełnia wymagania nie tylko pod względem skuteczności, ale także elastyczności, odporności na zakłócenia i praktyczności wdrożeniowej.

Wnioski z badań pozwalają na sformułowanie następujących, syntetycznych obserwacji o szczególnym znaczeniu praktycznym i badawczym:

1) Skuteczność detekcji nie wymaga długich nagrań.

Już 3-sekundowe próbki pozwalają osiągnąć ponad 97% skuteczności klasyfikacji, co umożliwia błyskawiczną reakcję systemu i ograniczenie zasobów (pamięć, obliczenia). To czyni system idealnym do zastosowań mobilnych i niskomocowych.

2) Najmniejsza liczba współczynników MFCC (10) była najskuteczniejsza.

To nietypowa obserwacja – oznacza, że dla konkretnych maszyn istotne informacje zawarte są w niższych pasmach częstotliwości. Można więc znacząco uprościć ekstrakcję cech i przyspieszyć cały proces analizy bez utraty jakości.

3) Klasyfikacja binarna i wieloklasowa mają różne zastosowania.

W trybie binarnym uzyskujemy 100% detekcji usterek, ale bez informacji o ich rodzaju – idealne dla systemów ostrzegawczych. Tryb wieloklasowy (np. serwisowy) umożliwia wskazanie typu usterki przy nadal bardzo wysokiej skuteczności (ok. 99,6%).

4) Szeroka strefa niepewności nie pogarsza działania.

Próg 30/70% był równie skuteczny jak bardziej rygorystyczny 20/80%, a dopiero przy 40/60% zauważono spadek jakości. To oznacza, że użytkownik może regulować czułość i tolerancję systemu w zależności od zastosowania (np. produkcja vs testy R&D).

5) System działa skutecznie nawet przy małym próbkowaniu.

Redukcja z 48 kHz do 16 kHz lub 8 kHz miała minimalny wpływ na skuteczność. To otwiera drogę do użycia tanich mikrofonów i integracji z prostym sprzętem pomiarowym.

6) Stabilność systemu i odporność na fałszywe alarmy.

Brak nadmiarowych alertów przy pracy normalnej i niewielka liczba błędnych klasyfikacji między klasami usterkowymi świadczą o wysokiej dojrzałości rozwiązania. To sprzyja zaufaniu użytkownika i zwiększa ergonomię pracy.

7) Architektura modelu ma dużą tolerancję.

Zmiany liczby neuronów, filtrów, warstw agregujących czy dropout nie wpływają znacząco na wynik, co pozwala na elastyczne dostosowanie systemu do różnych platform sprzętowych i ograniczeń.

7. Podsumowanie i perspektywy rozwoju

7.1 Ograniczenia obecnego rozwiązania

Pomimo wielu zalet, obecna wersja systemu posiada pewne ograniczenia natury technicznej i diagnostycznej, które należy wziąć pod uwagę przy praktycznym zastosowaniu oraz dalszym doskonaleniu rozwiązania. Pierwszym ograniczeniem jest zakres danych treningowych i klas usterek. W ramach projektu system został nauczony rozróżniać ograniczony zestaw stanów: jedynie trzy konkretne typy uszkodzeń mechanicznych wentylatora oraz stan normalny. Taki zestaw klas pozwolił co prawda na skuteczną demonstrację działania algorytmu, jednak nie obejmuje on pełnego spektrum możliwych uszkodzeń maszyn. W realnych zastosowaniach liczba potencjalnych usterek jest znacznie większa – obecny model nie rozpoznaje awarii innego rodzaju niż te uwzględnione w treningu, a skuteczność klasyfikacji nowych, nieznanych modelowi stanów będzie znacznie ograniczona. System w aktualnej postaci działa więc w trybie klasyfikacji zamkniętego zbioru zdarzeń (ang. *closed set*), co oznacza, że potrafi wykryć tylko te nieprawidłowości, z którymi miał do czynienia podczas uczenia. Rozszerzenie wiedzy modelu wymaga stałego dostarczania dodatkowych danych i ponownego treningu. Sam proces pozyskania i przygotowania reprezentatywnych nagrań audio bywa utrudniony ze względu na to, że wiele poważnych usterek występuje rzadko lub rozwija się długo, a to ogranicza dostępność materiału dźwiękowego do nauki. Każda maszyna może emitować nieco inne sygnały akustyczne nawet przy podobnym uszkodzeniu, zatem ograniczona ilość i różnorodność danych testowych stanowi znaczne wyzwanie. W praktyce przemysłowej może być konieczne gromadzenie obszernych baz nagrań dla różnych maszyn i scenariuszy awarii, aby zapewnić modelowi odpowiednią zdolność uogólniania.

Kolejnym ograniczeniem są warunki środowiskowe i skalowalność systemu. Chociaż prototyp z powodzeniem przetestowano w środowisku zbliżonym do rzeczywistego, to jednak typowa hala przemysłowa cechuje się wysokim poziomem hałasu otoczenia, pogłosem oraz współwystępowaniem dźwięków z wielu maszyn. W takich warunkach akustycznych może dojść do pogorszenia czułości i precyzji diagnostyki. Obecny system nie posiada zaawansowanych mechanizmów separacji dźwięków czy adaptacji do zmiennej akustyki pomieszczenia i zakłada się, że mikrofon

zbiera dominujący sygnał z monitorowanej maszyny. Przy dużym zagęszczeniu urządzeń lub głośnych zakłóceniach, model klasyfikacyjny może popełniać więcej błędów lub generować alarmy na skutek nietypowych odgłosów niezwiązanych z diagnozowanym obiektem. Skalowanie systemu na wiele maszyn implikuje też wyzwania wydajnościowe: przetwarzanie w czasie rzeczywistym wielu równoległych strumieni audio wymaga odpowiedniej mocy obliczeniowej i przepustowości systemu akwizycji danych. Prototyp został stworzony i przetestowany dla pojedynczego źródła dźwięku; jego działanie w przypadku rozbudowy do kilkunastu czy kilkudziesięciu punktów pomiarowych nie było analizowane. Może się okazać, że konieczna będzie optymalizacja kodu lub zastosowanie bardziej wydajnego sprzętu w celu utrzymania pracy w czasie rzeczywistym przy dużej skali systemu. Należy mieć na uwadze, że wraz ze zwiększeniem liczby rozróżnianych klas usterek rośnie złożoność zadania klasyfikacji, zwiększając tym samym prawdopodobieństwo pojawienia się błędnych klasyfikacji – modele muszą wtedy rozstrzygać między wieloma możliwymi stanami, często o podobnych cechach akustycznych. Dlatego rozszerzanie systemu na kolejne typy awarii będzie wymagało nie tylko dodatkowych danych, ale i szczególnej ostrożności przy utrzymaniu wysokiej skuteczności (np. poprzez bardziej złożone modele lub dodatkowe cechy, by uniknąć pomyłek między klasami).

W kontekście implementacji i funkcjonalności aplikacji również można wskazać pewne ograniczenia typowe dla prototypowych rozwiązań. Interfejs użytkownika zaprojektowany w ramach pracy zapewnia jedynie podstawowe opcje konfiguracji i podglądu wyników – nie był on testowany pod kątem intensywnego użytkowania ani integracji z istniejącymi systemami przemysłowymi. Obecnie system funkcjonuje jako samodzielna aplikacja; brak jest gotowych mechanizmów połączenia np. z systemem SCADA czy bazami danych utrzymania ruchu. Ponadto, choć architektura modelu jest stosunkowo lekka, uruchamianie modułu sieci neuronowej w środowisku Python może nie być optymalne z punktu widzenia wykorzystania zasobów w warunkach przemysłowych. W dłuższej perspektywie czasowej może pojawić się potrzeba przeniesienia kluczowych algorytmów do bardziej wydajnych środowisk w celu zapewnienia stabilności i szybkości działania.

Reasumując, aktualna wersja systemu spełnia swoje zadanie w warunkach laboratoryjnych i testowych, ale przed pełnym wdrożeniem przemysłowym wymaga uwzględnienia opisanych ograniczeń: rozszerzenia bazy wiedzy (danych i klas),

przystosowania do trudnych warunków akustycznych, zapewnienia skalowalności oraz dopracowania integracji i wydajności oprogramowania.

7.2 Możliwości wdrożenia systemu w przemyśle

Opracowany system diagnostyki akustycznej maszyn przemysłowych może znaleźć szerokie zastosowanie w rzeczywistych warunkach przemysłowych. Dzięki elastycznej architekturze i modułowej budowie, rozwiązanie to daje się łatwo zaadaptować do różnych rodzajów urządzeń emitujących dźwięk i nie jest ono ściśle zależne od konkretnego typu maszyny. Mimo że testy przeprowadzono na przykładzie wentylatora, system można przystosować do diagnostyki innych maszyn, takich jak pompy, silniki elektryczne, sprężarki, przekładnie czy łożyska, bez konieczności modyfikacji kodu czy architektury modelu – wymagane jest jedynie zgromadzenie odpowiednich danych treningowych z nowych urządzeń i odpowiednia konfiguracja parametrów. Ta uniwersalność sprawia, że proponowane rozwiązanie nadaje się do wdrożenia w różnych gałęziach przemysłu, od zakładów produkcyjnych i montażowych, przez systemy transportowe, po instalacje energetyczne, wszędzie tam, gdzie monitoring stanu technicznego maszyn jest kluczowy dla utrzymania ciągłości pracy.

Istotną zaletą systemu jest możliwość działania w czasie rzeczywistym, co stanowi warunek skutecznego wdrożenia w przemyśle. Zaprojektowany moduł monitorowania potrafi na bieżąco przechwytywać sygnał z mikrofonów i natychmiast analizować go pod kątem anomalii akustycznych. Dzięki temu możliwe jest ciągle nasłuchiwanie wielu urządzeń jednocześnie – na przykład sieć czujników akustycznych rozmieszczonych w hali produkcyjnej może stale monitorować kluczowe maszyny, a centralne oprogramowanie będzie automatycznie generować alarm, gdy tylko rozpozna charakterystyczny dźwięk świadczący o usterce. Niewątpliwym atutem proponowanego rozwiązania jest również niski koszt i prostota wdrożenia w porównaniu do tradycyjnych metod diagnostycznych. System wykorzystuje standardowy przetwornik akustyczny (mikrofon) jako czujnik – jest one tańszy i łatwiejszy w instalacji niż specjalistyczne czujniki drgań czy tensometry, a dodatkowo nie wymaga fizycznego kontaktu z maszyną. Dzięki temu implementacja systemu nie ingeruje w proces technologiczny i może być prowadzona nawet na istniejących, działających już instalacjach. Co więcej, eksperymenty pokazały, że system działa skutecznie nawet przy obniżonych parametrach akwizycji, np. zmniejszonej częstotliwości próbkowania dźwięku, co otwiera drogę do

użycia tańszych mikrofonów i mniej wydajnego sprzętu komputerowego bez znaczącego pogorszenia wyników diagnostycznych. To szczególnie ważne z perspektywy przemysłowej, ponieważ umożliwia tworzenie mobilnych lub rozproszonych węzłów monitorujących przy zachowaniu wysokiej skuteczności rozpoznawania usterek.

Na podstawie wyników uzyskanych w pracy można stwierdzić, że system spełnia kluczowe wymagania stawiane przemysłowym systemom diagnostycznym. Osiągnięta wysoka skuteczność klasyfikacji (rzędu 97, 99%, a nawet 100% w wykrywaniu usterek) gwarantuje wiarygodność diagnostyki akustycznej porównywalną z klasycznymi metodami, a jednocześnie niski odsetek fałszywych alarmów zapewnia użyteczność w praktyce.

Podsumowując, zaproponowany system może realnie wzbogacić istniejące strategie utrzymania ruchu w przemyśle, dostarczając narzędzie do wczesnego, automatycznego wykrywania usterek na podstawie dźwięku, które jest elastyczne, szybkie i ekonomiczne w implementacji.

7.3 Kierunki dalszego rozwoju i badań nad systemem

Zidentyfikowane ograniczenia wskazują jednoznacznie na obszary, w których warto kontynuować prace badawczo-rozwojowe nad systemem. Pierwszym naturalnym kierunkiem rozwoju jest rozszerzenie zakresu diagnostycznego modelu – planuje się zwiększenie liczby rozpoznawanych klas usterek i stanów pracy. Wymagać to będzie zebrania szerszego zbioru danych z różnych maszyn i sytuacji awaryjnych, a następnie ponownego wytrenowania sieci neuronowej. Równolegle rozważane jest przetestowanie innych architektur głębokich sieci neuronowych, zwłaszcza konwolucyjnych i ich odmian. Obecny model CNN okazał się skuteczny, jednak istnieje możliwość poprawy wyników lub zwiększenia efektywności poprzez eksperymenty z różnymi konfiguracjami. W dalszych badaniach planuje się porównać działanie alternatywnych architektur, np. głębszych sieci konwolucyjnych, modeli hybrydowych łączących cechy CNN i RNN do uwzględnienia zależności czasowych czy też sieci opartej na transformatrach dedykowanych danym audio.

Drugim ważnym obszarem dalszych usprawnień jest poszerzenie funkcjonalności i integracja systemu z innymi narzędziami. Jedną z propozycji jest włączenie dodatkowych typów sensorów i stworzenie rozwiązania multimodalnego – połączenie

danych akustycznych z informacjami z czujników wibracji, temperatury czy pomiarów elektrycznych. Taka fuzja danych mogłaby znacząco zwiększyć niezawodność diagnostyki, pozwalając wykrywać usterki, które nie zawsze manifestują się wyraźnie w samym sygnale dźwiękowym. Integracja systemu akustycznego z istniejącymi systemami monitorowania stanu maszyn w ramach koncepcji Przemysłu 4.0 stanowi naturalny krok – w przyszłości przewiduje się wyposażenie platformy w interfejsy API umożliwiające wymianę danych z zewnętrznymi aplikacjami i bazami.

Z punktu widzenia zastosowań praktycznych istotne będzie także przetestowanie systemu na większą skalę w rzeczywistych warunkach – planowane jest wdrożenie pilotażowe na wybranych obiektach przemysłowych w celu oceny stabilności działania w długim horyzoncie czasowym. Tego rodzaju testy dostarczą danych do dalszej optymalizacji algorytmów pracy w czasie rzeczywistym, w tym ewentualnego usprawnienia wydajności przetwarzania. Można rozważyć implementację krytycznych fragmentów kodu w językach niskopoziomowych lub wykorzystanie specjalizowanych platform sprzętowych (jak moduły FPGA/SoC z akceleratorami AI) celem redukcji opóźnień analizy.

7.4 Uwagi końcowe

Przeprowadzone prace pozwoliły zrealizować postawiony cel, jakim było zaprojektowanie i implementacja prototypu inteligentnego systemu do diagnostyki akustycznej maszyn. Osiągnięte rezultaty potwierdzają, że zastosowanie zaawansowanej analizy sygnałów dźwiękowych oraz algorytmów głębokiego uczenia stanowi efektywne podejście do automatycznego rozpoznawania usterek. Opracowany system uzyskał bardzo wysoką skuteczność w wykrywaniu awarii na podstawie nagrań audio, jednocześnie zachowując wymaganą szybkość działania i elastyczność konfiguracji. Takie połączenie cech sprawia, że rozwiązanie ma dużą wartość praktyczną, z uwagi na możliwość przyczynienia się do poprawy strategii utrzymania ruchu poprzez wcześniejsze wychwytywanie symptomów uszkodzeń urządzeń. Wdrożenie opisanego systemu w środowisku przemysłowym potencjalnie przełoży się na zwiększenie bezpieczeństwa i niezawodności maszyn, redukcję nieplanowanych przestojów oraz obniżenie kosztów związanych z awariami.

Niniejsza praca stanowi przykład skutecznej integracji metod przetwarzania sygnałów, technik uczenia maszynowego i wiedzy inżynierskiej z obszaru diagnostyki.

Uzyskane doświadczenia i wyniki mogą zostać wykorzystane jako baza do kolejnych projektów rozwijających inteligentne systemy monitorowania stanu technicznego urządzeń. Zaproponowana aplikacja spełniła postawione wymagania, a uzyskane wyniki dowodzą, że tego rodzaju rozwiązanie może realnie wspierać utrzymanie ruchu maszyn. Przedstawione wnioski i perspektywy rozwoju wytyczają kierunek dalszych badań, których realizacja pozwoli uczynić system jeszcze bardziej wszechstronnym i niezawodnym. Reasumując, cele pracy zostały osiągnięte, a powstały prototyp stanowi solidną podstawę do rozwijania kolejnych generacji inteligentnych systemów diagnostyki akustycznej, mogących znaleźć zastosowanie w nowoczesnym przemyśle.

Spis źródeł literaturowych

- [1] U. Zölzer *et al.*, “Digital Audio Signal Processing Third Edition,” 2022.
- [2] K. B. T. Shaikh, N. P. Jawarkar, and V. Ahmed, “Machine diagnosis using acoustic analysis: A review,” in *2021 IEEE Conference on Norbert Wiener in the 21st Century: Being Human in a Global Village, 21CW 2021*, Institute of Electrical and Electronics Engineers Inc., Jul. 2021. doi: 10.1109/21CW48944.2021.9532537.
- [3] F. Ahmad, S. Ahsan, A. Kumar, G. Sarwer, and S. Sonu, “Electric Motor Fault Detection using Artificial Intelligence,” 2024.
- [4] O. O. Abayomi-Alli, R. Damaševičius, A. Qazi, M. Adedoyin-Olowe, and S. Misra, “Data Augmentation and Deep Learning Methods in Sound Classification: A Systematic Review,” Nov. 01, 2022, *MDPI*. doi: 10.3390/electronics11223795.
- [5] Żółtowski B., “Deskryptory diagnostycznego systemu eksploatacji maszyn,” Bydgoszcz, 2002.
- [6] E. P. Kowalski C. T., “Zastosowanie sieci neuronowych do monitorowania nieosiowości napędów elektrycznych z silnikami indukcyjnymi,” 2009.
- [7] F. Banaras and W. Nasir, “Real-Time IoT-Based Monitoring of Mechanical Systems Using Acoustic and Vibration Data,” 2025, doi: 10.13140/RG.2.2.26261.64487.
- [8] Szymaniec S.; Kasperak M., *Utrzymanie ruchu w przemyśle*. Warszawa: Wydawnictwo Naukowe PWN, 2021.
- [9] International Organization for Standardization, “Condition monitoring and diagnostics of machines — Thermography — General procedures. ISO 17359:2018; Condition monitoring and diagnostics of machines — Electrical signature analysis. ISO 22096:2020; Condition monitoring and diagnostics of machines — Requirements for qualification and assessment of personnel. ISO 18436-1:2014,” Geneva.
- [10] Shannon C. E., “Communication in the Presence of Noise Classic Paper,” 1949.
- [11] Pawlak M., “Zastosowanie analizy sygnałów akustycznych do wykrywania uszkodzeń wirników i łożysk w silnikach indukcyjnych,” 2011.

- [12] Siegfried Gündert, “Mel Filter Bank - pyfilterbank.” Accessed: Jun. 21, 2025. [Online]. Available: <https://siggigue.github.io/pyfilterbank/melbank.html>
- [13] M. P. Davis S. B., “Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences,” 1980.
- [14] F. Akbalık, A. Yıldız, Ö. F. Ertuğrul, and H. Zan, “Engine Fault Detection by Sound Analysis and Machine Learning,” *Applied Sciences (Switzerland)*, vol. 14, no. 15, Aug. 2024, doi: 10.3390/app14156532.
- [15] F. Rustam *et al.*, “Railway Track Fault Detection Using Selective MFCC Features from Acoustic Data,” *Sensors (Basel)*, vol. 23, no. 16, Aug. 2023, doi: 10.3390/s23167018.
- [16] Centralny Instytut Ochrony Pracy – Państwowy Instytut Badawczy, “Poziom dźwięku A hałasu w ciągu 8 godzin,” https://m.ciop.pl/CIOPPortalWAR/appmanager/ciop/mobi?_nfpb=true&_pageLabel=P424002461497875434734&html_tresc_root_id=300007402&html_tresc_id=300007392, Zasady oceny narażenia i metody badań, Dostęp: 02.06.2025.
- [17] T. Marwala and C. B. Vilakazi, “Computational intelligence for condition monitoring,” 2007.
- [18] C. E. Shannon, “A Mathematical Theory of Communication,” 1948.
- [19] K. J. Piczak, “Klasyfikacja dźwięku za pomocą splotowych sieci neuronowych,” 2018.
- [20] Z. Alom, T. M. Taha, C. Yakopcic, S. Westberg, P. Sidike, M. S. Nasrin, B. C. Van Esesn, A. A. S. Awwal, and V. K. Asari, “The History Began from AlexNet: A Comprehensive Survey on Deep Learning Approaches,” 2018.
- [21] R. Y. Yang and R. Rai, “Machine auscultation: enabling machine diagnostics using convolutional neural networks and large-scale machine audio data,” *Adv Manuf*, vol. 7, no. 2, pp. 174–187, Jun. 2019, doi: 10.1007/s40436-019-00254-5.
- [22] NVIDIA, “Convolutional Neural Network (CNN),” <https://www.nvidia.com/en-us/glossary/convolutional-neural-network/>, Dostęp: 04.06.2025.

- [23] P. Nayak *et al.*, “2D-convolutional neural network based fault detection and classification of transmission lines using scalogram images,” *Heliyon*, vol. 10, no. 19, Oct. 2024, doi: 10.1016/j.heliyon.2024.e38947.
- [24] J. G. Lee, K. S. Kim, and J. H. Lee, “Sound-Based Unsupervised Fault Diagnosis of Industrial Equipment Considering Environmental Noise,” *Sensors*, vol. 24, no. 22, Nov. 2024, doi: 10.3390/s24227319.
- [25] C., K. S., & E. L. Celebioglu, “Smartphone-based Bearing Fault Diagnosis in Rotating Machinery using Audio Data and 1D Convolutional Neural Networks,” 2024.
- [26] R. Kumar, P. Kumar, G. Vashishtha, S. Chauhan, R. Zimroz, S. Kumar, R. Kumar, M. K. Gupta, and N. S. Ross, “Fault Identification of Direct-Shift Gearbox Using Variational Mode Decomposition and Convolutional Neural Network,” Multidisciplinary Digital Publishing Institute (MDPI), Jul. 2024. doi: 10.3390/machines12070428.
- [27] Z. Alyafawi, “A NEW VOLTERRA NEURAL NETWORK LAYER LIBRARY USING TENSORFLOW,” 2023.
- [28] D. M. W. Powers, “Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation,” *Journal of Machine Learning Technologies*, 2011.

Spis rysunków

Rys. 1. Bank filtrów trójkątnych w skali mel używany przy obliczaniu współczynników MFCC [12].....	20
Rys. 2. Przykładowa architektura konwolucyjnej sieci neuronowej (CNN) [23].....	31
Rys. 3. Przykładowe powiadomienie wysłane przez system za pomocą API Pushbullet	74
Rys. 4. Główny panel administratora systemu diagnostycznego.....	76
Rys. 5. Główny panel pracownika systemu diagnostycznego	77

Rys. 6. Widok panelu administracyjnego Django używanego w systemie diagnostycznym.....	78
Rys. 7. Interfejs monitoringu akustycznego w czasie rzeczywistym - sekcja 1 z 2.....	80
Rys. 8. Interfejs monitoringu akustycznego w czasie rzeczywistym - sekcja 2 z 2.....	81
Rys. 9. Formularz uruchamiania procesu treningu modelu.....	82
Rys. 10. Widok postępu treningu z wykresami metryk.....	83
Rys. 11. Zakończenie procesu treningu i zapis wyników	84
Rys. 12. Komputer HP T630 oraz mikrofon Røde NT-USB Mini tworzące kompletny system.....	90
Rys. 13. Schemat ideowy układu pomiarowego	91
Rys. 14. Układ pomiarowy wykorzystany do stworzenia bazy nagrań oraz testów	92
Rys. 15. Przykładowe spektrogramy sygnałów akustycznych poszczególnych klas.....	93
Rys. 16. Wpływ wybranych ustawień monitoringu na skuteczność klasyfikacji.....	141
Rys. 17. Wpływ poszczególnych parametrów na końcowy wynik jakości modelu (<i>final_score</i>)	144

Spis tabel

Tab. 1. Struktura macierzy pomyłek dla klasyfikacji binarnej.....	63
Tab. 2. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika uczenia.....	99
Tab. 3. Wskaźniki złożone dla różnych wartości współczynnika uczenia.....	100
Tab. 4. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby epok	101
Tab. 5. Wskaźniki złożone dla różnych wartości liczby epok	101
Tab. 6. Zestawienie metryk klasyfikacyjnych dla różnych wartości rozmiaru batcha.....	102
Tab. 7. Wskaźniki złożone dla różnych wartości rozmiaru batcha	103

Tab. 8. Zestawienie metryk klasyfikacyjnych dla różnych liczby warstw konwolucyjnych i poolingowych.....	105
Tab. 9. Wskaźniki złożone dla różnych liczby warstw konwolucyjnych i poolingowych	106
Tab. 10. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby neuronów w warstwie gęstej.....	107
Tab. 11. Wskaźniki złożone dla różnych wartości liczby neuronów w warstwie gęstej	107
Tab. 12. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika dropout w warstwach konwolucyjnych	108
Tab. 13. Wskaźniki złożone dla różnych wartości współczynnika dropout w warstwach konwolucyjnych.....	109
Tab. 14. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika dropout w warstwach gęstych.....	109
Tab. 15. Wskaźniki złożone dla różnych wartości współczynnika dropout w warstwach gęstych	110
Tab. 16. Zestawienie metryk klasyfikacyjnych dla różnych wartości współczynnika regularyzacji L2	111
Tab. 17. Wskaźniki złożone dla różnych wartości współczynnika regularyzacji L2 ..	111
Tab. 18. Zestawienie metryk klasyfikacyjnych dla różnych typów warstwy agregującej	112
Tab. 19. Wskaźniki złożone dla różnych typów warstwy agregującej	112
Tab. 20. Zestawienie metryk klasyfikacyjnych dla różnych funkcji aktywacji.....	113
Tab. 21. Wskaźniki złożone dla różnych funkcji aktywacji	114
Tab. 22. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby współczynników MFCC	115
Tab. 23. Wskaźniki złożone dla różnych wartości liczby współczynników MFCC ...	115
Tab. 24. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości MFCC	116

Tab. 25. Wskaźniki złożone dla różnych wartości długości MFCC	117
Tab. 26. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości FFT ...	118
Tab. 27. Wskaźniki złożone dla różnych wartości długości FFT.....	118
Tab. 28. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości skoku	119
Tab. 29. Wskaźniki złożone dla różnych wartości długości skoku.....	120
Tab. 30. Zestawienie metryk klasyfikacyjnych dla różnych wartości poziomu dodawanego szumu	121
Tab. 31. Wskaźniki złożone dla różnych wartości poziomu dodawanego szumu	121
Tab. 32. Zestawienie metryk klasyfikacyjnych dla różnych wartości przesunięcia wysokości dźwięku.....	122
Tab. 33. Wskaźniki złożone dla różnych wartości przesunięcia wysokości dźwięku..	123
Tab. 34. Zestawienie metryk klasyfikacyjnych dla różnych wartości rozciągania czasowego dźwięku	124
Tab. 35. Wskaźniki złożone dla różnych wartości rozciągania czasowego dźwięku ..	124
Tab. 36. Zestawienie metryk klasyfikacyjnych dla różnych wartości wzmocnienia głośności.....	125
Tab. 37. Wskaźniki złożone dla różnych wartości wzmocnienia głośności.....	126
Tab. 38. Zestawienie metryk klasyfikacyjnych dla różnych wartości liczby augmentacji na próbkę	127
Tab. 39. Wskaźniki złożone dla różnych wartości liczby augmentacji na próbkę.....	127
Tab. 40. Zestawienie metryk klasyfikacyjnych dla różnych wartości długości próbki	128
Tab. 41. Wskaźniki złożone dla różnych wartości długości próbki	128
Tab. 42. Zestawienie metryk klasyfikacyjnych dla różnych proporcji podziału zbioru danych.....	129
Tab. 43. Wskaźniki złożone dla różnych proporcji podziału zbioru danych	130
Tab. 44. Zestawienie najlepszych wyników dla wszystkich testowanych parametrów modelu	131
Tab. 45. Zestawienie wyników końcowej konfiguracji modelu.....	132

Tab. 46. Zestawienie skuteczności klasyfikacji dla różnych długości próbki sygnału	135
Tab. 47. Zestawienie skuteczności klasyfikacji dla różnych trybów klasyfikacji	136
Tab. 48. Zestawienie skuteczności klasyfikacji dla różnych progów decyzyjnych.....	137
Tab. 49. Zestawienie skuteczności klasyfikacji dla różnych częstotliwości próbkowania sygnału audio	138